HUMAN-IN-THE-LOOP MACHINE LEARNING

Eric Nalisnick
Johns Hopkins University
Fall 2026

Contents

1 Background **3**

1.1 Supervised Learning . . . 3
1.2 Maximum Likelihood Estimation . . . 3
1.3 Reinforcement Learning . . . 5
1.4 Model Calibration . . . 8

2 Supervised Learning from Human-Generated Labels **12**

2.1 Pooled: Multinomial Model . . . 13
2.2 Unpooled: Dawid-Skene Model . . . 16
2.3 Jointly Learning Ground-Truth and a Predictive Model . . . 18
2.4 Active Learning . . . 21

3 Imitation Learning **28**

3.1 Behavior Cloning . . . 28
3.2 Policy Learning via an Interactive Demonstrator . . . 30
3.3 Distribution Matching . . . 30
3.4 Inverse Reinforcement Learning . . . 33
3.5 Reinforcement Learning with Human Feedback . . . 36

4 Human-AI Collaboration **39**

4.1 Design Patterns . . . 39
4.2 Combining Predictions from Humans and Models . . . 40
4.3 AI-Assisted Decision Making . . . 41
4.3.1 Recalibrating to Improve Human Performance . . . 42
4.3.2 Assisting Humans with Prediction Sets . . . 43
4.3.3 Indistinguishability: When is Human Effort Helpful? . . . 45
4.4 Learning to Reject . . . 46
4.4.1 Chow's Rule: Rejection via Calibrated Confidence Scores . . . 47
4.4.2 Rejection via Optimizing Selective Risk . . . 48
4.4.3 Consistent Surrogate Loss for Rejection Learning . . . 48
4.5 Learning to Defer to an Expert . . . 49
4.5.1 Bayesian Decision Theory for Deferral . . . 49
4.5.2 Consistent Surrogate Loss for L2D . . . 50
4.5.3 L2D to Multiple Experts . . . 51
4.6 Sequential Learning with Rejection or Deferral . . . 52
4.7 Model Misspecification and Complementing Human Behavior . . . 53

1 Background

We will consider both supervised learning and reinforcement learning and hence give brief introductions below.

1.1 Supervised Learning

The goal of supervised learning is to map feature vectors, the independent variable, to labels or responses, the dependent variable. Let $\mathcal{X}$ denote the feature space, and let $\mathcal{Y}$ denote the label / response space. $\mathbf{x}_n \in \mathcal{X}$ denotes a feature vector, and $\mathrm{y}_n \in \mathcal{Y}$ denotes the associated response / label defined by $\mathcal{Y}$. The N-element training sample is then $\mathcal{D} = \{\boldsymbol{x}_n, y_n\}_{n=1}^N$. We will almost always work with a probabilistic formulation, using the negative log-likelihood (NLL) as the training objective. Given a paramerization $f(\mathbf{x}; \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ are the model parameters, we can follow the framework of generalized linear models, letting $\mathbb{E}\mathrm{y}|\mathbf{x} = f(\mathbf{x}; \boldsymbol{\theta})$. The NLL is then:

$$\begin{aligned} \ell(\boldsymbol{\theta}; \mathcal{D}) &= -\log \left\{ \prod_{n=1}^{N} p(\mathrm{y}_n | \mathbf{x}_n, \boldsymbol{\theta}) \right\} \\ &= \sum_{n=1}^{N} -\log p(\mathrm{y}_n | \mathbf{x}_n, \boldsymbol{\theta}). \end{aligned} \tag{1}$$

When $\mathcal{Y} = \mathbb{R}$, real numbers, then the supervised learning problem is usually called *regression*, and when $\mathcal{Y}$ denotes discrete variables, then the task is *classification*. It is an easy calculation to show that, for the case of regression, plugging in the normal distribution to the NLL above yields the squared-error loss function. Similarly, for classification, plugging in the categorical distribution results in the cross-entropy loss function.

1.2 Maximum Likelihood Estimation

The NLL above is the training objective for the general procedure of *maximum likelihood estimation*. It can be derived from first principles as follows. Assume that $p^*(\mathbf{y})$ denotes the true data-generating distribution and $p(\mathbf{y}; \boldsymbol{\theta})$ denotes the model with parameters $\boldsymbol{\theta}$. The model can be fit to data by minimizing a statistical divergence between these two distributions. MLE corresponds to using the *Kullback–Leibler divergence* (KLD): $\mathbb{KLD}[p||q] = \int \log\{p/q\}dp$:

$$\begin{aligned} \mathbb{KLD}\left[\, p^*(\mathbf{y}) \;||\; p(\mathbf{y}; \boldsymbol{\theta}) \,\right] &= \mathbb{E}_{p^*}\left[\log \frac{p^*(\mathbf{y})}{p(\mathbf{y}; \boldsymbol{\theta})}\right] \\ &= \mathbb{E}_{p^*}\left[-\log p(\mathbf{y}; \boldsymbol{\theta})\right] \;-\; \mathbb{H}\left[p^*(\mathbf{y})\right] \\ &= \mathbb{E}_{p^*}\left[-\log p\left(\mathbf{y}; \boldsymbol{\theta}\right)\right] + \;\text{const.} \end{aligned} \tag{2}$$

where $\mathbb{H}[p] = \int(-\log p)dp$ denotes the entropy of the distribution p. The entropy is a constant with respect to (w.r.t.) $\boldsymbol{\theta}$ and thus it will drop out when taking a derivative w.r.t. $\boldsymbol{\theta}$. Of course, we do not usually know $p^*(\mathbf{y})$ in practice and instead have knowledge of it through samples: $\mathcal{Y} = \{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_N\}$, $y_n \sim p^*(\mathbf{y})$. We can use these samples to form a

Monte Carlo approximation of the expectation above:

$$\mathbb{E}_{p^*}\left[-\log p(\mathbf{y};\boldsymbol{\theta})\right] \;\approx\; \frac{1}{N}\sum_{n=1}^{N} -\log p(\boldsymbol{y}_n;\boldsymbol{\theta}), \tag{3}$$

where the approximation becomes exact as $N \to \infty$.

Example for the Categorical Distribution: Now we work through an example of maximum likelihood estimation when the data takes the form of categories or labels, i.e. $\boldsymbol{y}_n = [0,0,1,0]$, which means that there are $K = 4$ possible categories and the nth observation was of the 3rd category. The canonical model for categorical data of this form is the multinoulli (a.k.a. categorical) distribution:

$$p(\boldsymbol{y}_n;\boldsymbol{\pi}) \;=\; \texttt{Categorical}(\boldsymbol{y}_n;\boldsymbol{\pi}) \;=\; \prod_{k=1}^{K} \pi_k^{y_{n,k}},$$

where $\boldsymbol{\pi} = [\pi_1, \ldots, \pi_K]$ is the vector of parameters such that $\pi_k \in [0,1]$ and $\sum_{k=1}^{K} \pi_k = 1$. We can now compute the *maximum likelihood estimate* (MLE) by differentiating the likelihood w.r.t. $\hat{\pi}_d$ while obeying the constraint that the parameters sum to one. First, starting with the likelihood alone:

$$\begin{aligned} \ell\left(\boldsymbol{\pi};\mathcal{Y}\right) \;&=\; \frac{1}{N}\sum_{n=1}^{N} -\log p(\boldsymbol{y}_n;\boldsymbol{\pi}) \;=\; \frac{1}{N}\sum_{n=1}^{N} -\log\left\{\prod_{k=1}^{K} \pi_k^{y_{n,k}}\right\} \\ &=\; \frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K} -\log \pi_k^{y_{n,k}} \;=\; -\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K} y_{n,k}\log\pi_k. \end{aligned} \tag{4}$$

Now we introduce a lagrange multiplier $\lambda \in \mathbb{R}$ to ensure the constraint:

$$\tilde{\ell}\left(\boldsymbol{\pi},\lambda;\mathcal{Y}\right) \;=\; -\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K} y_{n,k}\log\pi_k \;+\; \lambda\cdot\left(\sum_{k=1}^{K}\pi_k - 1\right). \tag{5}$$

Taking the derivative first w.r.t. a single parameter π_d, we have:

$$\begin{aligned} \frac{\partial}{\partial\pi_d}\tilde{\ell}\left(\boldsymbol{\pi},\lambda;\mathcal{Y}\right) \;&=\; \frac{\partial}{\partial\pi_d}\left[-\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K} y_{n,k}\log\pi_k\right] \;+\; \lambda\cdot\frac{\partial}{\partial\pi_d}\left(\sum_{k=1}^{K}\pi_k - 1\right) \\ &=\; -\frac{1}{N}\sum_{n=1}^{N}\frac{y_{n,d}}{\pi_d} \;+\; \lambda. \end{aligned} \tag{6}$$

Setting the derivative to zero and rearranging, we have:

$$0 \;=\; -\frac{1}{N}\sum_{n=1}^{N}\frac{y_{n,d}}{\pi_d} \;+\; \lambda \quad\Longrightarrow\quad \pi_d \;=\; \frac{1}{\lambda}\frac{1}{N}\sum_{n=1}^{N} y_{n,d}. \tag{7}$$

Now we have to solve for λ:

$$\begin{aligned}\frac{\partial}{\partial\lambda}\tilde{\ell}\left(\boldsymbol{\pi},\lambda;\mathcal{Y}\right) &= \frac{\partial}{\partial\lambda}\left[-\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K}y_{n,k}\log\pi_k\right] + \frac{\partial}{\partial\lambda}\left[\lambda\cdot\left(\sum_{k=1}^{K}\pi_k-1\right)\right] \\ &= \sum_{k=1}^{K}\pi_k - 1.\end{aligned} \tag{8}$$

Now setting to zero and plugging in the expression for π_d from Equation 7:

$$\begin{aligned}1 &= \sum_{k=1}^{K}\pi_k = \sum_{k=1}^{K}\left(\frac{1}{\lambda}\frac{1}{N}\sum_{n=1}^{N}y_{n,k}\right) \\ &= \frac{1}{\lambda}\frac{1}{N}\sum_{k=1}^{K}\sum_{n=1}^{N}y_{n,k}.\end{aligned} \tag{9}$$

Solving for λ we have:

$$\lambda = \frac{1}{N}\sum_{k=1}^{K}\sum_{n=1}^{N}y_{n,k}.$$

For the final step, we plug in the expression for λ above into Equation 7:

$$\begin{aligned}\widehat{\pi}_d &= \frac{1}{\lambda}\frac{1}{N}\sum_{n=1}^{N}y_{n,d} \\ &= \frac{1}{\frac{1}{N}\sum_{k=1}^{K}\sum_{n=1}^{N}y_{n,k}}\frac{1}{N}\sum_{n=1}^{N}y_{n,d} = \frac{\sum_{n=1}^{N}y_{n,d}}{\sum_{k=1}^{K}\sum_{n=1}^{N}y_{n,k}} \\ &= \frac{\sum_{n=1}^{N}y_{n,d}}{N}.\end{aligned} \tag{10}$$

This estimator is simply the number of times the dth category appears in the training data divided by the number of total observations.

1.3 Reinforcement Learning

Reinforcement learning (RL) considers sequential decision-making problems in which models take actions so that their reward is maximized in an environment. Let $s \in \mathcal{S}$ be the space of states, and let $a \in \mathcal{A}$ be the space of actions. RL usually consider the underlying generative model to be a *Markov Decision Process* defined by

$$\mathrm{s}_{t+1} \sim \mathbb{P}(\mathrm{s}_{t+1}|\mathrm{s}_t,\mathrm{a}_t), \qquad r_t = \mathcal{R}(\mathrm{s}_t;\mathrm{a}_{t-1},,\mathrm{s}_{t-1})$$

where $\mathbb{P}(\mathrm{s}_{t+1}|\mathrm{s}_t,\mathrm{a}_t)$ is the *transition probability* of moving from state s_t to s_{t+1} by taking action a_t. Moreover, $\mathcal{R}(\mathrm{s}_t;\mathrm{a}_{t-1},,\mathrm{s}_{t-1})$ is a *reward function* that returns the reward of transitioning to s_t from s_{t-1} using action a_{t-1}. The reward not need be a function of the previous state and action, e.g. $r_t = \mathcal{R}(\mathrm{s}_t)$. The goal of RL is to obtain a policy $\pi : \mathcal{S}\times\mathcal{A} \to [0,1]$ that determines the appropriate action to take (such that long-term reward is maximized) when in a given state: $\pi(\mathrm{a}|\mathrm{s}) = \Pr(\mathrm{a}_t = a|\mathrm{s}_t = s_t)$. The policy is determined by parameters $\boldsymbol{\theta}$, so we write $\pi_{\boldsymbol{\theta}}(\mathrm{a}|\mathrm{s})$. Collecting a T-length (with T possibly being infinite)

series of state-action pairs

$$\{(s_t, a_t)\}_{t=1}^{T} \quad \text{where} \quad a_t \sim \pi_{\boldsymbol{\theta}}(\mathrm{a}|s_t), \quad s_{t+1} \sim \mathbb{P}(\mathrm{s}_{t+1}|s_t, a_t)$$

is called *rolling out a policy*, or a *rollout*.

Optimization Objective The goal is to learn a policy that maximizes *return*, which is a function of a state s_t:

$$G\left(\mathrm{s}_t; \{\mathrm{a}_t, \mathrm{a}_{t+1}, \ldots\}\right) \;=\; \sum_{t'=0}^{\infty} \gamma^{t'} \cdot r_{t+t'}$$

where $\gamma \in [0, 1)$ is a *discount factor* that emphasizes near-term rewards. The return compute the future rewards $r_{t+t'}$ are obtained by following actions $\{\mathrm{a}_t, \mathrm{a}_{t+1}, \ldots\}$ from an initial state s_t. Given the definition of return, we can then quantify the utility of a given state via the *value function*. It quantifies the expected return from a given state:

$$\begin{aligned} V^{\pi}(s) \;&=\; \mathbb{E}_{\pi}\left[\, G\left(\mathrm{s}_t; \{\mathrm{a}_t, \mathrm{a}_{t+1}, \ldots\}\right) \;|\; \mathrm{s}_t = s \,\right] \\ &=\; \sum_{a \in \mathcal{A}} \pi(a|s) \cdot \mathbb{E}_{\pi}\left[\, G\left(\mathrm{s}_t; \{\mathrm{a}_t, \mathrm{a}_{t+1}, \ldots\}\right) \;|\; \mathrm{s}_t = s, \mathrm{a}_t = a \,\right] \end{aligned}$$

where the expectation is taken over rollouts of π. The final RL optimization objective, known as the *Bellman equation*, is to find the policy whose stationary distribution over states places high probability on 'valuable' states:

$$\begin{aligned} \mathcal{J}(\boldsymbol{\theta}) \;&=\; \sum_{s \in \mathcal{S}} d^{\pi(\boldsymbol{\theta})}(s) \cdot V^{\pi(\boldsymbol{\theta})}(s) \\ &=\; \sum_{s \in \mathcal{S}} d^{\pi(\boldsymbol{\theta})}(s) \sum_{a \in \mathcal{A}} \pi_{\boldsymbol{\theta}}(a|s) \cdot \underbrace{\mathbb{E}_{\pi(\boldsymbol{\theta})}\left[\, G\left(\mathrm{s}_t; \{\mathrm{a}_t, \mathrm{a}_{t+1}, \ldots\}\right) \;|\; \mathrm{s}_t = s, \mathrm{a}_t = a \,\right]}_{Q^{\pi(\boldsymbol{\theta})}(s,a)} \\ &= \sum_{s \in \mathcal{S}} d^{\pi(\boldsymbol{\theta})}(s) \sum_{a \in \mathcal{A}} \pi_{\boldsymbol{\theta}}(a|s) \cdot Q^{\pi(\boldsymbol{\theta})}(s, a) \end{aligned} \tag{11}$$

where $d^{\pi(\boldsymbol{\theta})}$ is the *stationary distribution* under policy $\pi_{\boldsymbol{\theta}}$, meaning it is the marginal distribution over state visitations. The expectation within the value function has its own name, the *Q-function*, that quantifies the value of a state-action pair. I write $\pi(\boldsymbol{\theta})$ in the superscripts to emphasize that these quantities are dependent upon the parameters that are being optimized. Ideally, we want to find a policy such that

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta} \in \Theta} \mathcal{J}(\boldsymbol{\theta}).$$

There are many approaches to this optimization problem, of course, with crucial decisions needed about how to calculate various statistics of the states visited during finite-sample rollouts. We will mostly ignore these details, referring the reader to any standard text on RL if needed.

Entropy-Regularized RL Often MDPs are non-smooth and in need of regularization. Thus entropy-regularized MDPs are often considered, which use the modified reward func-

tion:

$$\mathcal{R}_{\mathbb{H}}(\mathrm{s}_t; \mathrm{a}_{t-1}, \mathrm{s}_{t-1}, \lambda) \;=\; \mathcal{R}(\mathrm{s}_t; \mathrm{a}_{t-1}, , \mathrm{s}_{t-1}) \;+\; \lambda \cdot \mathbb{H}\left[\pi(\mathrm{a}_t|\mathrm{s}_t)\right]$$

where $\lambda \in \mathbb{R}^+$ is a weighting constant and $\mathbb{H}\left[\pi(\mathrm{a}_t|\mathrm{s}_t)\right]$ is the entropy of the distribution over actions at the current state. Hence the modified reward encourages policies that not only obtain high reward but also lead to states that are not 'dead ends,' meaning that there are relatively many actions with non-negligible probability with which to transition out of the current state. Under entropy-regularization, the optimal policy has a nice (yet self-referential) form:

$$\pi^*(\mathrm{a}|s) \;=\; \exp\left\{\lambda^{-1}\left(Q^{\pi^*}(s,\mathrm{a}) - V^{\pi^*}(s)\right)\right\}. \tag{12}$$

We see that the optimal policy is the exponentiated difference between the Q- and value functions.

Advantage Function For a reference policy π, we may at times be curious what could be gained by taking an alternative action, possibly sampled from a difference policy: $\mathrm{a} \sim \pi^{'}$. This can be quantified by the *advantage function*, which is defined as:

$$A^{\pi}(s,a) \;=\; Q^{\pi}(s,a) \;-\; V^{\pi}(s).$$

We see it is the difference between the Q-function, evaluated at the current state and the action under consideration, and the value function defined from the reference policy π. Comparing this equation to Equation 12, we see that the optimal policy under entropy regularization is just the exponentiated advantage function:

$$\begin{aligned} \pi^*(\mathrm{a}|s) \;&=\; \exp\left\{\lambda^{-1}\left(Q^{\pi^*}(s,\mathrm{a}) - V^{\pi^*}(s)\right)\right\} \\ &=\; \exp\left\{\lambda^{-1} \cdot A^{\pi^*}(s,a)\right\} \end{aligned} \tag{13}$$

where λ is again the weight on the entropy term.

Performance Difference Lemma Lastly, the advantage function also has a useful theoretical property, which we show below in the form of the *Performance Difference Lemma* (PDL) (Kakade and Langford, 2002).

Theorem 1.1. Performance Difference Lemma: For two policies π and π', the difference between their value functions is equivalently expressed in terms of the advantage function:

$$V^{\pi^{'}}(s) - V^{\pi}(s) \;=\; \frac{1}{1-\gamma}\mathbb{E}_{\mathrm{s}\sim d(\mathrm{s};\pi')}\mathbb{E}_{\mathrm{a}\sim\pi'}\left[A^{\pi}(\mathrm{s},\mathrm{a})\right] \tag{14}$$

where γ is the discount, $d(\mathrm{s};\pi')$ is the stationary distribution over states from following π', and $A^{\pi}(\mathrm{s},\mathrm{a})$ is the advantage function under π.

See pages 6 and 7 of these notes for a proof: `https://nanjiang.cs.illinois.edu/files/cs542f22/note1.pdf`. An alternative, more explicit proof can be found here: `https:`

//wensun.github.io/CS4789_data/PDL.pdf. The intuition behind the PDL is that the difference in the value of two policies can be expressed as the advantage of one policy under rollouts (i.e the expectation) of the other. This will be a useful tool when comparing, for example, a policy found through an approximation vs an optimal one.

1.4 Model Calibration

Consider a probabilistic classifier $f : \mathcal{X} \mapsto \Delta^K$, where $\mathcal{X}$ is a feature space and Δ^K is a space of probability measures on the label space $\mathcal{Y}$. One can think of Δ^K as a point on the simplex, an output of a softmax function. A fundamental question to ask about f is: *to what extent does f capture the true underlying predictive distribution?* That is, we are not simply concerned with the accuracy of f but rather its ability to capture the full predictive distribution, i.e. $p(\mathrm{y}|f(\mathbf{x})) \approx \mathbb{P}(\mathrm{y}|\mathbf{x})$, where $p(\mathrm{y}|f(\mathbf{x}))$ denotes a categorical distribution over the labels as defined by the model and $\mathbb{P}(\mathrm{y}|\mathbf{x})$ is the true generative process of the data. If this model is a good approximation for the underlying generative process, we call the model *well calibrated*. I'll define this more formally below.

Calibration and Decision Making But first, why should we care if a model is well-calibrated? One major benefit of calibration is that it allows you to treat the $f(\mathbf{x})$ values as confidence scores. If the model is predicting class #1 with $f_1(\mathbf{x}) = 90\%$, then that level of confidence can be propagated into downstream decision making. In some settings, 90% confidence is more than enough to take some action, but perhaps in a medical setting, we require 99% or otherwise need to collect more data by running more tests. We can make this property more rigorous through the lens of *Bayesian decision theory*. Consider some loss function $L(\mathrm{a}(\mathbf{x}), \mathrm{y})$, which computes the loss incurred when taking action / decision $\mathrm{a}(\cdot)$ for input features $\mathbf{x}$ when the true class is y. We then say that the *risk* for the decision rule $\mathrm{a}(\cdot)$ is:

$$\mathcal{R}(\mathrm{a}) \;=\; \mathbb{E}_{\mathbb{P}(\mathbf{x})}\mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathbf{x})}\left[L(\mathrm{a}(\mathbf{x}), \mathrm{y})\right] \;=\; \int_{\mathbf{x}} \mathbb{P}(\mathbf{x}) \int_{\mathbf{y}} \mathbb{P}(\mathrm{y}|\mathbf{x}) \cdot L(\mathrm{a}(\mathbf{x}), \mathrm{y})\; d\mathrm{y}\; d\mathbf{x}.$$

Classification can be formulated in the above framework by setting $L(\mathrm{a}(\mathbf{x}), \mathrm{y}) = \mathbb{I}[a(\mathbf{x}) \neq \mathrm{y}]$, where the 'action' is choosing a particular class as the final prediction.

Of course, we want to take actions that minimize this risk, and thus the Bayes optimal decision is:

$$a^* = \arg\min_{\mathrm{a} \in \mathcal{A}} \; \mathcal{R}(\mathrm{a}),$$

where $\mathcal{A}$ is the space of all possible actions or decision rules. However, in the real world, we don't have access to $\mathbb{P}(\mathrm{y}|\mathbf{x})$—or else, why would we have to train a model to approximate it?—and so we have to instead work with the model-based risk:

$$\widetilde{\mathcal{R}}(\mathrm{a}) \;=\; \mathbb{E}_{\mathbb{P}(\mathbf{x})}\mathbb{E}_{p(\mathrm{y}|f(\mathbf{x}))}\left[L(\mathrm{a}(\mathbf{x}), \mathrm{y})\right] \;=\; \int_{\mathbf{x}} \mathbb{P}(\mathbf{x}) \int_{\mathbf{y}} p(\mathrm{y}|f(\mathbf{x})) \cdot L(\mathrm{a}(\mathbf{x}), \mathrm{y})\; d\mathrm{y}\; d\mathbf{x}.$$

Yet, if we should be so lucky that our model is *perfectly* calibrated, i.e. $p(\mathrm{y}|f(\mathbf{x})) = \mathbb{P}(\mathrm{y}|\mathbf{x})$,

then the decisions taken using the model will be Bayes optimal:

$$a^* \;=\; \arg\min_{\mathrm{a}\in\mathcal{A}} \; \mathcal{R}(\mathrm{a}) \;=\; \arg\min_{\mathrm{a}\in\mathcal{A}} \; \widetilde{\mathcal{R}}(\mathrm{a}).$$

This is the core benefit of (perfect) calibration: decisions made based on our model will be principled (Grünwald, 2016; Noarov and Roth, 2024). Otherwise, any calibration error will propagate, leading to sub-optimal actions in downstream decision making. As we will see later, having a well-calibrated model is especially valuable when we want to do joint decision making, such as when a model and human are working together (Chow, 1957b).

Forms of Calibration We now define calibration more precisely. Starting with the aforementioned ideal case of *perfect* calibration, we call a classifier ***perfectly* calibrated** (or *distribution calibrated* or *canonically calibrated*) if it perfectly matches the probability assigned by the true generative process:

$$\mathbb{P}(\mathrm{y} = y | f_y(\mathbf{x})) \;=\; f_y(\mathbf{x}), \quad \forall\, \mathbf{x} \in \mathcal{X}, \;\; \forall\, y \in \mathcal{Y}.$$

Unfortunately, it is not only hard to achieve perfect calibration, it is even hard to *check* if a classifier is perfectly calibrated (except in the case of binary classification). This is because we usually only see one label per feature vector, and evaluating perfect calibration would require multiple labels (exponentially many in the number of classes), in order to form a histogram approximation of $\mathbb{P}(\mathrm{y}|\mathbf{x})$.

In machine learning, it is instead common to check for ***confidence* calibration**, which instead checks for calibration in the top-ranked class:

$$\mathbb{P}\left(\mathrm{y} = \arg\max_{k\in[1,|\mathcal{Y}|]} f_k(\mathbf{x}) \;\middle|\; \max_{k\in[1,|\mathcal{Y}|]} f_k(\mathbf{x})\right) \;=\; \max_{k\in[1,|\mathcal{Y}|]} f_k(\mathbf{x}), \quad \forall\, \mathbf{x} \in \mathcal{X},$$

where $\arg\max_{k\in[1,|\mathcal{Y}|]} f_k(\mathbf{x})$ denotes the modal prediction from the model and $\max_{k\in[1,|\mathcal{Y}|]} f(\mathbf{x})$ denotes the corresponding confidence score. This is a much more tractable quantity to evaluate since we have reduced the scope of concern from matching the probability of all classes to just the most likely one (under the model). This will be tractable to quantify via a *reliability diagram*. These are plots that report, for held-out data, the accuracy (y-axis) for all points that evaluate to approximately the same confidence score, $\max_{k\in[1,|\mathcal{Y}|]} f_k(\mathbf{x})$. The intuition can be had from weather forecasting: we collect all the past instances for which the weatherperson said that there would be an $X\%$ chance of some weather event. We then look to see, of those cases, how many times the event actually occurred. For a good forecaster, their accuracy on all points at a stated confidence level should match that confidence score. In practice, some binning of confidence scores is required to get reliable statistics (e.g. check the accuracy for all forecasts with confidences between 27% and 33%, hoping for roughly 30% accuracy). A perfectly confidence calibrated model will have a reliability diagram with results that lie perfectly on the $y = x$ line. Of course, some degree of mis-calibration usually

exists, and we can summarize this error via the *expected calibration error* (ECE):

$$\begin{aligned} \text{ECE}(f; \mathbb{P}(\text{y}|\mathbf{x})) &= \int_{\mathbf{x}} P(f_{\max}(\mathbf{x})) \cdot \left| f_{\max}(\mathbf{x}) - \mathbb{E}_{\mathbb{P}(\text{y}|\mathbf{x})}[\text{y} \mid f_{\max}(\mathbf{x})] \right| \, d\mathbf{x} \\ &\approx \sum_{v \in \mathtt{B}(f)} \hat{P}(f_{\max}(\mathbf{x}) = v) \cdot \left| v - \hat{\mathbb{E}}_{\mathbb{P}(\text{y}|\mathbf{x})}[\text{y} \mid f_{\max}(\mathbf{x}) = v] \right|, \end{aligned} \tag{15}$$

where $f_{\max}(\mathbf{x}) = \max_{k \in [1, |\mathcal{Y}|]} f_k(\mathbf{x})$. The second line represents the discrete approximation used in practice, with $\mathtt{B}(f)$ denoting a binning of f's output range, $\hat{P}(f_{\max}(\mathbf{x}) = v)$ denoting a histogram estimate of the density, and $\hat{\mathbb{E}}_{\mathbb{P}(\text{y}|\mathbf{x})}$ denoting a Monte Carlo approximation of the expectation. The ECE can be thought of as measuring how far the model is from achieving a perfect $y = x$ reliability diagram.

Confidence calibration can be too blunt at times. Again let's consider the weather forecasting example. Confidence calibration is a statement about reliability over all classes, but you may want to assess how good the weather forecaster is at predicting particular weather events. In other words, if the forecaster is predicting there will be a hurricane with 90% confidence, we may want to check all historical cases in which the forecaster predicted hurricanes with 90% confidence, not *any* weather event with 90% confidence. This calibration variant—where both the confidence and class prediction are conditioned upon—is called ***top-label* calibration** (Gupta and Ramdas, 2022):

$$\mathbb{P}\left(\text{y} = \underset{k \in [1, |\mathcal{Y}|]}{\arg\max} f_k(\mathbf{x}) \;\middle|\; \max_{k \in [1, |\mathcal{Y}|]} f_k(\mathbf{x}), \; \underset{k \in [1, |\mathcal{Y}|]}{\arg\max} f_k(\mathbf{x}) \right) = \max_{k \in [1, |\mathcal{Y}|]} f_k(\mathbf{x}), \quad \forall \, \mathbf{x} \in \mathcal{X}.$$

Measuring top-label calibration requires more data than confidence calibration since the data is now stratified by confidence level *and* class prediction. Hence, while top-label calibration gives a finer-grained notions of reliability, we must 'pay' in data when evaluating a model's degree of top-label calibration.

Generalization and Multi-Calibration For a complete generalization, we can think of calibration as always being defined with respect to a *reference class* (Hájek, 2007). This reference class is necessary as, when evaluating calibration in practice, we will need to pool over this reference class for statistical signal. Let $\Phi : \mathcal{X} \mapsto \mathcal{Z}_\Phi$ be a *grouping function*. The equivalence classes of this grouping function then define the reference classes: $[\mathbf{x}]_\Phi = \{\boldsymbol{x}' : \Phi(\boldsymbol{x}) = \Phi(\boldsymbol{x}')\}$. For a general reference class structure defined via Φ, we can define calibration with respect to this choice of reference classes as:

$$\mathbb{E}\left[\mathbb{P}(\text{y} = y|\mathbf{x}) \mid \mathbf{x} \in [\mathbf{x}]_\Phi\right] = \mathbb{P}(\text{y} = y \mid \Phi(\mathbf{x})) = f(\mathbf{x}), \quad \forall \, \mathbf{x} \in \mathcal{X}, \; \forall \, \text{y} \in \mathcal{Y}.$$

When Φ is the identity function, then this definition recovers perfect (or canonical) calibration. However, this formulation is not useful for all definitions of Φ. For example, if Φ is a constant function, e.g. $\Phi : \mathcal{X} \mapsto 1$, then $\mathbb{E}\left[\mathbb{P}(\text{y} = y|\mathbf{x}) \mid \mathbf{x} \in \mathcal{X}\right]$ is simply the *marginal* class probability, which is $1/K$ for data with a balanced number of classes. Thus, a constant model, $f : \mathcal{X} \mapsto 1/K$ will be calibrated under this definition, while in practice such a model would be meaningless. Confidence calibration can be recovered by choosing $\Phi : \mathcal{X} \mapsto f_{\max}(\mathbf{x})$ and only evaluating the class that is the mode of $f(\mathbf{x})$. This generalized definition is a gateway to considering even more general definitions with other aggregation

functions, such as the median, and has deep connections to algorithmic fairness (Höltgen and Williamson, 2023). Due to this flexibility in the underlying definition, one may worry that they are evaluating calibration for an appropriate grouping function. This is the motivation for *multi-calibration* (Hebert-Johnson et al., 2018): calibrating with respect to a series of grouping functions with increasing granularity. Yet, still the choice of grouping functions as simply uniformly random subsets of $\mathcal{X}$ allows a predictor to be approximately multi-calibrated by simply outputting the marginal class probability (the aforementioned trivial case). In the other extreme, if the grouping functions become so granular as to uniquely identify the points in $\mathcal{X}$, then it is testing for perfect (canonical) calibration. Thus, like before, some balance must be struck in the resolution for which to partition $\mathcal{X}$, but at least mutli-calibration allows us to 'hedge our bets' over multiple resolutions.

Improving Calibration If one's model is not satisfactorily calibrated, then there are several strategies to improve it. The most basic is to simply improve the model in all of the traditional ways : more data, hyper-parameter tuning, parameters, etc. If the model cannot be improved in the traditional sense or already has satisfactory accuracy, then one could attempt to improve calibration by optimizing the expected calibration error itself (Bohdal et al., 2023; Karandikar et al., 2021). Yet a simpler strategy that is commonly employed for neural networks is *temperature scaling* (Guo et al., 2017). Let $\boldsymbol{a}(\boldsymbol{x}) = [a_1(\boldsymbol{x}), \ldots, a_K(\boldsymbol{x})]$ be the logits of a neural-network-based classifier for K classes. Temperature scaling adds a temperature parameter $T \in \mathbb{R}^+$ as follows: $\texttt{softmax}\left(a_1(\boldsymbol{x})/T, \ldots, a_K(\boldsymbol{x})/T\right)$. As $T \to 0^+$, the softmax output concentrates to the dimension of the maximum logit. As $T \to \infty$, the softmax output has higher and higher entropy. Usually T is fit on a held-out set using a log-likelihood objective (while keeping all other model parameters fixed). Temperature scaling, notably, preserves the ranking of the original logit values, meaning that it does not affect the model's accuracy.

Conformal Prediction *Conformal prediction* (CP) (Shafer and Vovk, 2008) constructs a confidence interval (or set) for predictive inference. In the traditional multiclass classification setting, given a new observation $\boldsymbol{x}_{n+1}$, we wish to determine the correct associated label $\mathrm{y}_{n+1} = y^*_{n+1}$, where y^*_{n+1} denotes the true class label. CI allows us to construct a confidence set $\mathsf{C}(\boldsymbol{x}_{n+1})$ that will cover the true label with *marginal* probability $1 - \alpha$:

$$\mathbb{P}\left(y^*_{n+1} \notin \mathsf{C}(\boldsymbol{x}_{n+1})\right) \;\leq\; \alpha.$$

Denote the test statistic as $S(\boldsymbol{x}, y; \mathcal{D})$. It is known as a *non-conformity* function: a higher value of S means that $(\boldsymbol{x}, y)$ is less conforming to the distribution represented (empirically) by $\mathcal{D}$. Despite this guarantee, CP is only as good as its test statistic in practice. For instance, the marginal coverage is naively satisfied if we construct the set randomly by setting $\mathsf{C}(\boldsymbol{x}) = \mathcal{Y}$ with probability $1 - \alpha$ and returning the empty set otherwise.

CI is implemented by calculating the non-conformity function on a validation set and computing the empirical $1 - \alpha$ quantile (with a finite sample correction). At test time, elements are added to the set until the non-conformity function passes the previously-computed quantile. For multi-class classification, a common way to define the aforementioned non-conformity score is via the model's softmax scores, $f(\mathbf{x}) \in \Delta^K$. Let $\pi_1, \ldots, \pi_K$ denote the

indices for a descending ordering of the softmax scores, i.e. $f_{\pi_1}(\mathbf{x})$ is the score corresponding to the modal prediction. The resulting non-conformity function and test statistic are:

$$S\left(\boldsymbol{x}, y; \mathcal{D}\right) = \sum_{k=1}^{J} f_{\pi_k}(\boldsymbol{x}) \tag{16}$$

where $\pi_J = y$, the index that matches the true label. Hence $J = K$ only when the classifier ranks the true label last and $J < K$ otherwise. CP can be thought of as a re-calibration method that, instead of improving the individual confidence scores, computes a threshold for which we can apply an interpretable level of confidence.

2 Supervised Learning from Human-Generated Labels

When conducting supervised learning, we often don't have access to some objective mechanism for obtaining the labels. Rather, we need to ask usually several human annotators to provide one. This process is also known as *crowdsourcing* and whole platforms such as *Amazon Mechanical Turk* exist in order to make it easy to query human annotators at scale. Specifically, assume we have N feature vectors $\{\boldsymbol{x}_n\}_{n=1}^N$, pass them to L annotators, and they each produce a response $\mathbf{y}_{n,l} \in \{0,1\}^K$, where K is the total number of classes (i.e. a multi-class classification task) and $\mathbf{y}_{n,l,k} = 1$ denotes that the annotator has produced a label for the kth class. All other dimensions are zero.

We can, of course, train a supervised model on this data directly by assuming each response is an independent observation:

$$p\left(\{\boldsymbol{y}_{n,1}, \ldots, \boldsymbol{y}_{n,L}\}_{n=1}^N | \boldsymbol{x}_n, \ldots, \boldsymbol{x}_N\right) = \prod_{n=1}^{N} \prod_{l=1}^{L} p\left(\boldsymbol{y}_{n,l} | \boldsymbol{x}_n\right),$$

but this model is in jeopardy of being mislead by noisy labelers. Alternatively, one could use all data, but give a weight to each annotator that reflects their quality:

$$\prod_{n=1}^{N} \prod_{l=1}^{L} \left[\left(p\left(\boldsymbol{y}_{n,l} | \boldsymbol{x}_n\right)\right]^{w_l}, \text{ where } w_l \geq 0.\right.$$

This is a better option, but the weights w_l are hard to determine without access to a ground-truth label to separate the experts from the incompetent labelers.

Given these challenges of using all provided labels, methods that infer some ground-truth label have received much attention. The simplest and most common of these methods—already alluded to above—is majority voting:

$$\hat{t}_{n,k} \;=\; 1 \quad \text{if} \;\; k \;=\; \underset{k \in [1,K]}{\arg\max} \sum_{l=1}^{L} y_{n,l,k}.$$

All methods for inferring ground-truth work by checking for some form of consensus across the annotators. This has the drawback that there may be one expert annotator and four incompetent ones, but if the incompetent ones all agree, then the expert will be seen as an outlier and likely in the wrong. But, of course, this problem is nearly impossible to prevent

without an external signal of quality.

Below we will consider more sophisticated models that estimate a confusion matrix to understand how the crowd is generating their labels. A crucial modeling in this setting is whether to model the ability of the human labelers. Doing so, of course, incurs a significant modeling and computational overhead, which can lead one astray if the number of labels observed per annotator is small. Below we present two common probabilistic models for determining consensus labels in crowdsourcing applications—pooled and unpooled. Here, 'pooling' refers to whether we pool all annotators together or try to model their individual abilities.

2.1 Pooled: Multinomial Model

The simplest model of this annotation scheme is a *pooled multinomial* model—*pooled* because we don't model individual annotators. Rather, we assume we have $K \times K$ parameters $\pi_{k,j} \in [0, 1]$, which represents the probability that the labelers will return class j when the true class is k. This implies that these parameters are normalized across potential mislabelings: $\sum_{j=1}^{K} \pi_{k,j} = 1$. We can think of the full $K \times K$ matrix of parameters, denote it as $\mathfrak{C}$, as a *confusion matrix*, again, representing the crowd's mislabeling tendencies. We will see how to extend this model to assess the quality of individual annotators in the next sub-section.

Observed Truth We first discuss an easier case of the model above where we assume that we know the true annotation, denoted by an indicator vector $\boldsymbol{t}$ such that $t_k = 1$ denotes that the true class is k and all other dimensions are zero. The generative process can then be written as:

$$\mathbf{t}_n \sim \text{Categorical}(\text{p}^*), \quad \mathbf{y}_{n,l} \sim \text{Categorical}\left(\mathbf{y}_{n,l};\ \langle \mathfrak{C}, \mathbf{t}_n \rangle\right), \forall\ n \in [1, N], l \in [1, L] \tag{17}$$

where p^* are the underlying truth (i.e. class) probabilities, which are unknown. The inner product $\langle \mathfrak{C}, \mathbf{t} \rangle = \boldsymbol{\pi}_k$, and thus we can think of $\mathbf{t}$ as an indicator vector, selecting out a particular row of $\mathfrak{C}$ corresponding to the ground-truth class. Assume we have observed a data set $\mathcal{Y} = \left\{ \{\boldsymbol{y}_{n,l}\}_{l=1}^{L} \right\}_{n=1}^{N}$ and the corresponding ground-truths $\boldsymbol{T} = \{\boldsymbol{t}_n\}_{n=1}^{N}$. We can

write the log-likelihood for the above model as:

$$\begin{aligned}
\ell\left(\{\pi_{k,j}\}_{k,j=1}^{K}; \mathcal{Y}, \boldsymbol{T}\right) &= \log\left\{\prod_{n=1}^{N}\prod_{k=1}^{K}\prod_{l=1}^{L} p\left(\boldsymbol{y}_{n,l}|t_{n,k}\right)\right\} \\
&= \log\left\{\prod_{n=1}^{N}\prod_{k=1}^{K}\left[\prod_{l=1}^{L}\text{Categorical}\left(\boldsymbol{y}_{n,l};\boldsymbol{\pi}_k\right)\right]^{t_{n,k}}\right\} \\
&= \log\left\{\prod_{n=1}^{N}\prod_{k=1}^{K}\left[\prod_{l=1}^{L}\prod_{j=1}^{K}\pi_{k,j}^{y_{n,l,j}}\right]^{t_{n,k}}\right\} \\
&= \log\left\{\prod_{n=1}^{N}\prod_{k=1}^{K}\left[\prod_{j=1}^{K}\pi_{k,j}^{\sum_l y_{n,l,j}}\right]^{t_{n,k}}\right\} \\
&= \sum_{n=1}^{N}\sum_{k=1}^{K} t_{n,k}\cdot\sum_{j=1}^{K}\left(\sum_{l=1}^{L} y_{n,l,j}\right)\cdot\log\pi_{k,j}.
\end{aligned} \tag{18}$$

Now to find a maximum likelihood estimator (MLE) for $\pi_{k,j}$, we can take the derivative and set it equal to zero. Yet, since $\sum_{j=1}^{K}\pi_{k,j}=1$, we also need to incorporate this constraint as a lagrangian:

$$\begin{aligned}
\frac{\partial}{\partial\pi_{k,j}}\ell\left(\{\pi_{k,j}\}_{k,j=1}^{K},\lambda;\mathcal{Y},\boldsymbol{T}\right) &= \frac{\partial}{\partial\pi_{k,j}}\ell\left(\{\pi_{k,j}\}_{k,j=1}^{K};\mathcal{Y},\boldsymbol{T}\right) + \frac{\partial}{\partial\pi_{k,j}}\lambda\sum_{k=1}^{K}\left(1-\sum_{j=1}^{K}\pi_{k,j}\right) \\
&= \left[\sum_{n=1}^{N} t_{n,k}\left(\sum_{l=1}^{L} y_{n,l,j}\right)\cdot\frac{1}{\pi_{k,j}}\right]-\lambda
\end{aligned} \tag{19}$$

Setting this equation to zero, we have

$$\begin{aligned}
0 &= \left[\sum_{n=1}^{N} t_{n,k}\left(\sum_{l=1}^{L} y_{n,l,j}\right)\cdot\frac{1}{\pi_{k,j}}\right]-\lambda \\
\pi_{k,j} &= \frac{1}{\lambda}\left[\sum_{n=1}^{N} t_{n,k}\sum_{l=1}^{L} y_{n,l,j}\right].
\end{aligned} \tag{20}$$

Now making sure we obey the sum-to-one constraint, we have

$$\begin{aligned}
1 &= \sum_{j=1}^{K}\pi_{k,j} \\
&= \frac{1}{\lambda}\sum_{j=1}^{K}\sum_{n=1}^{N} t_{n,k}\sum_{l=1}^{L} y_{n,l,j} \\
\lambda &= \sum_{j=1}^{K}\sum_{n=1}^{N} t_{n,k}\sum_{l=1}^{L} y_{n,l,j}.
\end{aligned} \tag{21}$$

Algorithm 1: Expectation-Maximization for Pooled Model

Input: Labels $\mathcal{Y}$, prior $p(\mathbf{t})$, initialized confusion matrix $\mathfrak{C}_0$, max number of EM iterations T

for $t = [1, T]$ **do**

  update truth posterior $p\left(\mathbf{t} \mid \mathcal{Y}, \widehat{\mathfrak{C}}_{t-1}\right)$ (Equation 24)

  update confusion matrix to obtain $\widehat{\mathfrak{C}}_t$ (Equation 26)

end

Output: Estimated confusion matrix $\widehat{\mathfrak{C}}_T$

Putting everything together, we then have the final form for the MLE:

$$\hat{\pi}_{k,j} = \frac{\sum_{n=1}^{N} t_{n,k} \sum_{l=1}^{L} y_{n,l,j}}{\sum_{i=1}^{K} \sum_{n=1}^{N} t_{n,k} \sum_{l=1}^{L} y_{n,l,i}}. \tag{22}$$

We can interpret this quantity as the number of times class j is reported by an annotator when the true class is k, divided by the total number of annotations of any result when the true class is k.

Unobserved Truth Previously we assumed we had access to the ground-truth class $\mathbf{t}_n$, but often this is an unrealistic assumption. If we have the ground-truth, then we probably wouldn't need to query the annotators in the first place. Luckily, this model is amenable to a standard missing data treatment via the *expectation-maximization* (EM) algorithm. We can construct a conditionally conjugate posterior distribution over $\mathbf{t}$, compute its expected value, and then perform the maximization step above using these expectations instead of the ground-truth. Firstly, the aforementioned posterior can be obtained via a prior distribution over $\mathbf{t}$:

$$\begin{aligned} p\left(\mathrm{t}_{n,k} = 1 \mid \boldsymbol{y}_{n,1}, \ldots, \boldsymbol{y}_{n,L}\right) &\propto \left(\prod_{l=1}^{L} p\left(\boldsymbol{y}_{n,l} | t_{n,k} = 1\right)\right) \cdot p\left(\mathrm{t}_{n,k} = 1\right) \\ &= p_k \cdot \prod_{l=1}^{L} \text{Categorical}\left(\boldsymbol{y}_{n,l}; \boldsymbol{\pi}_k\right) \end{aligned} \tag{23}$$

where $p_k = p\left(\mathrm{t}_{n,k} = 1\right)$, as it is assumed to be the same across all n. Normalizing this distribution, we have

$$\begin{aligned} p\left(\mathrm{t}_{n,k} = 1 \mid \boldsymbol{y}_{n,1}, \ldots, \boldsymbol{y}_{n,L}\right) &= \frac{p_k \cdot \prod_{l=1}^{L} \text{Categorical}\left(\boldsymbol{y}_{n,l}; \boldsymbol{\pi}_k\right)}{\sum_{i=1}^{K} p_i \cdot \prod_{l=1}^{L} \text{Categorical}\left(\boldsymbol{y}_{n,l}; \boldsymbol{\pi}_i\right)} \\ &= \frac{p_k \cdot \prod_{j=1}^{K} \pi_{k,j}^{\sum_l y_{n,l,j}}}{\sum_{i=1}^{K} p_i \cdot \prod_{j=1}^{K} \pi_{i,j}^{\sum_l y_{n,l,j}}} \\ &= \hat{p}_{n,k}^{t} \end{aligned} \tag{24}$$

where the denominator is simply a sum over all classes. If the prior probabilities are equal ($p_k = p_i,\ \forall i \in [1, K]$), then these terms would cancel out. We denote the above quantity as $\hat{p}_{n,k}^{t}$ for the sake of notational brevity.

Now, formally, the expectation step (i.e. E-step) of the EM procedure computes the

expected log-likelihood:

$$\begin{aligned}
\mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\ell\left(\{\pi_{k,j}\}_{k,j=1}^{K};\mathcal{Y},\boldsymbol{T}\right)\right] &= \mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\sum_{n=1}^{N}\sum_{k=1}^{K} t_{n,k}\cdot\sum_{j=1}^{K}\left(\sum_{l=1}^{L} y_{n,l,j}\right)\cdot\log\pi_{k,j}\right] \\
&= \sum_{n=1}^{N}\sum_{k=1}^{K}\mathbb{E}_{\mathrm{t}|\boldsymbol{y}}\left[t_{n,k}\right]\cdot\sum_{j=1}^{K}\left(\sum_{l=1}^{L} y_{n,l,j}\right)\cdot\log\pi_{k,j} \\
&= \sum_{n=1}^{N}\sum_{k=1}^{K}\hat{p}_{n,k}^{t}\cdot\sum_{j=1}^{K}\left(\sum_{l=1}^{L} y_{n,l,j}\right)\cdot\log\pi_{k,j}.
\end{aligned} \tag{25}$$

The M-step is to maximize this equation, just as before, which yields the estimator:

$$\hat{\pi}_{k,j} = \frac{\sum_{n=1}^{N}\hat{p}_{n,k}^{t}\sum_{l=1}^{L} y_{n,l,j}}{\sum_{i=1}^{K}\sum_{n=1}^{N}\hat{p}_{n,k}^{t}\sum_{l=1}^{L} y_{n,l,i}} \tag{26}$$

which can be interpreted as a 'soft' version of the MLE presented in Equation 22. The EM procedures is summarized by iteratively computing Equation 24 and Equation 26 until convergence.

2.2 Unpooled: Dawid-Skene Model

In many applications with crowdsourced data, we want models that can not only understand how the crowd is mislabeling, but we want that information *per annotator*. Having such information allows poor annotators to be excluded from either the current or future requests. That can be done with the multinomial model above but with one change, having a confusion matrix $\mathfrak{C}_l$ for every labeler, $l \in [1, L]$. Doing so expands the number of parameters from $K \times K$ to $L \times K \times K$. Let $\pi_{l,k,j}$ denote the probability of the l-th labeler reporting class j when the true class is k. And again, $\sum_j \pi_{l,k,j} = 1$. This model is known as the *Dawid-Skene* model, named after the authors of the paper in which it was first introduced (Dawid and Skene, 1979).

Observed Truth Again we first discuss the easier case of having access to the true annotation, denoted by an indicator vector $\boldsymbol{t}$ such that $t_k = 1$ denotes that the true class is k and all other dimensions are zero. The generative process can then be written as:

$$\mathbf{t}_n \sim \text{Categorical}(\mathrm{p}^*), \quad \mathbf{y}_{n,l} \sim \text{Categorical}\left(\mathbf{y}_{n,l};\ \langle\mathfrak{C}_l, \mathbf{t}_n\rangle\right), \forall\ n \in [1, N], l \in [1, L] \tag{27}$$

where p^* are the underlying truth (i.e. class) probabilities, which are unknown. Assume we have observed a data set $\mathcal{Y} = \left\{\{\boldsymbol{y}_{n,l}\}_{l=1}^{L}\right\}_{n=1}^{N}$ and the corresponding ground-truths

$\boldsymbol{T} = \{\boldsymbol{t}_n\}_{n=1}^N$. We can write the log-likelihood for the above model as:

$$\begin{aligned}
\ell\left(\{\mathfrak{C}_l\}_{l=1}^L; \mathcal{Y}, \boldsymbol{T}\right) &= \log\left\{\prod_{n=1}^N \prod_{k=1}^K \prod_{l=1}^L p\left(\boldsymbol{y}_{n,l} | t_{n,k}\right)\right\} \\
&= \log\left\{\prod_{n=1}^N \prod_{k=1}^K \left[\prod_{l=1}^L \text{Categorical}\left(\boldsymbol{y}_{n,l}; \boldsymbol{\pi}_{l,k}\right)\right]^{t_{n,k}}\right\} \\
&= \log\left\{\prod_{n=1}^N \prod_{k=1}^K \left[\prod_{l=1}^L \prod_{j=1}^K \pi_{l,k,j}^{y_{n,l,j}}\right]^{t_{n,k}}\right\} \\
&= \sum_{n=1}^N \sum_{k=1}^K t_{n,k} \cdot \sum_{l=1}^L \sum_{j=1}^K y_{n,l,j} \cdot \log \pi_{l,k,j}.
\end{aligned} \tag{28}$$

Now to find a maximum likelihood estimator (MLE) for $\pi_{l,k,j}$, we can take the derivative and set it equal to zero. Yet, since $\sum_{j=1}^K \pi_{l,k,j} = 1$, we also need to incorporate this constraint as a lagrangian:

$$\begin{aligned}
\frac{\partial}{\partial \pi_{l,k,j}} \ell\left(\{\mathfrak{C}_l\}_{l=1}^L, \lambda; \mathcal{Y}, \boldsymbol{T}\right) &= \frac{\partial}{\partial \pi_{l,k,j}} \ell\left(\{\mathfrak{C}_l\}_{l=1}^L; \mathcal{Y}, \boldsymbol{T}\right) + \frac{\partial}{\partial \pi_{l,k,j}} \lambda \sum_{k=1}^K \left(1 - \sum_{j=1}^K \pi_{l,k,j}\right) \\
&= \left[\sum_{n=1}^N t_{n,k} \cdot y_{n,l,j} \cdot \frac{1}{\pi_{l,k,j}}\right] - \lambda
\end{aligned} \tag{29}$$

Setting this equation to zero, we have

$$\begin{aligned}
0 &= \left[\sum_{n=1}^N t_{n,k} \cdot y_{n,l,j} \cdot \frac{1}{\pi_{l,k,j}}\right] - \lambda \\
\pi_{l,k,j} &= \frac{1}{\lambda}\left[\sum_{n=1}^N t_{n,k} \cdot y_{n,l,j}\right].
\end{aligned} \tag{30}$$

Making sure we obey the sum-to-one constraint, we have

$$\begin{aligned}
1 &= \sum_{j=1}^K \pi_{l,k,j} \\
&= \frac{1}{\lambda} \sum_{j=1}^K \sum_{n=1}^N t_{n,k} \cdot y_{n,l,j} \\
\lambda &= \sum_{j=1}^K \sum_{n=1}^N t_{n,k} \cdot y_{n,l,j}.
\end{aligned} \tag{31}$$

Putting everything together, we then have the final form for the MLE:

$$\hat{\pi}_{l,k,j} = \frac{\sum_{n=1}^N t_{n,k} \cdot y_{n,l,j}}{\sum_{i=1}^K \sum_{n=1}^N t_{n,k} \cdot y_{n,l,i}}. \tag{32}$$

We can interpret this quantity as the number of times class j is reported by annotator l when the true class is k, divided by the total number of annotations reported by annotator

l, of any result, when the true class is k.

Unobserved Truth We again turn to the more interesting case in which the ground-truth class $\mathbf{t}_n$ is not observed. Again we can apply the EM algorithm to fill in the missing truth variables. The first step is the same as above: placing a prior distribution over $\mathbf{t}$:

$$\begin{aligned} p\left(\mathrm{t}_{n,k} \mid \boldsymbol{y}_{n,1}, \ldots, \boldsymbol{y}_{n,L}\right) &\propto \left(\prod_{l=1}^{L} p\left(\boldsymbol{y}_{n,l} | t_{n,k}\right)\right) \cdot p\left(\mathrm{t}_{n,k}\right) \\ &= p_k \cdot \prod_{l=1}^{L} \text{Categorical}\left(\boldsymbol{y}_{n,l}; \boldsymbol{\pi}_{l,k}\right) \end{aligned} \tag{33}$$

where $p_k = p\left(\mathrm{t}_{n,k}\right)$, as it is assumed to be the same across all n. Normalizing this distribution, we have

$$p\left(\mathrm{t}_{n,k} \mid \boldsymbol{y}_{n,1}, \ldots, \boldsymbol{y}_{n,L}\right) = \frac{p_k \cdot \prod_{l=1}^{L} \prod_{j=1}^{K} \pi_{l,k,j}^{y_{n,l,j}}}{\sum_{i=1}^{K} p_i \cdot \prod_{l=1}^{L} \prod_{j=1}^{K} \pi_{l,i,j}^{y_{n,l,j}}} = \hat{p}_{n,k}^{t} \tag{34}$$

where the denominator is simply a sum over all classes. Again the expectation step (i.e. E-step) of the EM procedure computes the expected log-likelihood:

$$\begin{aligned} \mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\ell\left(\{\mathfrak{C}_l\}_{l=1}^{L}; \mathcal{Y}, \boldsymbol{T}\right)\right] &= \mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\sum_{n=1}^{N}\sum_{k=1}^{K} t_{n,k} \cdot \sum_{l=1}^{L}\sum_{j=1}^{K} y_{n,l,j} \cdot \log \pi_{l,k,j}\right] \\ &= \sum_{n=1}^{N}\sum_{k=1}^{K} \mathbb{E}_{\mathrm{t}|\boldsymbol{y}}\left[t_{n,k}\right] \cdot \sum_{l=1}^{L}\sum_{j=1}^{K} y_{n,l,j} \cdot \log \pi_{l,k,j} \\ &= \sum_{n=1}^{N}\sum_{k=1}^{K} \hat{p}_{n,k}^{t} \cdot \sum_{l=1}^{L}\sum_{j=1}^{K} y_{n,l,j} \cdot \log \pi_{l,k,j}. \end{aligned} \tag{35}$$

The M-step is to maximize this equation, just as before, which yields the estimator:

$$\hat{\pi}_{l,k,j} = \frac{\sum_{n=1}^{N} \hat{p}_{n,k}^{t} \cdot y_{n,l,j}}{\sum_{i=1}^{K}\sum_{n=1}^{N} \hat{p}_{n,k}^{t} \cdot y_{n,l,i}} \tag{36}$$

which can be interpreted as a 'soft' version of the MLE presented in Equation 32. The EM procedures is summarized by iteratively computing Equation 34 and Equation 36 until convergence.

2.3 Jointly Learning Ground-Truth and a Predictive Model

In the setting of unobserved truth, the Multinomial and Dawid-Skene models are first applied to infer a ground-truth label, which would then be used to train a traditional classifier. However, if obtaining this downstream classifier is our ultimate, then perhaps it is sub-optimal to have learning be decoupled into a two-stage process. Rather, as proposed by Raykar et al. (2010), it is better to have a joint formulation. A joint objective can be defined as follows. Let $p(\mathbf{t}|\mathbf{x}, \boldsymbol{\theta})$ denote the classifier defined by the conditional probability of the ground-truth variable $\mathbf{t}$, given the features $\mathbf{x}$ and classifier parameters $\boldsymbol{\theta}$. The generative

Algorithm 2: Expectation-Maximization for Dawid-Skene Model

Input: Labels $\mathcal{Y}$, prior $p(\mathbf{t})$, initialized L confusion matrices $\mathfrak{C}_{1,0}, \ldots, \mathfrak{C}_{L,0}$, max number of EM iterations T

for $t = [1, T]$ **do**

  update truth posterior $p\left(\mathbf{t} \mid \mathcal{Y}, \widehat{\mathfrak{C}}_{1,t-1}, \ldots, \widehat{\mathfrak{C}}_{L,t-1}\right)$ (Equation 34)

  for $l = [1, L]$ **do**

    update confusion matrix to obtain $\widehat{\mathfrak{C}}_{l,t}$ (Equation 36)

  end

end

Output: L estimated confusion matrices $\widehat{\mathfrak{C}}_{1,T}, \ldots, \widehat{\mathfrak{C}}_{L,T}$

process is then:

$$\mathbf{t}_n \sim p(\mathbf{t}_n|\mathbf{x}_n, \boldsymbol{\theta}), \quad \mathbf{y}_{n,l} \sim \text{Categorical}\left(\mathbf{y}_{n,l};\ \langle \mathfrak{C}_l, \mathbf{t}_n \rangle\right), \forall\ n \in [1, N], l \in [1, L]. \tag{37}$$

Note that this model is essentially the Dawid-Skene model but with now the classifier generating the higher-level distribution over truth.

To learn in this model, the conditional probability of just the annotations can be written by marginalizing out the ground-truth:

$$\begin{aligned}
p\left(\mathcal{Y} \mid \boldsymbol{x}_1, \ldots, \boldsymbol{x}_N, \boldsymbol{\theta}\right) &= \prod_{n=1}^{N} p\left(\boldsymbol{y}_{n,1}, \ldots, \boldsymbol{y}_{n,L} \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \\
&= \prod_{n=1}^{N} \sum_{\mathbf{t}_n} p\left(\boldsymbol{y}_{n,1}, \ldots, \boldsymbol{y}_{n,L} \mid \mathbf{t}_n\right) \cdot p\left(\mathbf{t}_n \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \\
&= \prod_{n=1}^{N} \sum_{\mathbf{t}_n} \left(\prod_{l=1}^{L} p\left(\boldsymbol{y}_{n,l} \mid \mathbf{t}_n\right) \right) \cdot p\left(\mathbf{t}_n \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \\
&= \prod_{n=1}^{N} \sum_{\mathbf{t}_n} \prod_{k=1}^{K} \left[p\left(\mathrm{t}_{n,k} \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot \prod_{l=1}^{L} p\left(\boldsymbol{y}_{n,l} \mid \mathrm{t}_{n,k} = 1\right) \right]^{t_{n,k}} \\
&= \prod_{n=1}^{N} \sum_{\mathbf{t}_n} \prod_{k=1}^{K} \left[p\left(\mathrm{t}_{n,k} \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot \prod_{l=1}^{L} \text{Categorical}\left(\boldsymbol{y}_{n,l};\ \boldsymbol{\pi}_{l,k}\right) \right]^{t_{n,k}} \\
&= \prod_{n=1}^{N} \sum_{\mathbf{t}_n} \prod_{k=1}^{K} \left[p\left(\mathrm{t}_{n,k} \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot \prod_{l=1}^{L} \prod_{j=1}^{K} \pi_{l,k,j}^{y_{n,l,j}} \right]^{t_{n,k}}
\end{aligned} \tag{38}$$

where we see that this joint model is essentially the Dawid-Skene model with the truth variable $\mathbf{t}$ marginalized away via the classifier.

Learning can again be done via the EM algorithm, but here we need to work with a

Algorithm 3: Expectation-Maximization for Raykar et al. Model

Input: Labels $\mathcal{Y}$, features $\mathcal{X}$, prior $p(\mathbf{t})$, initialized L confusion matrices $\mathfrak{C}_{1,0}, \ldots, \mathfrak{C}_{L,0}$, max number of EM iterations T

for $t = [1, T]$ **do**

  update truth posterior $p\left(\mathbf{t} \mid \mathcal{Y}, \widehat{\mathfrak{C}}_{1,t-1}, \ldots, \widehat{\mathfrak{C}}_{L,t-1}\right)$ (Equation 34)

  fit classifier to posterior probabilities to obtain parameters $\widehat{\boldsymbol{\theta}}_t$ (Equation 42)

  for $l = [1, L]$ **do**

    update confusion matrix to obtain $\widehat{\mathfrak{C}}_{l,t}$ (Equation 41)

  end

end

Output: L estimated confusion matrices $\widehat{\mathfrak{C}}_{1,T}, \ldots, \widehat{\mathfrak{C}}_{L,T}$, classifier parameters $\widehat{\boldsymbol{\theta}}_T$

lower-bound on the marginal likelihood from above:

$$
\begin{aligned}
\ell\left(\{\mathfrak{C}_l\}_{l=1}^L, \boldsymbol{\theta}; \mathcal{Y}\right) &= \log\left\{\prod_{n=1}^N \sum_{\mathbf{t}_n} \prod_{k=1}^K \left[p\left(\mathrm{t}_{n,k} \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot \prod_{l=1}^L \prod_{j=1}^K \pi_{l,k,j}^{y_{n,l,j}}\right]^{t_{n,k}}\right\} \\
&= \sum_{n=1}^N \log\left\{\sum_{\mathbf{t}_n} \prod_{k=1}^K \left[p\left(\mathrm{t}_{n,k} \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot \prod_{l=1}^L \prod_{j=1}^K \pi_{l,k,j}^{y_{n,l,j}}\right]^{t_{n,k}}\right\} \\
&\geq \sum_{n=1}^N \sum_{\mathbf{t}_n} \sum_{k=1}^K t_{n,k} \cdot p\left(\mathrm{t}_{n,k} \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot \sum_{l=1}^L \sum_{j=1}^K y_{n,l,j} \cdot \log \pi_{l,k,j} \\
&= \sum_{n=1}^N \sum_{k=1}^K p\left(\mathrm{t}_{n,k} = 1 \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot \sum_{l=1}^L \sum_{j=1}^K y_{n,l,j} \cdot \log \pi_{l,k,j} \\
&= \tilde{\ell}\left(\{\mathfrak{C}_l\}_{l=1}^L, \boldsymbol{\theta}; \mathcal{Y}\right)
\end{aligned}
\tag{39}
$$

where the inequality is obtained via Jensen's inequality for convex functions.

Now finding an estimator for $\pi_{l,k,j}$:

$$
\begin{aligned}
\frac{\partial}{\partial \pi_{l,k,j}} \tilde{\ell}\left(\{\mathfrak{C}_l\}_{l=1}^L, \boldsymbol{\theta}, \lambda; \mathcal{Y}\right) &= \frac{\partial}{\partial \pi_{l,k,j}} \tilde{\ell}\left(\{\mathfrak{C}_l\}_{l=1}^L, \boldsymbol{\theta}; \mathcal{Y}\right) \; + \; \lambda \sum_{k=1}^K \left(1 - \sum_{j=1}^K \pi_{l,k,j}\right) \\
&= \sum_{n=1}^N p\left(\mathrm{t}_{n,k} = 1 \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot y_{n,l,j} \cdot \frac{1}{\pi_{l,k,j}} - \lambda \\
\pi_{l,k,j} &= \frac{1}{\lambda}\left[\sum_{n=1}^N p\left(\mathrm{t}_{n,k} = 1 \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot y_{n,l,j}\right] \\
\lambda &= \sum_{j=1}^K \sum_{n=1}^N p\left(\mathrm{t}_{n,k} = 1 \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot y_{n,l,j}.
\end{aligned}
\tag{40}
$$

Putting it all together, we then have:

$$
\hat{\pi}_{l,k,j} \;=\; \frac{\sum_{n=1}^N p\left(\mathrm{t}_{n,k} = 1 \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot y_{n,l,j}}{\sum_{i=1}^K \sum_{n=1}^N p\left(\mathrm{t}_{n,k} = 1 \mid \boldsymbol{x}_n, \boldsymbol{\theta}\right) \cdot y_{n,l,i}} \tag{41}
$$

Then given the confusion matrix parameters, we can estimate the classifier as follows. Com-

puting the posterior over truth values via Equation 34, the training objective for the classifier is:

$$\begin{aligned}
\mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\ell(\boldsymbol{\theta};\mathbf{T},\boldsymbol{X})\right] &= \mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\log p(\mathbf{T}|\boldsymbol{X},\boldsymbol{\theta})\right] \\
&= \mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\sum_{n=1}^{N}\log p(\mathbf{t}_b|\boldsymbol{x}_n,\boldsymbol{\theta})\right] \\
&= \mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\sum_{n=1}^{N}\log \text{Categorical}(\mathbf{t}_b; f(\boldsymbol{x}_n;\boldsymbol{\theta}))\right] \\
&= \mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\sum_{n=1}^{N}\sum_{k=1}^{K}\mathrm{t}_{n,k}\log f_k(\boldsymbol{x}_n;\boldsymbol{\theta})\right] \\
&= \sum_{n=1}^{N}\sum_{k=1}^{K}\mathbb{E}_{\mathbf{T}|\mathcal{Y}}\left[\mathrm{t}_{n,k}\right]\log f_k(\boldsymbol{x}_n;\boldsymbol{\theta}) \\
&= \sum_{n=1}^{N}\sum_{k=1}^{K}\hat{p}^{t}_{n,k}\log f_k(\boldsymbol{x}_n;\boldsymbol{\theta})
\end{aligned} \tag{42}$$

where $\hat{p}^{t}_{n,k}$ is again from Equation 34. We can see this is the usually classifier loss but with 'soft' labels. If the classifier would perfectly recreate these posterior probabilities, then they would be plugged into Equation 41, which in turn would recover Equation 36.

Alternative Formulation Yan et al. (2010) proposed a variant of this model that conditions both the truth and observed label on the features:

$$\mathbf{t}_n \sim p\left(\mathbf{t}_n|\mathbf{x}_n,\boldsymbol{\theta}_t\right), \quad \mathbf{y}_{n,l} \sim p\left(\mathbf{y}_{n,l}|\mathbf{x}_n,\mathbf{t}_n,\boldsymbol{\theta}_l\right), \quad \forall\ n\in[1,N], l\in[1,L]. \tag{43}$$

The logic behind this alteration is that, by having $\mathrm{y}\sim p(\mathrm{y}|\mathrm{t})$, the Raykar et al. (2010) model assumes that the observation is a noisy version of the ground-truth and that noise is completely determined by the ground-truth class. However, in many real-world cases, the label error might be due to noise in the input features as well. For example, perhaps the human is annotating an image, and the image is of very poor quality. The low-quality of the input could be the reason why the annotator produces the wrong label—i.e. the noise from the input propagating into the label—and not due to any systematic labeling bias they have for a particular class. This model also has benefits for active learning, as we will consider in the next subsection.

2.4 Active Learning

In the previous subsection, we were mostly concerned with how to filter out noise or come to consensus when given multiple annotations. This subsection focuses on how we obtain labels from humans in the first place and, ideally, through the most efficient means possible. Collecting human annotations is expensive as it costs humans their time. For example, annotating the words in an audio recording takes about ten times longer than the audio recording itself. In turn, the person wanting the labels often must compensate the workers with payment. Hence we'd like methodologies that minimize these costs by minimizing the number of labels collected. This brings us to the sub-field of *active learning*; most of the

information from this section is reproduced from the authoritative survey of Settles (2012).

Definition & Types *Active learning* (AL) allows predictive models to choose their own training data, similarly to how a math student might ask their teacher for guidance on a particular problem they find difficult. The high-level idea is that the model will inspect an unlabeled data point and then determine if its label were acquired, then the model's performance would substantially improve. This process is repeated until the model's performance has either reached an acceptable level or plateaus to the level yielded by training on all available data. There are three standard scenarios considered for AL:

- **Membership Query Synthesis** (MQS): This is the most general form of AL. In this setting, the model can request *any* feature vector in the feature space $\mathcal{X}$ as well as its label. This works well when every point in the feature space has a clearly defined label. For example, the task of predicting if a robot arm, in a particular configuration, can hold a glass of water without spilling is a good use case for MQS (assuming $\mathcal{X}$ does not contain any impossible arm configurations). For every feature vector chosen by the model, it is quite easy to test if the arm position is good or not simply by running the experiment and seeing if any water spills from the cup. On the other hand, MQS is not good for image classification since it is very likely feature vectors could be requested for which there is no discernible label. For example, if we are considering the space of binary images representing digits, there are many possible binary images that correspond no recognizable digit.

- **Stream-Based Selective Sampling**: In this setting, feature vectors are assumed to appear sequentially, and the model must decided whether to request its label or discard the vector as 'uninteresting.' This use case is prevalent in data collection on low-resource devices. Imagine there is an autonomous vehicle driving in a new location, and its designers would like the car to save its sensor readings during 'interesting' sections of the road or during novel events. It would be too costly to save all information from the whole driving run and so the system must decide for itself which data points to save for later annotation.

- **Pool-Based Active Learning**: This setting—which is the most commonly studies formulation—assumes that there is an unlabeled data set known as the *pool set*, and the model is allowed to request the label for a particular member of the pool set. Once the label is acquired, that point is removed from the pool, and the model is retrained to include that newly selected point and label. The process repeats until the model reaches a satisfactory performance or the pool set is empty. There is also a batch variant in which the model can request multiple points for labeling at each round.

Acquisition Functions for Pool-Based Active Learning We will discuss *pool-based AL* exclusively from here forward, and thus when writing 'AL', we mean the pool-based formulation. The setting can be defined more formally as follows. We consider a predictive model $p(\mathbf{y}|\boldsymbol{x}, \boldsymbol{\theta})$, where $\mathbf{y}$ is the label, $\mathbf{x}$ are the features, and $\boldsymbol{\theta} \in \Theta$ are the parameters. We assume this model is trained on an initial labeled dataset $\mathcal{D}_0 = \{\boldsymbol{x}_n, \boldsymbol{y}_n\}_{n=1}^N$, and thus we denote this model as $p(\mathbf{y}|\boldsymbol{x}, \boldsymbol{\theta}_{t=0})$ where the subscript on the parameter indexes time. In addition to the initial dataset $\mathcal{D}_0$, we assume access to (i) an unlabeled pool

set $\mathcal{X}_p^{t=0} = \{\boldsymbol{x}_m\}_{m=1}^M$, and (ii) an oracle labeling mechanism which can provide labels $\mathcal{Y}_p = \{\boldsymbol{y}_m\}_{m=1}^M$ for the corresponding features in the pool set.

At each step in the AL loop, an *acquisition function* (AF) $\mathcal{A}(\boldsymbol{x}; \boldsymbol{\theta}_t)$ is evaluated for every member of the pool set. $\mathcal{A}$ is written as a function of $\boldsymbol{\theta}_t$ because the AF changes along with the state of the predictive model. We want the AF to acquire the most interesting point to the current model and thus will collect the label of the point that maximizes the AF, denoted $\boldsymbol{x}^*$:

$$\boldsymbol{x}^* \;=\; \underset{\boldsymbol{x} \in \mathcal{X}_p^t}{\arg\max}\; \mathcal{A}(\boldsymbol{x}; \boldsymbol{\theta}_t). \tag{44}$$

Once the point has been chosen, the oracle provides it's label, and the new labeled set becomes:

$$\mathcal{D}_{t+1} = \mathcal{D}_t \cup \{\boldsymbol{x}^*, \boldsymbol{y}^*\}.$$

Moreover, the chosen point is removed from the pool set: $X_p^{t+1} = X_p^t \setminus \{\boldsymbol{x}^*\}$. The model is then re-trained on $\mathcal{D}_{t+1}$ to produce a new set of parameters $\boldsymbol{\theta}_{t+1}$, and the process repeats by collecting a new point to form $\mathcal{D}_{t+2}$, which in turn produces $\boldsymbol{\theta}_{t+2}$, and so on until either the pool set is empty or a satisfactory level of performance is reached (which would require evaluating the model on a held-out set after every re-training step). We now go on to describe several ways to implement the AF.

Idealized Setting: Error Reduction Ideally, we want to collect $(\boldsymbol{x}^*, \boldsymbol{y}^*)$ that, once the model is trained on the pair, will reduce the model's error on all future data points the model might see. This formulation as been called *optimal active learning* (Roy and McCallum, 2001), and the corresponding idealized AF is:

$$\boldsymbol{x}^* \;=\; \underset{\boldsymbol{x} \in \mathcal{X}_p^t}{\arg\max} \int_{\mathbf{x}} \left(\mathbb{D}\left[\mathbb{P}(\mathrm{y}|\mathbf{x}) \;||\; p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}_t\right)\right] \;-\; \mathbb{D}\left[\mathbb{P}(\mathrm{y}|\mathbf{x}) \;||\; p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}(\boldsymbol{x}, \boldsymbol{y}^*)\right)\right] \right) \mathbb{P}(\mathbf{x})\, d\mathbf{x} \tag{45}$$

where $\boldsymbol{\theta}(\boldsymbol{x}, \boldsymbol{y}^*)$ denotes the parameters produced by training on $\mathcal{D}_t \cup \{\mathbf{x}, \boldsymbol{y}^*\}$ and $\mathbb{D}$ is some measure of discrepancy or divergence between the true generative process $\mathbb{P}(\mathrm{y}|\mathbf{x})$ and the model—either at the current time step ($\boldsymbol{\theta}_t$) or updated with $\mathbf{x}$ ($\boldsymbol{\theta}(\boldsymbol{x}, \boldsymbol{y}^*)$). If $\mathbb{D}$ is the Kullback–Leibler divergence, then this AF is picking the point that results in the best improvement under maximum likelihood estimation. Thus we can think of this—again idealized—AF as looking ahead to the future, if we were to train the model on a particular point from the pool set, and see if that new model would better minimize the divergence between the model and the true distribution that is generating the data. This construction is called 'idealized' because we never have access to $\mathbb{P}(\mathbf{x})$, $\mathbb{P}(\mathrm{y}|\mathbf{x})$, and even if we did, it requires collecting the labels for all elements of the pool set—the very thing we wish to avoid. If $\left(\mathbb{D}\left[\mathbb{P}(\mathrm{y}|\mathbf{x}) \;||\; p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}_t\right)\right] \;-\; \mathbb{D}\left[\mathbb{P}(\mathrm{y}|\mathbf{x}) \;||\; p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}(\boldsymbol{x}, \boldsymbol{y}^*)\right)\right]\right) < 0$ for every element of the pool set, then AL should be stopped with $\boldsymbol{\theta}_t$ being the final model.

However, Roy and McCallum (2001) gives the procedure that aims to approximate the above idealized AF. Firstly, when $\mathbb{D}$ is taken to be the KL divergence, notice that:

$$\begin{aligned}
&\mathbb{KLD}\left[\mathbb{P}(\mathrm{y}|\mathbf{x}) \;||\; p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}_t\right)\right] \;-\; \mathbb{KLD}\left[\mathbb{P}(\mathrm{y}|\mathbf{x}) \;||\; p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}(\boldsymbol{x}, \boldsymbol{y}^*)\right)\right] \\
&= \mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathbf{x})}\left[\log p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}(\boldsymbol{x}, \boldsymbol{y}^*)\right) - \log p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}_t\right)\right] + \mathbb{H}\left[\mathbb{P}(\mathrm{y}|\mathbf{x})\right] - \mathbb{H}\left[\mathbb{P}(\mathrm{y}|\mathbf{x})\right] \\
&= \mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathbf{x})}\left[\log p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}(\boldsymbol{x}, \boldsymbol{y}^*)\right) - \log p\left(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}_t\right)\right].
\end{aligned} \tag{46}$$

Secondly, they use the pool set to approximate the integral over the underlying feature distribution: $\mathbb{P}(\mathbf{x}) \approx \frac{1}{M}\sum_{m=1}^{M}\delta\left[|\mathbf{x}-\boldsymbol{x}_n|\right]$. Thirdly, this still leaves the expectation over the unknown distribution $\mathbb{P}(\mathrm{y}|\mathbf{x})$; they recommend to approximate this with the current model: $\mathbb{P}(\mathrm{y}|\mathbf{x}) \approx p\left(\mathbf{y}|\mathbf{x},\boldsymbol{\theta}_t\right)$. Note that under this assumption, the difference of divergences further simplifies to:

$$\begin{aligned}\mathbb{E}_{\mathbb{P}(\mathrm{y}|\mathbf{x})}\left[\log p\left(\mathbf{y}|\mathbf{x},\boldsymbol{\theta}(\boldsymbol{x},\boldsymbol{y}^*)\right) - \log p\left(\mathbf{y}|\mathbf{x},\boldsymbol{\theta}_t\right)\right] \\ \approx \mathbb{E}_{p(\mathbf{y}|\mathbf{x},\boldsymbol{\theta}_t)}\left[\log p\left(\mathbf{y}|\mathbf{x},\boldsymbol{\theta}(\boldsymbol{x},\boldsymbol{y}^*)\right)\right] + \mathbb{H}\left[p\left(\mathbf{y}|\mathbf{x},\boldsymbol{\theta}_t\right)\right],\end{aligned} \tag{47}$$

where the second term is the entropy of the current model, and when integrated over the empirical distribution of the pool set, will be a constant (and thus can be dropped). The only remaining difficulty is that the above expression still depends on the true label $\boldsymbol{y}^*$, which we do not want to assume is know. Roy and McCallum (2001) again leverage the existing model for this, assuming that $\mathbf{y}^* \sim p\left(\mathbf{y}|\boldsymbol{x},\boldsymbol{\theta}_t\right)$. The final realizable AF is then:

$$\begin{aligned}\boldsymbol{x}^* &= \underset{\boldsymbol{x}\in\mathcal{X}_p^t}{\arg\max}\ \frac{1}{M}\sum_{m=1}^{M}\mathbb{E}_{p(\mathbf{y}'|\boldsymbol{x},\boldsymbol{\theta}_t)}\mathbb{E}_{p(\mathbf{y}|\boldsymbol{x}_m,\boldsymbol{\theta}_t)}\left[\log p\left(\mathbf{y}|\boldsymbol{x}_m,\boldsymbol{\theta}(\boldsymbol{x},\mathbf{y}')\right)\right] \\ &\approx \underset{\boldsymbol{x}\in\mathcal{X}_p^t}{\arg\max}\ \frac{1}{M}\cdot\frac{1}{J}\cdot\frac{1}{S}\sum_{m=1}^{M}\sum_{j=1}^{J}\sum_{s=1}^{S}\log p\left(\hat{\boldsymbol{y}}_s|\boldsymbol{x}_m,\boldsymbol{\theta}(\boldsymbol{x},\hat{\boldsymbol{y}'}_j)\right)\end{aligned} \tag{48}$$

where the approximation in the second line uses J samples of the proxy true label, $\hat{\boldsymbol{y}'}_j \sim p\left(\mathbf{y}'|\boldsymbol{x},\boldsymbol{\theta}_t\right)$, and S samples $\hat{\boldsymbol{y}}_s \sim p\left(\mathbf{y}|\boldsymbol{x}_m,\boldsymbol{\theta}_t\right)$ to approximate the two expectations. The intuition behind this AF is that it will look for points from the pool set that will result in models that 'agree' with the current distribution. Or as Roy and McCallum (2001) put it: "An example will be selected if it dramatically reinforces the learner's existing belief over unlabeled examples for which it is currently unsure." While it may seem this could cause a self-reinforcing effect, with the model selecting points that are already probable under the current model, this behavior is prevented by calculating the outer sum (over M) over the pool set, which by definition, are points that have not yet been used for training. Despite the aggressive approximations used above, the above AF is still computationally expensive as it requires the model be re-trained J times for one evaluation of the AF. Roy and McCallum (2001) get around this problem by using naive Bayes classifiers that can be built using one-pass sufficient statistics. Thus, 're-training' simply requires the sufficient statistics be changed by one count.

Uncertainty Sampling Now that we have covered an optimal (but unrealizable) AF, we turn to simpler alternatives. A very popular approach to constructing AFs is known as *uncertainty sampling*, which simply takes the point from the pool set for which the model is most uncertain. One way of quantifying this uncertainty is through the entropy of the

current model:

$$\begin{aligned}
\boldsymbol{x}^* &= \underset{\boldsymbol{x}\in\mathcal{X}_p^t}{\arg\max}\ \mathcal{A}(\boldsymbol{x};\boldsymbol{\theta}_t) \\
&= \underset{\boldsymbol{x}\in\mathcal{X}_p^t}{\arg\max}\ \mathbb{H}\left[p(\mathrm{y}|\boldsymbol{x},\boldsymbol{\theta}_t)\right] \\
&= \underset{\boldsymbol{x}\in\mathcal{X}_p^t}{\arg\max}\ -\int_{\mathbf{y}} p(\mathbf{y}|\boldsymbol{x},\boldsymbol{\theta}_t)\cdot\log p(\mathbf{y}|\boldsymbol{x},\boldsymbol{\theta}_t)\ d\mathbf{y}.
\end{aligned}$$

An often-competitive alternative is to simply look for the predictive distribution with the least-confident mode:

$$\begin{aligned}
\boldsymbol{x}^* &= \underset{\boldsymbol{x}\in\mathcal{X}_p^t}{\arg\max}\ \mathcal{A}(\boldsymbol{x};\boldsymbol{\theta}_t) \\
&= \underset{x\in\mathcal{X}_p^t}{\arg\max}\ 1\ -\ \max_{\boldsymbol{y}\in\mathcal{Y}}\ p(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}_t).
\end{aligned}$$

For binary models, recall that both their entropy and modal probability are maximized at $p(\mathbf{y}|\boldsymbol{x},\boldsymbol{\theta}_t) = 0.5$. So for logistic regression, the logistic function equals 0.5 when its input is 0, and thus uncertainty sampling looks for the point in the pool set that is closest to the decision boundary of the current model.

Query-by-Committee Another well known AF construction is Query-by-Committee (QC). This method uses a C-sized ensemble of models to perform AL: $p(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}_{1,t}),\ldots,p(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}_{C,t})$. The standard AF then checks for disagreement across these models, e.g. in a pair-wise fashion:

$$\mathcal{A}(\boldsymbol{x};\boldsymbol{\theta}_{1,t},\ldots,\boldsymbol{\theta}_{C,t})\ =\ \sum_{c=1}^{C}\sum_{j\neq c}\ \text{disagreement}\left[p(\mathbf{y}|\boldsymbol{x},\boldsymbol{\theta}_{c,t})\ ||\ p(\mathbf{y}|\boldsymbol{x},\boldsymbol{\theta}_{j,t})\right]$$

where $p(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}_{c,t})$ and $p(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}_{j,t})$ are two ensemble different models. Higher-order comparisons are possible but become expensive. For implementing the disagreement function, one simple procedures is to compare the top-ranked predictions:

$$\mathcal{A}(\boldsymbol{x};\boldsymbol{\theta}_{1,t},\ldots,\boldsymbol{\theta}_{C,t})\ =\ \sum_{c=1}^{C}\sum_{j\neq c}\ \mathbb{I}\left[\hat{\boldsymbol{y}}_c\neq\hat{\boldsymbol{y}}_j\right]$$

where $\hat{\boldsymbol{y}}_c = \arg\max_{\boldsymbol{y}\in\mathcal{Y}} p(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}_{c,t})$, the prediction from the cth model and $\mathbb{I}$ is an indicator function equal to one when its argument is true. Another example would be to compare the models via some statistical divergence function. Like uncertainty sampling, QC is also computing a notion of uncertainty but across an ensemble, not just via one model.

Bayesian Active Learning by Disagreement For a Bayesian predictive model, the most popular approach at AL is known as *Bayesian active learning by disagreement* (BALD) (MacKay, 1992; Houlsby et al., 2011). For a model

$$\mathbf{y}\sim p(\mathbf{y}|\mathbf{x},\boldsymbol{\theta}),\quad \boldsymbol{\theta}\sim p(\boldsymbol{\theta}),$$

where $p(\boldsymbol{\theta})$ is the prior, the BALD AF is the mutual information between the parameters and labels, i.e.:

$$\begin{aligned}
\mathcal{A}(\mathbf{x}; \mathcal{D}_t) &= \mathcal{I}[\mathbf{y}, \boldsymbol{\theta} | \boldsymbol{x}, \mathcal{D}_t] \\
&= \int_{\boldsymbol{\theta}} \sum_{\boldsymbol{y}} p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}_t) \log \frac{p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}_t)}{p(\boldsymbol{y}|\boldsymbol{x}, \mathcal{D}_t)\ p(\boldsymbol{\theta}|\mathcal{D}_t)}\ d\boldsymbol{\theta} \\
&= \int_{\boldsymbol{\theta}} \sum_{\boldsymbol{y}} p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{D}_t) \log \frac{p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\theta})}{p(\boldsymbol{y}|\boldsymbol{x}, \mathcal{D}_t)}\ d\boldsymbol{\theta} \\
&= \mathbb{E}_{\boldsymbol{\theta}|\mathcal{D}_t} \mathbb{KLD} \left[p(\mathbf{y}|\boldsymbol{x}, \boldsymbol{\theta}) \ || \ p(\mathbf{y}|\boldsymbol{x}, \mathcal{D}_t) \right],
\end{aligned}$$

which, as seen above, can be written as the KL divergence between the likelihood and predictive distribution, averaged over the posterior distribution. However, Houlsby et al. (2011) recommend working with an equivalent formulation, written in terms of entropy:

$$\begin{aligned}
\mathcal{A}(\mathbf{x}; \mathcal{D}_t) &= \mathcal{I}[\mathbf{y}, \boldsymbol{\theta} | \boldsymbol{x}, \mathcal{D}_t] \\
&= \mathbb{H} \left[p(\mathbf{y}|\boldsymbol{x}, \mathcal{D}_t) \right] \ - \ \mathbb{E}_{\boldsymbol{\theta}|\mathcal{D}_t} \left[\mathbb{H} \left[p(\mathbf{y}|\boldsymbol{x}, \boldsymbol{\theta}) \right] \right]
\end{aligned}$$

where the first term is the entropy of the posterior predictive distribution and the second term is the expected entropy of the likelihood, with the expectation taken over the posterior. The intuition behind this AF is that it will be maximized by the feature vector yielding the most marginal uncertainty (high-entropy posterior predictive) but for which the likelihood demonstrates certainty (low entropy) for any given setting of the parameters. Or as Houlsby et al. (2011) say: it "seek[s] the $\boldsymbol{x}$ for which the parameters under the posterior disagree about the outcome the most,"—hence the word *disagreement* in the name BALD.

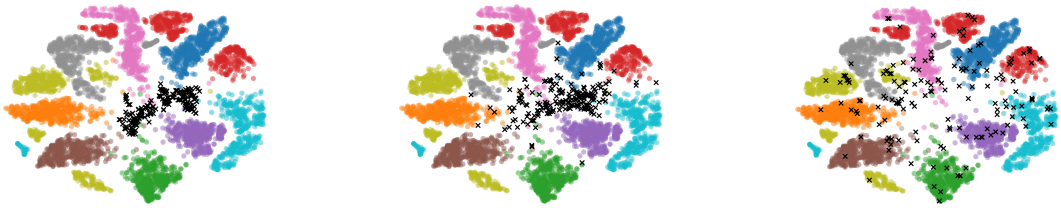
Batch Active Learning While AL that acquires single points greedily can be near-optimal in certain cases (Golovin and Krause, 2011; Dasgupta, 2005), it becomes severely limited in large-scale settings. One reason is the burden of re-training the model after every acquired data point: re-training a deep neural networks thousands of times is clearly impractical. Even if computation was not a concern, adding just one single point to the labeled set will often result in a negligible change to the updated parameters (Sener and Savarese, 2018). Moreover, since changes in the model will be small, subsequent AL steps will result in acquiring very similar points.

Due to these limitations of single-point acquisition, there is wide interest in *batch* AL methodologies. Given a maximum batch size of B, we can write the batch AL AF as:

$$\boldsymbol{X}^* \ = \ \underset{\{\boldsymbol{x}_b\}_{b=1}^B \in \mathcal{X}_p^t}{\arg\max} \ \mathcal{A}\left(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_B; \boldsymbol{\theta}_t\right), \tag{49}$$

where $\boldsymbol{X}^* = \{\boldsymbol{x}_1^*, \ldots, \boldsymbol{x}_B^*\}$ are the B points that maximize the joint AF $\mathcal{A}\left(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_B; \boldsymbol{\theta}_t\right)$. Unsurprisingly, this general formulation of batch AL is quite challenging, as it is a combinatorial optimization problem that requires checking the AF for all B-sized subsets of the pool set. Often a greedy approximation is made in which the AF is assumed to decompose point-wise:

$$\mathcal{A}\left(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_B; \boldsymbol{\theta}_t\right) \ \approx \ \sum_{b=1}^{B} \mathcal{A}\left(\boldsymbol{x}_b; \boldsymbol{\theta}_t\right). \tag{50}$$



(a) Entropy (b) BALD (c) Bayesian Coreset

Figure 1: Batch construction of different AL methods on *cifar10*, shown as a t-SNE projection (Maaten and Hinton, 2008). Given 5000 labeled points (colored by class), a batch of 200 points (black crosses) is queried.

This approximation is comparatively easy to implement: compute the AF for each point in the pool set and choose the B points who have the highest values of their AF. However, such naive batch construction methods still result in highly correlated queries Sener and Savarese (2018). We demonstrate this in Figure 1, where Subfigure (a) shows a the batch (black dots) collected by maximizing point-wise entropy and Subfigure (b) shows a batch collected by point-wise BALD. On the other hand, Subfigure (c) shows a batch AL method (Pinsler et al., 2019) that does encourage diversity across the batch. See Kirsch et al. (2019) for a batch AL extension of BALD.

Active Learning with Noisy Labels So far we have assumed that an oracle mechanism exists that can provide the true label y^*. As discussed in the previous section on crowdsourcing, such a mechanism does not always—and usually does not—exist. Thus, at each acquisition step, we may be provided with one or more noisy labels: $y_1, \dots, y_L$, where L is the number of annotators. Thus this situation would require marrying the consensus-making techniques from crowdsourcing with the AL loop. This synthesis can be done most simply by using a naive aggregation mechanism over the labels, such as majority voting. In this case, the label with the most votes would then be added to the label set as the ground-truth. On the other hand, if we are using a more sophisticated model from Section 2, the first step would be to update the label model $p(\mathrm{y}|\cdot)$. Then the updated posterior over the truth, $p(\mathrm{t}|\mathrm{y})$, can be used in several ways. One would be to take the mode of $p(\mathrm{t}|\mathrm{y})$ and use that as y^*. Note that one major departure from the usual AL framework is that, in the case of noisy labelers, we are learning more about the label noise at every AL iteration, and thus we may want to use the new observations to update the truth inference for points obtained during previous iterations. In other words, the labeled set $\mathcal{D}_t$—which we usually assume is fixed except for the newly collected point / batch—can have arbitrary changes over time, driven by the updated model of label noise. If obtaining the labels is especially costly, we may want to go a step further and only query one of multiple labelers for each AL iteration. This is the setting considered and addressed by Yan et al. (2011). They use the conditional label model $p_l(\mathrm{y}_l|\mathrm{x}, \mathrm{t})$ from Yan et al. (2010) to query the (estimated) best labeler at each AL step.

3 Imitation Learning

Imitation learning (IL) is a variant of RL in which the reward function ($\mathcal{R}$) is unknown, and instead, observations of human behavior are provided instead. It is assumed that this human behavior is drawn from a policy that is nearly (but not exactly) optimal under the unknown reward function. Below I describe variants of IL, including reductions to supervised learning and ones that allow multiple human queries.

3.1 Behavior Cloning

Behavior cloning (BC) is the simplest of IL strategies, essentially reducing the problem to that of traditional supervised learning. BC assumes access to a data set:

$$\mathcal{D} \;=\; \{(s_n, a_n)\}_{n=1}^{N}, \qquad a_t \sim \pi_E(\mathrm{a}|s_t) \tag{51}$$

where π_E is the human expert's policy. These state-action pairs do not have to be sequential in time. BC then fits a policy $\pi_{\boldsymbol{\theta}}(\mathrm{a}|\mathrm{s})$ to this dataset using traditional supervised learning:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}\in\Theta} \sum_{n=1}^{N} \ell\left(a_n, \pi_{\boldsymbol{\theta}}(\cdot|s_n)\right) \tag{52}$$

where ℓ is a suitable loss function that quantifies the distance between the expert's action and the one suggested by the policy. For example, one could use maximum likelihood estimation:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}\in\Theta} \; \mathbb{E}_{\mathrm{s}}\left[\mathbb{KLD}\left[\; \pi_E(\mathrm{a}|\mathrm{s}) \;||\; \pi_{\boldsymbol{\theta}}(\mathrm{a}|\mathrm{s}) \;\right]\right] \;\approx\; \arg\max_{\boldsymbol{\theta}\in\Theta} \sum_{n=1}^{N} \log \pi_{\boldsymbol{\theta}}(a_n|s_n) \;+\; \text{const.}$$

Notice that the expectation over states is defined by the expert interacting with the environment—not the policy we aim to learn—this is a problem in that our policy is disconnected with the underlying MDP. Thus, while easy to implement, BC suffers from some obvious limitations:

- Training the policy is disconnected from the underlying MDP. In other words, the policy never 'sees' states that are generated by itself.
- The efficacy of BC depends on having excellent coverage of the state space.
- The policy could have good supervised learning performance, but this does not guarantee good performance under the MDP. In fact, theory has shown that supervised learning error can be small, but the policy still has quadratic error w.r.t. the reward.

Hence, BC seems best suited for fine-tuning policies that are already quite performant.

We can quantify BC's sub-optimality as follows. Assume that supervised learning has succeeded such that the optimal policy and the learned policy agree in their top-ranked action with high probability:

$$\mathbb{E}_{\mathrm{s}\sim d(\pi^*)}\left[\mathbb{I}\left[\arg\max_{a\in\mathcal{A}} \pi^*(\mathrm{a}|\mathrm{s}) \neq \arg\max_{a\in\mathcal{A}} \hat{\pi}(\mathrm{a}|\mathrm{s})\right]\right] = P(\pi^* \neq \hat{\pi}|s) \leq \epsilon \tag{53}$$

where ϵ is a small non-negative constant $\epsilon \in \mathbb{R}^{\geq 0}$ and the expectation is taken w.r.t. π^*'s stationary distribution over states. We are then interested in the difference in value functions under the optimal and learned policies, which can be founded as follows:

Proposition 3.1. Assume a normalized reward function $r_t \in [-1, 1]$ and both policies are deterministic such that they choose the top-ranked action. Moreover, assume that $P(\pi^* \neq \hat{\pi}|s) \leq \epsilon$ for $\epsilon \in \mathbb{R}^{\geq 0}$. The difference in value functions is then:

$$V^{\pi^*}(s) \;-\; V^{\hat{\pi}}(s) \;\leq\; \frac{2}{(1-\gamma)^2} \cdot \epsilon$$

where $\gamma \in [0, 1)$ is the discount factor.

The upper bound is linear in the supervised learning error ϵ but quadratic in the discount, meaning that the more future rewards are considered (i.e. γ closer to one), the worse the performance gap will be when the policy is deployed in the MDP. Thus we can say $2/(1-\gamma)^2$ 'amplifies' the supervised learning error ϵ.

Before going to the proof, first note that the maximum value of the value function is when the reward is maximized at one at every state: $\sum_{t=0}^{\infty} \gamma^t r_t = \sum_{t=0}^{\infty} \gamma^t = 1/(1-\gamma)$, where the infinite series converges due to it being a standard geometric series. Similarly, the value functions can take a minimum of $-1/(1-\gamma)$. Thus $2/(1-\gamma) \geq V^{\pi^*}(s) - V^{\hat{\pi}}(s) \geq 0$ and so we'd like an upper bound that's a function of ϵ. The proof is then:

$$\begin{aligned}
V^{\pi^*}(s) - V^{\hat{\pi}}(s) &= \frac{1}{1-\gamma} \cdot \mathbb{E}_{\mathrm{s}\sim d(\pi^*)} \left[A^{\hat{\pi}}(\mathrm{s}, \arg\max_{a \in \mathcal{A}} \pi^*(a|\mathrm{s})) \right] \\
&= \frac{1}{1-\gamma} \cdot \mathbb{E}_{\mathrm{s}\sim d(\pi^*)} \left[Q^{\hat{\pi}}(\mathrm{s}, \arg\max_{a \in \mathcal{A}} \pi^*(a|\mathrm{s})) \;-\; V^{\hat{\pi}}(\mathrm{s}) \right] \\
&= \frac{1}{1-\gamma} \cdot \mathbb{E}_{\mathrm{s}\sim d(\pi^*)} \left[Q^{\hat{\pi}}(\mathrm{s}, \arg\max_{a \in \mathcal{A}} \pi^*(a|\mathrm{s})) - \sum_{a' \in \mathcal{A}} \mathbb{I}[a' = \arg\max_{a \in \mathcal{A}} \hat{\pi}(a|\mathrm{s})] \cdot Q^{\hat{\pi}}(\mathrm{s}, a') \right] \\
&= \frac{1}{1-\gamma} \cdot \mathbb{E}_{\mathrm{s}\sim d(\pi^*)} \left[Q^{\hat{\pi}}(\mathrm{s}, \arg\max_{a \in \mathcal{A}} \pi^*(a|\mathrm{s})) - Q^{\hat{\pi}}(\mathrm{s}, \arg\max_{a \in \mathcal{A}} \hat{\pi}(a|\mathrm{s})) \right] \\
&\leq \frac{1}{1-\gamma} \cdot \mathbb{E}_{\mathrm{s}\sim d(\pi^*)} \Bigg[\mathbb{I} \left[\arg\max_{a \in \mathcal{A}} \pi^*(\mathrm{a}|\mathrm{s}) \neq \arg\max_{a \in \mathcal{A}} \hat{\pi}(\mathrm{a}|\mathrm{s}) \right] \cdot \frac{2}{1-\gamma} \\
&\qquad\qquad + \mathbb{I} \left[\arg\max_{a \in \mathcal{A}} \pi^*(\mathrm{a}|\mathrm{s}) = \arg\max_{a \in \mathcal{A}} \hat{\pi}(\mathrm{a}|\mathrm{s}) \right] \cdot 0 \Bigg] \\
&= \frac{1}{1-\gamma} \cdot \mathbb{E}_{\mathrm{s}\sim d(\pi^*)} \left[\mathbb{I} \left[\arg\max_{a \in \mathcal{A}} \pi^*(\mathrm{a}|\mathrm{s}) \neq \arg\max_{a \in \mathcal{A}} \hat{\pi}(\mathrm{a}|\mathrm{s}) \right] \cdot \frac{2}{1-\gamma} \right] \\
&= \frac{1}{1-\gamma} \cdot \frac{2}{1-\gamma} \cdot \mathbb{E}_{\mathrm{s}\sim d(\pi^*)} \left[\mathbb{I} \left[\arg\max_{a \in \mathcal{A}} \pi^*(\mathrm{a}|\mathrm{s}) \neq \arg\max_{a \in \mathcal{A}} \hat{\pi}(\mathrm{a}|\mathrm{s}) \right] \right] \\
&= \frac{1}{1-\gamma} \cdot \frac{2}{1-\gamma} \cdot \epsilon \\
&= \frac{2}{(1-\gamma)^2} \cdot \epsilon.
\end{aligned}$$

where the first identify is the *performance difference lemma*. The upper-bound created in the fifth line arises from the fact that if the policies choose the same actions ($\pi^* = \pi$), then the difference in the Q-functions is zero. Otherwise, we assume the difference is maximal at

$2/(1-\gamma)$. This then reduces the problem into the probability of the policies being equal, which we have already defined to be ϵ. Perhaps a tighter bound exists by making a stronger assumption than that the difference in returns will be less than maximal.

3.2 Policy Learning via an Interactive Demonstrator

Policy Learning via an Interactive Demonstrator (PLID) is an extension of Behavior Cloning that allows the expert to be queried multiple times. Again we start with a set of demonstrations:

$$\mathcal{D}_0 \;=\; \{(s_{0,n}, a_{0,n})\}_{n=1}^{N}, \qquad a_{0,t} \sim \pi_E(\mathrm{a}|s_{0,t}). \tag{54}$$

Moreover, assume that we have used BC to fit a policy $pi_0(\mathrm{a}|\mathrm{s})$ to $\mathcal{D}_0$. PLID then proceeds with the following loop:

1. For $m = [1, M]$:
2. Rollout $\pi_{m-1}(\mathrm{a}|\mathrm{s})$ to collect a sequence of states $\{s_{m,n}\}_{n=1}^{N}$.
3. Query expert to gather corresponding actions for the observed states:

$$\mathcal{D}_m \;=\; \{(s_{m,n}, a_{m,n})\}_{n=1}^{N}, \qquad a_{m,t} \sim \pi_E(\mathrm{a}|s_{m,t}).$$

4. Apply BC to fit a policy $\pi_m(\mathrm{a}|\mathrm{s})$ to $\mathcal{D}_{0:m} = \mathcal{D}_0 \cup \ldots \cup \mathcal{D}_m$.

The PLID formulation above is known as *DAGGER* (Ross et al., 2011), as it aggregates the data collected from each loop. Alternatively, policies could be trained individually at each loop and then some form of model fusion performed. This is a better approach when $\mathcal{D}_m$ is large such that re-training on the combined data set takes a long time.

PLID solves BC's problems of being disconnected from sequential decision making (by rolling out the current policy at step #2) and possibly having limited state coverage (by re-querying the expert). We can see this explicitly by considering the behavior cloning objective after one step of interactive demonstration:

$$\hat{\boldsymbol{\theta}}_1 = \underset{\boldsymbol{\theta}\in\Theta}{\arg\min}\; \mathbb{E}_{\mathrm{s}\sim\pi_0}\left[\mathbb{KLD}\left[\;\pi_E(\mathrm{a}|\mathrm{s})\;||\;\pi_{\boldsymbol{\theta}_0}(\mathrm{a}|\mathrm{s})\;\right]\right] \;\approx\; \underset{\boldsymbol{\theta}\in\Theta}{\arg\max} \sum_{n=1}^{N} \log \pi_{\boldsymbol{\theta}_0}(a_n|s_n) \;+\; \text{const.}$$

Unlike above, where the expectation over states was under rollouts of the expert's policy, here they are states obtained by rolling out π_0 (the policy we're learning). This comes at the price, of course, of needing much more participation and effort from the expert. Moreover, if the state-space is continuous, it may be difficult for the expert to provide a demonstration exactly at the state found during the rollout. Imagine the case of finding a policy for driving a car: the car's position and the exact configuration of the controls must re-created and the expert thrust into the task of driving at that exact instant.

3.3 Distribution Matching

Distribution matching (DM) aims to solve BC's problems (namely, disconnection from sequential decision making / the underlying MDP) while not having PLID's limitation of intensive expert supervision. The key insight that differentiates DM is to consider the

joint distribution over actions *and* states, instead of just the conditional distribution of actions given states. Call a T-length sequence of states an actions a *trajectory*, denoted as $\tau = (s_0, a_1, s_1, \ldots, a_T, s_T)$, and consider a distribution over trajectories:

$$p_\pi(\boldsymbol{\tau}) = p(\mathrm{s}_0) \prod_{t=1}^{T} \pi(\mathrm{a}_t|\mathrm{s}_{t-1}) \; \mathbb{P}(\mathrm{s}_t|\mathrm{s}_{t-1}, \mathrm{a}_t) \tag{55}$$

where π is a policy and $\mathbb{P}(\mathrm{s}_{t+1}|\mathrm{s}_t, \mathrm{a}_t)$ is the MDP's transition probability. Let the probability of a trajectory under the model be denoted $p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau})$ and the expert demonstrator's distribution be denoted $p_E(\boldsymbol{\tau})$. We will not have an exact analytical form for $p_E(\boldsymbol{\tau})$; rather we see only samples

$$\mathcal{D} = \{(s_{n,0}, a_{n,1}, s_{n,1}, \ldots, a_{n,T}, s_{n,T})\}_{n=1}^{N}, \quad a_{j,t} \sim \pi_E(\mathrm{a}|s_{j,t}), \; s_{j,t+1} \sim \mathbb{P}(\mathrm{s}_{t+1}|s_{j,t}, a_{j,t}),$$

with the expert's policy $\pi_E(\mathrm{a}|s)$ again being assumed to be near optimal.

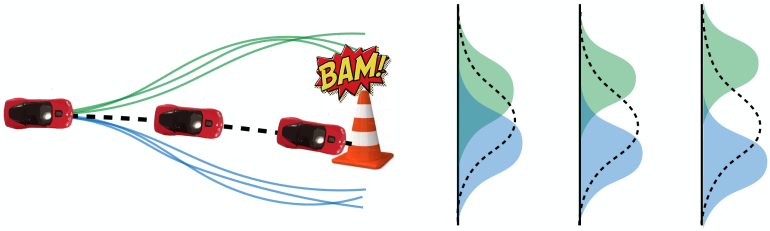


Figure 2: Imitation learning can fail when the target policy is multi-modal and the learned policy is not sufficiently expressive to cover multiple modes. Image reproduced from Ke et al. (2021).

Divergence Function Consider fitting a policy $\pi_{\boldsymbol{\theta}}$ by defining a divergence function that measures a notion of distance or dissimilarity between the model's distribution over trajectories and the expert's:

$$\mathbb{D}\left[\, p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau}) \;||\; p_E(\boldsymbol{\tau}) \,\right].$$

Specifically, Englert et al. (2013) proposes the Kullback–Leibler divergence:

$$\begin{aligned} \mathbb{KLD}\left[\, p_E(\boldsymbol{\tau}) \;||\; p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau}) \,\right] &= \sum_{\boldsymbol{\tau} \in (\mathcal{S}\times\mathcal{A})^T} p_E(\boldsymbol{\tau}) \; \log \frac{p_E(\boldsymbol{\tau})}{p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau})} \\ &\approx \sum_{n=1}^{N} -\log p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau}_n) \; + \; \text{const.} \end{aligned} \tag{56}$$

where the second line corresponds to the negative log-likelihood computed under N samples. This formulation can be thought of as similar to BC (with a log-likelihood objective) but different in that the distribution over states is modeled as well as the conditional distribution over actions. However, sampling whole trajectories can be statistically difficult (for large

T), and thus a factorization over state-action pairs is often assumed (Englert et al., 2013):

$$\begin{aligned}\mathbb{KLD}\left[\, p_E(\boldsymbol{\tau}) \;||\; p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau}) \,\right] \;&\approx\; \sum_{t=1}^{T} \mathbb{KLD}\left[\, p_E(\mathrm{s}_t, \mathrm{a}_t) \;||\; p_{\pi(\boldsymbol{\theta})}(\mathrm{s}_t, \mathrm{a}_t) \,\right] \\ &\approx\; \sum_{n=1}^{N}\sum_{t=1}^{T} -\left\{\log \pi_{\boldsymbol{\theta}}\left(a_{n,t} | s_{n,t-1}\right) \;+\; \log q_{\boldsymbol{\theta}}\left(s_{n,t}\right)\right\} \;+\; \text{const.}\end{aligned} \tag{57}$$

where $q_{\boldsymbol{\theta}}$ is a marginal distribution over states. We can think of this objective as a form of regularized BC, as we don't just want to fit the policy but also a regularizer that ensures the distribution over states matches that of the expert's trajectories. Despite the success of the above formulation, Ke et al. (2021) point out a general limitation in using the Kullback–Leibler divergence from expert to model, as p_π will be forced to place support everywhere that p_E does, and if our model is not sufficiently expressive, then we will learn solutions that try to interpolate across modes but capture none of them. Figure 2 demonstrates this problem: due to the unimodal distribution's need to cover both modes, the model tries to satisfy both modes, which causes the car to crash directly into the thing it was trying to avoid. One may then wish to turn to the reverse KLD, which is mode seeking since the expectation is now taken w.r.t. $p_{\pi(\boldsymbol{\theta})}$:

$$\mathbb{KLD}\left[\, p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau}) \;||\; p_E(\boldsymbol{\tau}) \,\right] \;=\; \sum_{\boldsymbol{\tau} \in (\mathcal{S}\times\mathcal{A})^T} p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau}) \;\log \frac{p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau})}{p_E(\boldsymbol{\tau})}. \tag{58}$$

However, this objective is hard to optimize due to needing to evaluate probability density / mass for p_π *and* p_E—the latter of which we only can observe through samples.

Adversarial Imitation Learning *Generative Adversarial Imitation Learning* (GAIL)—inspired by the similarly named *Generative Adversarial Networks* (GANs)—are one scalable approach to estimate high-dimensional, mode-seeking divergences. They operate by changing the problem to one of two-sample testing: given samples from the model $\boldsymbol{\tau}_j \sim p_\pi$ and expert $\boldsymbol{\tau}_n \sim p_E$, can a binary classifier correctly predict the source of each?

$$\begin{aligned}\mathbb{J}_{\boldsymbol{\psi}}\left(\, p_{\pi(\boldsymbol{\theta})},\; p_E \,\right) \;&=\; \mathbb{E}_E\left[-\log h\left(\boldsymbol{\tau}; \boldsymbol{\psi}\right)\right] \;+\; \mathbb{E}_\pi\left[-\log(1 - h\left(\boldsymbol{\tau}; \boldsymbol{\psi}\right))\right] \\ &\approx\; \left(\frac{1}{N}\sum_{n=1}^{N} -\log h\left(\boldsymbol{\tau}_n; \boldsymbol{\psi}\right)\right) \;+\; \left(\frac{1}{J}\sum_{j=1}^{J} -\log(1 - h\left(\boldsymbol{\tau}_j; \boldsymbol{\psi}\right))\right)\end{aligned} \tag{59}$$

where $h\left(\boldsymbol{\tau}; \boldsymbol{\psi}\right) : \tau \mapsto (0, 1)$ represents the binary classifier. $\mathbb{J}_{\boldsymbol{\psi}}$ will be minimized when the model's and expert's sample trajectories are perfectly discriminated. GAIL can also be formulated over state-action pairs, if full trajectories are too challenging, as done above:

$$\approx\; \left(\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T} -\log h\left(s_{n,t}, a_{n,t}; \boldsymbol{\psi}\right)\right) \;+\; \left(\frac{1}{J}\sum_{j=1}^{J}\sum_{t=1}^{T} -\log(1 - h\left(s_{j,t}, a_{j,t}; \boldsymbol{\psi}\right))\right). \tag{60}$$

With the adversarial objective in hand, we can use it to optimize the policy $\pi_{\boldsymbol{\theta}}$ by an adversarial max-min problem where the classifier seeks to minimize the classification loss while the policy seeks to maximize it (so that the samples look to be from indistinguishable

sources):

$$(\boldsymbol{\theta}^*, \boldsymbol{\psi}^*) = \arg\max_{\boldsymbol{\theta}} \ \arg\min_{\boldsymbol{\psi}} \ \mathbb{J}_{\boldsymbol{\psi}} \left(\ p_{\pi(\boldsymbol{\theta})}, \ p_E \ \right). \tag{61}$$

The optimal discriminator $\boldsymbol{\psi}^*$ satisfies:

$$\log h\left(\boldsymbol{\tau}; \boldsymbol{\psi}^*\right) \ - \ \log\left(1 - h\left(\boldsymbol{\tau}; \boldsymbol{\psi}^*\right)\right) \ = \ \log \frac{p_E(\boldsymbol{\tau})}{p_{\pi(\boldsymbol{\theta})}(\boldsymbol{\tau})}, \tag{62}$$

and hopefully the ratio $p_E(\boldsymbol{\tau})/p_{\pi(\boldsymbol{\theta}^*)}(\boldsymbol{\tau}) \approx 1$, meaning that the distributions have been successfully matched. In summary, GAIL is an attractive method as it does not require any instantiation of a density / mass function and can operate solely using samples from the expert and policy (i.e. rollouts). This comes at some cost to sample efficiency and a more difficult optimization problem, but those tradeoffs seem to not be a limitation in practice.

3.4 Inverse Reinforcement Learning

Inverse RL (IRL) aims to learn the *reward function* from expert demonstrations. Hence the term *inverse* since RL usually assumes the reward function is given. A policy is learned as well, using the learned reward function. Thus, learning both functions could be a challenge, but IRL assumes that learning the reward function is statistically easier than learning the policy. Specifically, we assume the reward is a parameterized function: $\mathcal{R}(\mathrm{s}_t) \approx R_{\boldsymbol{\phi}}(\mathrm{s}_t)$ where $\boldsymbol{\phi}$ are the parameters of the model used to learn the reward function. For simplicity, we assume the reward is only a function of the current state. Now the RL optimization objective is a function of both the policy's parameters ($\boldsymbol{\theta}$) and reward model's ($\boldsymbol{\phi}$):

$$\begin{aligned} \mathcal{J}(\boldsymbol{\theta}, \boldsymbol{\phi}) \ &= \ \sum_{s \in \mathcal{S}} d^{\pi(\boldsymbol{\theta})}(s) \sum_{a \in \mathcal{A}} \pi_{\boldsymbol{\theta}}(a|s) \cdot \mathbb{E}_{\pi(\boldsymbol{\theta})} \left[\ G_{\boldsymbol{\phi}} \left(\mathrm{s}_t; \{\mathrm{a}_t, \mathrm{a}_{t+1}, \ldots\} \right) \ | \ \mathrm{s}_t = s, \mathrm{a}_t = a \ \right] \\ &= \ \sum_{s \in \mathcal{S}} d^{\pi(\boldsymbol{\theta})}(s) \sum_{a \in \mathcal{A}} \pi_{\boldsymbol{\theta}}(a|s) \cdot \mathbb{E}_{\pi(\boldsymbol{\theta})} \left[\ \sum_{t'=1}^{\infty} \gamma_{t'} \cdot R_{\boldsymbol{\phi}} \left(\mathrm{s}_{t+t'} \right) \ | \ \mathrm{s}_t = s, \mathrm{a}_t = a \ \right]. \end{aligned} \tag{63}$$

As we will see below, solving this difficult optimization problem will require assuming some constraints on the policy and/or reward function. Yet in general, IRL follows the algorithmic sketch below.

1. Fit $R_{\boldsymbol{\phi}}$ to the expert demonstrations.
2. Given $R_{\boldsymbol{\phi}}$, fit $\pi_{\boldsymbol{\theta}}$ using traditional RL.
3. Compare $\pi_{\boldsymbol{\theta}}$ vs π_E, the model and expert policies.
4. If the difference in policies is sufficiently large, repeat the loop.

This sketch should make clear that IRL can be computationally expensive, since traditional RL is an *inner loop* of the procedure. Yet we must pay some price for training under a sequential setting and not assuming repeated queries to the human.

Linear Reward Function Let's start simple, by considering a linear reward function:

$$R_{\boldsymbol{\phi}}(\mathrm{s}) = \boldsymbol{\phi}^\top \boldsymbol{\psi}(\mathrm{s})$$

where $\boldsymbol{\phi} \in \mathbb{R}^D$, such that $|\boldsymbol{\phi}|_1 \leq 1$, are the parameters and $\boldsymbol{\psi} : \mathcal{S} \mapsto [0,1]^D$ is a binary vector of features that describes state $\mathbf{s}$. Under this assumption, the value function is also linear:

$$\begin{aligned} V^{\pi}(s) &= \mathbb{E}_{\pi}\left[\, G\left(\mathrm{s}_t; \{\mathrm{a}_t, \mathrm{a}_{t+1}, \ldots\}\right) \mid \mathrm{s}_t = s \,\right] \\ &= \mathbb{E}_{\pi}\left[\sum_{t'=1}^{\infty} \gamma_{t'} \cdot R_{\boldsymbol{\phi}}\left(\mathrm{s}_{t+t'}\right) \mid \mathrm{s}_t = s \right] \\ &= \mathbb{E}_{\pi}\left[\sum_{t'=1}^{\infty} \gamma_{t'} \cdot \boldsymbol{\phi}^{\top} \boldsymbol{\psi}\left(\mathrm{s}_{t+t'}\right) \mid \mathrm{s}_t = s \right] \\ &= \boldsymbol{\phi}^{\top} \underbrace{\mathbb{E}_{\pi}\left[\sum_{t'=1}^{\infty} \gamma_{t'} \cdot \boldsymbol{\psi}\left(\mathrm{s}_{t+t'}\right) \mid \mathrm{s}_t = s \right]}_{\boldsymbol{\mu}_{\pi}(s)} \end{aligned}$$

where $\boldsymbol{\mu}_{\pi}$ is a feature vector quantifying the states expected to be visited under policy π. For the expert, we see a finite set of states and thus can compute the demonstrator's empirical embedding as:

$$\boldsymbol{\mu}_E = \frac{1}{N} \sum_{n=1}^{N} \sum_{t'=1}^{\infty} \gamma_{t'} \cdot \boldsymbol{\psi}\left(s_{n,t'}\right).$$

Writing the IRL objective from Equation 63, we have:

$$\begin{aligned} \mathcal{J}(\boldsymbol{\theta}, \boldsymbol{\phi}) &= \sum_{s \in \mathcal{S}} d^{\pi(\boldsymbol{\theta})}(s) \cdot V^{\pi}(s) \\ &= \sum_{s \in \mathcal{S}} d^{\pi(\boldsymbol{\theta})}(s)\ \boldsymbol{\phi}^{\top}\ \mathbb{E}_{\pi}\left[\sum_{t'=1}^{\infty} \gamma_{t'} \cdot \boldsymbol{\psi}\left(\mathrm{s}_{t+t'}\right) \mid \mathrm{s}_t = s \right] \\ &= \boldsymbol{\phi}^{\top}\ \sum_{s \in \mathcal{S}} d^{\pi(\boldsymbol{\theta})}(s)\ \mathbb{E}_{\pi}\left[\sum_{t'=1}^{\infty} \gamma_{t'} \cdot \boldsymbol{\psi}\left(\mathrm{s}_{t+t'}\right) \mid \mathrm{s}_t = s \right] \\ &= \boldsymbol{\phi}^{\top}\ \sum_{s \in \mathcal{S}} d^{\pi(\boldsymbol{\theta})}(s)\ \boldsymbol{\mu}_{\pi}\left(s\right) \\ &= \boldsymbol{\phi}^{\top}\ \mathbb{E}_{d(s;\boldsymbol{\theta})}\left[\boldsymbol{\mu}_{\pi}\left(s\right)\right] \end{aligned} \tag{64}$$

We can then consider the difference between the objective achieved by the expert and model:

$$\begin{aligned} \mathcal{L}(\boldsymbol{\phi}) &= \left| \mathcal{J}(\boldsymbol{\theta}^{*}_{E}, \boldsymbol{\phi}) - \mathcal{J}(\boldsymbol{\theta}, \boldsymbol{\phi}) \right| \\ &= \left| \boldsymbol{\phi}^{\top}\ \boldsymbol{\mu}_E - \boldsymbol{\phi}^{\top}\ \mathbb{E}_{d(s;\boldsymbol{\theta})}\left[\boldsymbol{\mu}_{\pi}\left(s\right)\right] \right| \\ &\leq \left|\left| \boldsymbol{\mu}_E - \mathbb{E}_{d(s;\boldsymbol{\theta})}\left[\boldsymbol{\mu}_{\pi}\left(s\right)\right] \right|\right|_2^2 \end{aligned} \tag{65}$$

where the inequality arises from the assumption that $|\boldsymbol{\phi}|_1$ is bounded. Firstly, this inequality demonstrates a fundamental connection to distribution matching, as under these assumptions, matching the first moment of the state features upper-bounds $\mathcal{L}(\boldsymbol{\phi})$. Thus, minimizing the difference between the expected state features (i.e. moment matching) will minimize the gap in between the expert's and model's objectives $\mathcal{J}$, for any choice of $\boldsymbol{\phi}$.

However, simply examining the state distribution neglects learning $\boldsymbol{\phi}$, and in turn, obtaining a form for the reward function. One could question why a form for the reward is

even needed, if obtaining the policy simply by distribution matching will do, but if it is, then $\boldsymbol{\phi}$ can be obtained as follows. Abbeel and Ng (2004) propose an iterative max-margin approach that first fits $\boldsymbol{\phi}$, runs RL to find $\boldsymbol{\theta}$, and repeats. Yet a more general approach was proposed by Syed and Schapire (2007) based on an adversarial game:

$$\boldsymbol{\theta}^* \;=\; \arg\max_{\boldsymbol{\theta}} \; \min_{\boldsymbol{\phi}} \; \left(\boldsymbol{\phi}^\top \; \mathbb{E}_{d(s;\boldsymbol{\theta})} \left[\boldsymbol{\mu}_\pi \left(s \right) \right] \;-\; \boldsymbol{\phi}^\top \; \boldsymbol{\mu}_E \right). \tag{66}$$

The intuition is that, for a fixed reward (i.e. given $\boldsymbol{\theta}$), the policy can be chosen so that the model achieves a better reward than the expert achieved. However, the environment is then free to change the reward in order to minimize the quantity, thus choosing in favor of the expert. This bares some similarity to GAIL, with $\boldsymbol{\phi}$ acting in the spirit of the discriminator. Yet, of course, now the 'discriminator' has the interpretation as a reward function.

Another approach to IRL of note is *maximum entropy inverse reinforcement learning* (MaxEnt IRL). This approach provides a more explicit bridge between distribution matching and previous IRL approach. Let the state features for one of the expert's trajectories be denoted $\boldsymbol{\mu}_{E,n} = \sum_{t'=1} \gamma_{t'} \cdot \boldsymbol{\psi}(s_{n,t'})$, such that $\boldsymbol{\mu}_E = (1/N) \sum_n \boldsymbol{\mu}_{E,n}$. MaxEnt IRL then models the probability of a trajectory by exponentiating the linear reward model from above:

$$p\left(\boldsymbol{\tau}_n; \boldsymbol{\phi}\right) = \frac{1}{Z(\boldsymbol{\phi})} \; \exp\left\{ \boldsymbol{\phi}^\top \boldsymbol{\mu}_{E,n} \right\}, \quad Z(\boldsymbol{\phi}) = \int_{\boldsymbol{\tau} \in (\mathcal{S},\mathcal{A})^T} \exp\left\{ \boldsymbol{\phi}^\top \boldsymbol{\mu}_{\boldsymbol{\tau}} \right\} \; d\boldsymbol{\tau} \tag{67}$$

where $Z(\boldsymbol{\phi})$ is known as the partition function (or normalizing constant) that ensures the probability is normalized by computing the exponentiated reward over all possible states. Now consider optimizing $\boldsymbol{\phi}$ via gradient ascent of the log-likelihood under the expert's demonstrations. The gradient calculation is:

$$\begin{aligned}
\nabla_{\boldsymbol{\phi}} \frac{1}{N} \sum_{n=1}^{N} \log p\left(\boldsymbol{\tau}_n; \boldsymbol{\phi}\right) \;&=\; \frac{1}{N} \nabla_{\boldsymbol{\phi}} \sum_{n=1}^{N} \left[\boldsymbol{\phi}^\top \boldsymbol{\mu}_{E,n} \;-\; \log Z(\boldsymbol{\phi}) \right] \\
&=\; \frac{1}{N} \sum_{n=1}^{N} \nabla_{\boldsymbol{\phi}} \left[\boldsymbol{\phi}^\top \boldsymbol{\mu}_{E,n} \right] \;-\; \frac{N}{N} \nabla_{\boldsymbol{\phi}} \log Z(\boldsymbol{\phi}) \\
&=\; \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{\mu}_{E,n} \;-\; \frac{1}{Z(\boldsymbol{\phi})} \int_{\boldsymbol{\tau} \in (\mathcal{S},\mathcal{A})^T} \nabla_{\boldsymbol{\phi}} \exp\left\{ \boldsymbol{\phi}^\top \boldsymbol{\mu}_{\boldsymbol{\tau}} \right\} \; d\boldsymbol{\tau} \\
&=\; \boldsymbol{\mu}_E \;-\; \frac{1}{Z(\boldsymbol{\phi})} \int_{\boldsymbol{\tau} \in (\mathcal{S},\mathcal{A})^T} \exp\left\{ \boldsymbol{\phi}^\top \boldsymbol{\mu}_{\boldsymbol{\tau}} \right\} \; \boldsymbol{\mu}_{\boldsymbol{\tau}} \; d\boldsymbol{\tau} \\
&=\; \boldsymbol{\mu}_E \;-\; \int_{\boldsymbol{\tau} \in (\mathcal{S},\mathcal{A})^T} p\left(\boldsymbol{\tau}; \boldsymbol{\phi}\right) \; \boldsymbol{\mu}_{\boldsymbol{\tau}} \; d\boldsymbol{\tau} \\
&=\; \boldsymbol{\mu}_E \;-\; \mathbb{E}_{\tau|\phi} \left[\boldsymbol{\mu}_{\boldsymbol{\tau}} \right]
\end{aligned} \tag{68}$$

where $\boldsymbol{\mu}_E$ is the expert's state features (over all trajectories) and $\mathbb{E}_{\tau|\phi} \left[\boldsymbol{\mu}_{\boldsymbol{\tau}} \right]$ is the expected state features under $p\left(\boldsymbol{\tau}; \boldsymbol{\phi}\right)$. Note the similarity to the upper-bound above where the expert's and policy's expected features are matched.

Yet also note that no policy has been introduced here. To parameterize a policy, MaxEnt IRL now assumes the following implied policy: $\pi(\mathrm{a}|\mathrm{s}) \;\propto\; Q^\pi(\mathrm{s}, \mathrm{a})$, which is simply the policy that chooses actions with probability proportional to their expected return. Now consider that $p\left(\boldsymbol{\tau}; \boldsymbol{\phi}\right)$ also places high probability on states with the highest rewards. Thus the

contribution of the partition function in the gradient is approximated with a policy as:

$$\mathbb{E}_{\tau|\phi}\left[\boldsymbol{\mu}_{\boldsymbol{\tau}}\right] \;\approx\; \mathbb{E}_{d(s;\boldsymbol{\theta})}\left[\boldsymbol{\mu}_{\pi}\left(s\right)\right] \tag{69}$$

where the RHS term is the expected state features under policy $\pi_{\boldsymbol{\theta}}$(a|s). Mechanistically, this is doing the correct thing as the gradient will be zero when $\mathbb{E}_{d(s;\boldsymbol{\theta})}\left[\boldsymbol{\mu}_{\pi}\left(s\right)\right]$ and $\boldsymbol{\mu}_{E}$ have been matched, i.e. their difference is zero. The full gradient update can be computed as:

$$\boldsymbol{\phi}_{t+1} \;=\; \boldsymbol{\phi}_{t} \;+\; \alpha \cdot \left(\boldsymbol{\mu}_{E} \;-\; \mathbb{E}_{d(s;\boldsymbol{\theta}_{t})}\left[\boldsymbol{\mu}_{\pi}\left(s\right)\right]\right)$$

where $\boldsymbol{\theta}_t$ is obtained by running RL using the reward function $R_{\boldsymbol{\phi}_t}(\mathrm{s}) = \boldsymbol{\phi}_t^{\top}\boldsymbol{\psi}(\mathrm{s})$.

3.5 Reinforcement Learning with Human Feedback

Reinforcement Learning with Human Feedback (RLHF) (Christiano et al., 2017) is a form of IRL that has been made popular by its success in finetuning ChatGPT. Like previous methods, its core goal is to (i) learning a reward function from human demonstrations, and then (2) train (or more often, finetune an already trained model) using the learned reward function as the learning signal. Unlike previous imitation learning strategies discussed above, RLHF assumes we have access to ranked or paired demonstrations:

$$\mathcal{D}_{+,-} \;=\; \left\{\left(\boldsymbol{\tau}_n^{+}, \boldsymbol{\tau}_n^{-}\right)\right\}_{n=1}^{N}$$

where τ_n^+ is a positive trajectory (i.e. a sequences of states and actions) that has been deemed by a human to encode more desirable behavior than the negative trajectory τ_n^-. These trajectory pairs could be describing a similar state in the environment or could simply be randomly paired by have two large batches of positive and negative demonstrations.

Given these ranked trajectories, the next step is to define a reward model $R_{\boldsymbol{\phi}} : \mathbf{T} \mapsto \mathbb{R}^{\geq 0}$ with parameters $\boldsymbol{\phi}$. We then define a *Bradley-Terry model* (Bradley and Terry, 1952) that encodes the probability of one trajectory being better than another, as a function of the reward model:

$$p_{\boldsymbol{\phi}}(\boldsymbol{\tau}^{+} \succ \boldsymbol{\tau}^{-}) \;=\; \sigma\left(R\left(\boldsymbol{\tau}^{+};\boldsymbol{\phi}\right) \;-\; R\left(\boldsymbol{\tau}^{-};\boldsymbol{\phi}\right)\right) \tag{70}$$

where $\sigma(\cdot)$ denotes the logistic function. Thus we see that the probability of one trajectory being preferable over another is simply the difference in their reward functions normalized to $(0, 1)$. Using this model, we can then define likelihood of the reward parameters $\boldsymbol{\phi}$:

$$\begin{aligned}
\ell\left(\boldsymbol{\phi};\mathcal{D}_{+,-}\right) \;&=\; \log\left\{\prod_{n=1}^{N} p_{\boldsymbol{\phi}}(\boldsymbol{\tau}_n^{+} \succ \boldsymbol{\tau}_n^{-})\right\} \\
&=\; \sum_{n=1}^{N} \log p_{\boldsymbol{\phi}}(\boldsymbol{\tau}_n^{+} \succ \boldsymbol{\tau}_n^{-}) \\
&=\; \sum_{n=1}^{N} \log \sigma\left(R\left(\boldsymbol{\tau}_n^{+};\boldsymbol{\phi}\right) \;-\; R\left(\boldsymbol{\tau}_n^{-};\boldsymbol{\phi}\right)\right).
\end{aligned}$$

Usually the reward function is taken to be a neural network of some form and $\ell\left(\boldsymbol{\phi};\mathcal{D}_{+,-}\right)$ is optimized with gradient ascent. After the reward model is fit, then $R\left(\boldsymbol{\tau}_n^{+};\boldsymbol{\phi}\right)$ can be

plugged into any suitable RL framework to learn a policy.

Difference from Traditional Inverse RL Notice that this setup is much simpler than the inverse RL formulations above, which usually require additional assumptions about the form of the reward function or how it can be learned in tandem with a policy. In other words, before we only has *positive* demonstrations and thus couldn't train a reward function on them directly since it has no exampled of what negative behaviors might look like. One had to assume that roughly everything not in the demonstration set was a negative behavior. Here have positive and negative trajectories allows the reward model to see both extremes and thus be learned directly without an inner-loop of RL / policy fitting.

Why won't Behavior Cloning suffice? Given that we have demonstrations from reliable humans, it is tempting to also consider a behavior cloning objective that uses both positive and negative examples:

$$\hat{\theta} = \arg\min_{\theta \in \Theta} \; \mathbb{E}_{\boldsymbol{\tau}^-} \left[\log \pi_{\theta}(\mathrm{a}^-|\mathrm{s}^-) \right] - \mathbb{E}_{\boldsymbol{\tau}^+} \left[\log \pi_{\theta}(\mathrm{a}^+|\mathrm{s}^+) \right] \tag{71}$$

where this objective aims to directly maximize the probability of the positive trajectories while minimizing the probability of the negative trajectories under the policy. This could work in the usual cases in which BC succeeds (such as a small state space). Yet, especially in domains such as language in which there are multiple equivalent solutions, interacting with the underlying MDP allows a policy to discover these symmetries instead of overfitting to precisely what's given in the positive demonstrations.

Direct Preference Optimization One method that bridges RLHF and behavior cloning is *direct preference optimization* (DPO) (Rafailov et al., 2023). It works as follows. Firstly, they notice that in RLHF, the final policy is usually trained with a regularized objective, to keep the final policy near the initial (assuming some good scheme exists for pre-training the policy, like next-token modeling with language):

$$\boldsymbol{\theta}^* \;=\; \arg\max_{\theta \in \Theta} \mathbb{E}_{\boldsymbol{\tau} \sim \pi_{\theta}} \left[R\left(\boldsymbol{\tau}; \boldsymbol{\phi}\right) \;-\; \mathbb{KLD}\left[\pi_{\theta}(\mathrm{a}|\mathrm{s}) || \pi_0(\mathrm{a}|\mathrm{s}) \right] \right].$$

Under this optimization problem, the optimal policy has the form:

$$\pi^*(\mathrm{a}|\mathrm{s}) \;=\; \pi_0(\mathrm{a}|\mathrm{s}) \cdot \frac{1}{\mathcal{Z}(\boldsymbol{\phi})} \cdot \exp\left\{ R\left(\boldsymbol{\tau}; \boldsymbol{\phi}\right) \right\}, \quad \mathcal{Z}(\boldsymbol{\phi}) \;=\; \sum_{\boldsymbol{\tau} \in \mathbf{T}} \exp\left\{ R\left(\boldsymbol{\tau}; \boldsymbol{\phi}\right) \right\} \sum_{(s,a) \in \boldsymbol{\tau}} \pi_0(a|s).$$

Solving for the reward function (outside of $\mathcal{Z}(\boldsymbol{\phi})$), we then have:

$$R\left(\boldsymbol{\tau}; \boldsymbol{\phi}\right) \;=\; \sum_{(\mathrm{s},\mathrm{a}) \in \boldsymbol{\tau}} \log \frac{\pi^*(\mathrm{a}|\mathrm{s})}{\pi_0(\mathrm{a}|\mathrm{s})} \;+\; \log \mathcal{Z}(\boldsymbol{\phi}).$$

Plugging in this form for the reward function into the Bradley-Terry model, we have:

$$
\begin{aligned}
p_{\boldsymbol{\phi}}(\boldsymbol{\tau}^+ \succ \boldsymbol{\tau}^-) \;&=\; \sigma\left(R\left(\boldsymbol{\tau}^+;\boldsymbol{\phi}\right) \;-\; R\left(\boldsymbol{\tau}^-;\boldsymbol{\phi}\right)\right) \\
&=\; \sigma\left(\sum_{(\mathrm{s}^+,\mathrm{a}^+)\in\boldsymbol{\tau}^+} \log\frac{\pi^*(\mathrm{a}^+|\mathrm{s}^+)}{\pi_0(\mathrm{a}^+|\mathrm{s}^+)} \;+\; \log\mathcal{Z}(\boldsymbol{\phi}) \;-\; \sum_{(\mathrm{s}^-,\mathrm{a}^-)\in\boldsymbol{\tau}^-} \log\frac{\pi^*(\mathrm{a}^-|\mathrm{s}^-)}{\pi_0(\mathrm{a}^-|\mathrm{s}^-)} \;-\; \log\mathcal{Z}(\boldsymbol{\phi})\right) \\
&=\; \sigma\left(\sum_{(\mathrm{s}^+,\mathrm{a}^+)\in\boldsymbol{\tau}^+} \log\frac{\pi^*(\mathrm{a}^+|\mathrm{s}^+)}{\pi_0(\mathrm{a}^+|\mathrm{s}^+)} \;-\; \sum_{(\mathrm{s}^-,\mathrm{a}^-)\in\boldsymbol{\tau}^-} \log\frac{\pi^*(\mathrm{a}^-|\mathrm{s}^-)}{\pi_0(\mathrm{a}^-|\mathrm{s}^-)}\right).
\end{aligned}
\tag{72}
$$

where the difficult-to-compute term $\mathcal{Z}(\boldsymbol{\phi})$ cancels out. This equation is in terms of the optimal policy π^* for a specific (implied) reward model $R\left(\boldsymbol{\tau}^+;\boldsymbol{\phi}\right)$, but we can instead directly parameterize the policy to devise a learning objective that by-passes learning a reward function as an intermediate step:

$$
\begin{aligned}
\ell\left(\boldsymbol{\theta};\mathcal{D}_{+,-}\right) \;&=\; \sum_{n=1}^{N} \log p(\boldsymbol{\tau}_n^+ \succ \boldsymbol{\tau}_n^-) \\
&=\; \sum_{n=1}^{N} \log\sigma\left(\sum_{(s^+,a^+)\in\boldsymbol{\tau}_n^+} \log\frac{\pi_{\boldsymbol{\theta}}(a^+|s^+)}{\pi_0(\mathrm{a}^+|\mathrm{s}^+)} \;-\; \sum_{(s^-,a^-)\in\boldsymbol{\tau}_n^-} \log\frac{\pi_{\boldsymbol{\theta}}(a^-|s^-)}{\pi_0(a^-|s^-)}\right).
\end{aligned}
$$

Removing the pre-trained policy π_0, we have:

$$
\ell\left(\boldsymbol{\theta};\mathcal{D}_{+,-}\right) \;=\; \sum_{n=1}^{N} \log\sigma\left(\sum_{(s^+,a^+)\in\boldsymbol{\tau}_n^+} \log\pi_{\boldsymbol{\theta}}(a^+|s^+) \;-\; \sum_{(s^-,a^-)\in\boldsymbol{\tau}_n^-} \log\pi_{\boldsymbol{\theta}}(a^-|s^-)\right).
$$

Going back to the behavior cloning approach in Equation 71, we see that these objectives are very similar, and the only material difference is that DPO wraps the policy terms inside the logsistic function. We can get a better understanding of the mechanics by looking at the gradient:

$$
\begin{aligned}
&\nabla_{\boldsymbol{\theta}}\ell\left(\boldsymbol{\theta};\mathcal{D}_{+,-}\right) \;= \\
&\sum_{n=1}^{N}(1-p(\boldsymbol{\tau}_n^+ \succ \boldsymbol{\tau}_n^-))\left[\sum_{(s^+,a^+)\in\boldsymbol{\tau}_n^+} \nabla_{\boldsymbol{\theta}}\log\pi_{\boldsymbol{\theta}}(a^+|s^+) \;-\; \sum_{(s^-,a^-)\in\boldsymbol{\tau}_n^-} \nabla_{\boldsymbol{\theta}}\log\pi_{\boldsymbol{\theta}}(a^-|s^-)\right],
\end{aligned}
$$

and thus we see the gradient of the difference in the policies is weighted by one minus the probability of the positive trajectory being preferred. Thus the policy is updated only when the implied reward model has the incorrect preference and cannot continue to optimize the policy forever to overfit on the positive trajectory. This is experimentally confirmed by Rafailov et al. (2023), as their experiments show the model does not perform well without the $(1-p(\boldsymbol{\tau}_n^+ \succ \boldsymbol{\tau}_n^-))$ term.

4 Human-AI Collaboration

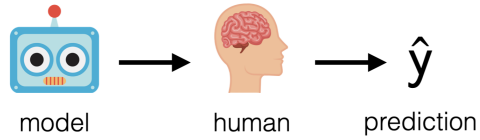
These notes so far have only considered the setting in which the human generates a training signal of some form. Once this is done and the model trained, it is assumed the usual predictive modeling workflow will be followed, with the model acting autonomously in the world. We now move on to the setting in which, even after deployment, the human and model work in tandem. There are not yet precise terms and definitions for human-AI collaboration, so these notes may use terms like "human-AI collaboration," "human-AI cooperation,", "human-AI teaming" or "hybrid intelligence" interchangeably, reflecting their use in the literature. No matter the name, human-AI collaboration (HAIC) must satisfy at least one of the following two criteria in order to be well-motivated:

- *Synergy*: The human-AI team must perform the task better than the human or AI model could perform the task alone.
- *Semi-Automation*: The human-AI team must perform the task approximately no worse than the human would perform it alone, while having the benefit of automation. The degree of 'approximately no worse' depends on the gains had by introducing automation.

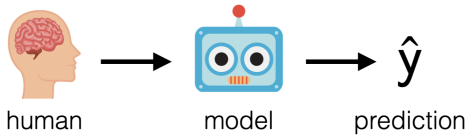
If the HAIC is performing worse than the human would alone and then there is no benefit to the automation, then the use of HAIC is not justified. Similarly, if a fully-automated system performs at an acceptable level, then there is no point to involving a human. Often humans are preferred, even when models have better benchmark performance, due to safety and risk concerns. In the above presentation, I consider the use of 'performance' to include other aspects and metrics, such as safety, that factor into whether a model is deployed.

4.1 Design Patterns

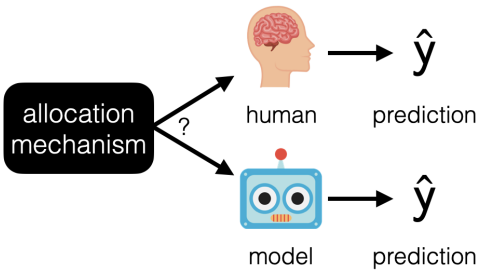
Most methods for HAIC follow or are compositions of four basic design patterns. They are shown in Figure 3. Pattern #1 in Subfigure 3a is likely most common in practice today, as giving the human final control is attractive from a safety and policy standpoint. Yet this has the downside that there is no improvements to automation. Rather we only expect the principle of *synergy*: the human's performance is improved by, at times, considering the model's output. Pattern #2 (Subfigure 3b) represents the inverse workflow, with the human informing the model's decision and the machine having the final say. This pattern is uncommon in the case of the human *always* providing advice to the machine. If the human is always engaged in this way, it likely makes better sense to use pattern #1. However, if the human is only sometimes providing input, then this pattern can be thought of as a form of semi-automation, with the model acting fully autonomously at times and being helped by the human at other times. Pattern #3 (Subfigure 3c) differs from the first two as it does not represent a sequential setting. Rather, decisions are allocated to either the human or machine, with presumably the riskier or more difficult decisions going to the human. This pattern enjoys the potential for both HAIC goal criteria: semi-automation cab be achieved since the human does not have to be involved in all decisions, and synergy can be achieved if the human and model have complementary strengths. Lastly, pattern #4 (Subfigure 3d) shows an alternative design in which the human and machine are independently queried



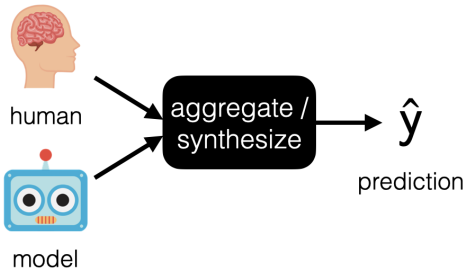
(a) Pattern #1: Human's prediction is informed by the model's.



(b) Pattern #2: Model's prediction is informed by the human's.



(c) Pattern #3: An allocation mechanism decides if the human or model should make the prediction.



(d) Pattern #4: Human's and model's predictions are made independently and combined.

Figure 3: *Design Patterns for Human-AI Collaboration.* The four diagrams above show common design patterns for how to combine human and machine decisions / predictions.

and their predictions / decisions combined to form a final, hopefully superior, one. Semi-automation is not possible here since the human is assumed to be queried for all test-time decisions.

4.2 Combining Predictions from Humans and Models

I now introduce a model that falls into design pattern #4 (Subfigure 3d): the human and model jointly produce a prediction for every case at test-time. As mentioned above, such a workflow clearly has no benefits of automation (since the human is always engaged in decision making), and thus the benefit must be that of synergy (better performance than the human or model could achieve alone). Consider the usual setting of multi-class classification, where $\mathbf{x} \in \mathcal{X}$ denotes the features and $\mathrm{y} \in \mathcal{Y}$ denotes the labels, of there there are K possibilities. Let $\mathcal{D} = \{(\boldsymbol{x}_n, y_n\}_{n=1}^N$ denote the training examples. Assume we have already trained a classifier on this data set: $f : \mathcal{X} \mapsto \Delta^K$. We can then parameterize a categorical distribution over labels, denoted $p(\mathrm{y}|f(\mathbf{x}))$. Moreover, assume we have access to a human who, when shown a feature vector, will output a classification label: $h : \mathcal{X} \mapsto \mathcal{Y}$. We do not ask the human for any additional information, such as their confidence in the prediction, as we deem it to be unreliable and/or for the sake of generality. We also assume that the cost of query the human is inconsequential. Given the model f and the human h, how might we fuse or ensemble their predictions into a final prediction that is more reliable and performant that either predictor alone? Of course, if $h(\mathbf{x}) = \arg\max_y f_y(\mathbf{x})$, then this is a good sign that they have converged to the correct prediction. But what if they do not agree?

Kerrigan et al. (2021a) propose the following model for this setting, with the key insight being that we need to obtain some notion of confidence for the human. This can be done by estimating a *confusion matrix* for them on the training set $\mathcal{D}$. Let Π denote this matrix

such that $\pi_{k,j} \in [0,1]$ and $\sum_{j=1}^{K} \pi_{k,j} = 1$. We then collect the human's predictions for every element of $\mathcal{D}$—$\{m_n = h(\boldsymbol{x}_n)\}_{n=1}^{N}$—and estimate the confusion matrix's parameter via maximum likelihood estimation: $\hat{\pi}_{k,j} = \sum_{n=1}^{N}(\mathbb{I}[y_n = k] \cdot \mathbb{I}[m_n = j])/N_k$ where N_k are the number of points in the training set that have class k.

With this confusion matrix in hand, we can combine the predictions via the following probabilistic model:

$$\begin{aligned} p(\mathrm{y}|h(\mathbf{x}), f(\mathbf{x})) &= \frac{p(h(\mathbf{x}), \mathrm{y}|f(\mathbf{x}))}{p\left(h(\mathbf{x})|f(\mathbf{x})\right)} \\ &= \frac{p(h(\mathbf{x})|\mathrm{y}, f(\mathbf{x})) \cdot p(\mathrm{y}|f(\mathbf{x}))}{p\left(h(\mathbf{x})|f(\mathbf{x})\right)} \\ &\approx \frac{p(h(\mathbf{x})|\mathrm{y}) \cdot p(\mathrm{y}|f(\mathbf{x}))}{p\left(h(\mathbf{x})|f(\mathbf{x})\right)} \end{aligned} \tag{73}$$

where the first line is obtained via Bayes rule and the second assumes that the first term on the LHS is independent of the classifier: $p(h(\mathbf{x})|y, f(\mathbf{x})) \approx p(h(\mathbf{x})|\mathrm{y})$. This is a reasonable assumption if the human has not looked at the classifier's output when making her prediction. Plugging in the confusion matrix entry for the first term and the classifier's softmax confidence for the second, we have:

$$\begin{aligned} p(\mathrm{y} = y|h(\mathbf{x}), f(\mathbf{x})) &\approx \frac{p(h(\mathbf{x})|\mathrm{y} = y) \cdot p(\mathrm{y} = y|f(\mathbf{x}))}{\sum_{y'} p(h(\mathbf{x})|\mathrm{y} = y') \cdot p(\mathrm{y} = y'|f(\mathbf{x}))} \\ &= \frac{\hat{\pi}_{y,h(\boldsymbol{x})} \cdot f_y(\mathbf{x})}{\sum_{y'} \hat{\pi}_{y',h(\boldsymbol{x})} \cdot f_{y'}(\mathbf{x})}. \end{aligned} \tag{74}$$

We can interpret this expression as re-weighting the classifier's outputs by the confusion matrix. However, for this formulation to work, the classifier must be reasonably well calibrated. In the pathological case, if $f_y(\mathbf{x}) = 1$, then the human's prediction will not influence the posterior: $(\hat{\pi}_{y,h(\boldsymbol{x})} \cdot f_y(\boldsymbol{x}))/(\hat{\pi}_{y,h(\boldsymbol{x})} \cdot f_y(\boldsymbol{x})) = 1$. Thus, prior to combination, Kerrigan et al. (2021b) apply temperature scaling to calibrate $f(\mathbf{x})$.

4.3 AI-Assisted Decision Making

Instead of the above workflow in which human and machine predictions are combined to produce the final prediction, a more common design pattern is to have a model's output or prediction *inform* but not *dictate* a human's decision. This workflow is more palatable from the standpoint of regulation and policy since the human retains ultimate control (and blame). Returning to the four design patterns that started this section, this workflow is an instance of design pattern #1 (Subfigure 3a). Again there are no gains in automation, since the human is involved in every decision, and thus the goal is to have the AI-human combination perform better than either could alone.

At first glance, one may assume this workflow is trivial to implement. The human simply looks at the confidence scores associated with each class: $f(\mathbf{x}) = [30\%, 40\%, 30\%]$. Assuming the classifier is sufficiently calibrated, the human is then free to propagate that uncertainty into their final decision, trusting the classifier to whatever degree the confidence statements warrant trust. In an ideal world, the classifier would also output some notion of interpretability or explainability for its decisions, but this often gets into application-specific

models and workflows. For the sake of generality, we will just assume confidence statements are given. Unfortunately, humans are known not to be well-calibrated decision makers themselves and often misinterpret probabilistic statements (Kahneman and Tversky, 1972). In turn, the confidence statements output by the classifier can range from being useless to even being misinterpreted by the human.

4.3.1 Recalibrating to Improve Human Performance

Vodrahalli et al. (2022) addressed this issue by modeling the AI-assistance process in a binary task in which the human is asked to give a confidence score of a statement being true. Such a model allows the end-to-end decision-making process to be optimized to improve the human's decision making via re-calibrating the AI's output. I reproduce *the spirit* of Vodrahalli et al. (2022)'s model below (e.g. the two step nature) but with some of my own modifications. Let $\mathbf{x} \in \mathcal{X}$ denote the feature space, and $\mathrm{y} \in \{0, 1\}$ denote the label space (i.e. a binary prediction task). Let $h(\mathbf{x}) \in [0, 1]$ denote the human's *confidence* that $y = 1$, let $f(\mathbf{x}) \in [0, 1]$ denote the AI confidence score, and let $h\left(\mathbf{x}, f(\mathbf{x})\right) \in [0, 1]$ denote the human's confidence score after seeing the model output. The goal will be to learn a transformation of the model's output that results in the human making better decisions:

$$g_{\alpha,\beta} \circ f(\mathbf{x}) \;=\; \texttt{logistic}(\alpha \cdot \texttt{logit}(f(\mathbf{x})) + \beta) \;=\; \frac{1}{1 + \exp\left\{-\alpha \cdot \texttt{logit}(f(\mathbf{x})) - \beta\right\}}$$

where $\alpha, \beta \in \mathbb{R}$ are the parameters to be learned. This transformation is a well-known re-calibration method known as *Platt scaling* (Platt, 1999). Essentially, it re-calibrates by applying logistic regression to the model's original logit values.

Activation Model The model takes the form of a two-step process, with the first step being to predict if the human will be 'activated,' meaning that their prediction will incorporate and therefore be influenced by the model's output:

$$\mathrm{a} \sim \text{Bernoulli}\left(\; p_{\mathrm{a}} = \phi(\mathbf{x}, h(\mathbf{x}), g_{\alpha,\beta} \circ f(\mathbf{x}))\;\right)$$

where $\mathrm{a} \in \{0, 1\}$ is the random variable denoting activation. If $a = 1$, then we should not have that $h(\mathbf{x})$ and $h\left(\mathbf{x}, f(\mathbf{x})\right)$ are close in value. $\phi(\cdot, \cdot, \cdot) \in (0, 1)$ is the model (e.g. a neural network with logistic output) that parameterizes the Bernoulli success probability p_{a}. ϕ is pre-trained using the original AI confidence score $f(\mathbf{x})$ and fixed thereafter.

Integration Model The second step is to determine the human's final prediction with an 'integration' model. The activation variable is essentially used as a 'switch' that determines if the final prediction will be the one the human produced before or after seeing the AI's output:

$$\mathrm{m} \;=\; (1 - \mathrm{a}) \cdot h(\mathbf{x}) \;+\; \mathrm{a} \cdot h\left(\mathbf{x}, g_{\alpha,\beta} \circ f(\mathbf{x})\right).$$

Unfortunately, the final prediction depends upon $h\left(\mathbf{x}, g_{\alpha,\beta} \circ f(\mathbf{x})\right)$—what the human would predict if the model is re-calibrated with a particular value of α and β. To evaluate this exactly, we would need to ask the human to produce confidence scores for every step of an iterative optimization procedure. One option is to train a *surrogate model* of the human's

decision-making process using the original $f(\mathbf{x})$ scores, similarly to how ϕ is trained:

$$h\left(\mathbf{x}, g_{\alpha,\beta} \circ f(\mathbf{x})\right) \approx \psi\left(\mathbf{x}, h(\mathbf{x}), g_{\alpha,\beta} \circ f(\mathbf{x})\right)$$

where the $h(\mathbf{x})$ term is also included as input so that the surrogate only needs to model $\Delta_h = h\left(\mathbf{x}, g_{\alpha,\beta} \circ f(\mathbf{x})\right) - h(\mathbf{x})$. A simpler but surely biased alternative is to just use the human's original scores: $h\left(\mathbf{x}, g_{\alpha,\beta} \circ f(\mathbf{x})\right) \approx h\left(\mathbf{x}, f(\mathbf{x})\right)$. In this case, optimizing α, β would only account for the activation potential rather than the human's precise score. We will proceed with the surrogate-based approach.

Training Objective Given true labels $\mathrm{y} \in \{0, 1\}$, we can minimize the expected *Brier score*, with the expectation taken over the activation variable (and data):

$$\begin{aligned}
\ell(\alpha, \beta) \;&=\; \mathbb{E}_{\mathbf{x},\mathrm{y}}\mathbb{E}_a\left[\left(\mathrm{y} - \mathrm{m}\right)^2\right] \;=\; \mathbb{E}_{\mathbf{x},\mathrm{y}}\left[\left(\mathrm{y} - \mathbb{E}_{\mathrm{a}}\left[\mathrm{m}\right]\right)^2 + \mathrm{Var}_{\mathrm{a}}\left[\mathrm{m}\right]\right] \\
&\approx\; \frac{1}{N}\sum_{n=1}^{N}\left(y_n - \mathbb{E}_{\mathrm{a}_n}\left[\mathrm{m}_n\right]\right)^2 + \mathrm{Var}_{\mathrm{a}_n}\left[\mathrm{m}_n\right], \\
\text{where} \quad \mathbb{E}_{\mathrm{a}}\left[\mathrm{m}\right] &= (1 - p_{\mathrm{a}}) \cdot h(\mathbf{x}) \;+\; p_{\mathrm{a}} \cdot \psi\left(\mathbf{x}, g_{\alpha,\beta} \circ f(\mathbf{x})\right), \\
\mathrm{Var}_{\mathrm{a}}\left[\mathrm{m}\right] &= p_{\mathrm{a}}(1 - p_{\mathrm{a}})\left(h(\mathbf{x}) - \psi\left(\mathbf{x}, g_{\alpha,\beta} \circ f(\mathbf{x})\right)\right)^2.
\end{aligned} \tag{75}$$

The approximation arises from having a finite training set of N samples. We can interpret the first term, $\left(\mathrm{y} - \mathbb{E}_{\mathrm{a}}\left[\mathrm{m}\right]\right)^2$, as representing the Brier score of the expected final prediction. The Brier score is a well-motivated choice, as is it a common approach to quantify forecasting error. The second term, $\mathrm{Var}_{\mathrm{a}_n}\left[\mathrm{m}_n\right]$, is more interesting. Recall that we want to minimize this objective and therefore we can minimize the variance term in two ways. The first way is to minimize $\left(h(\mathbf{x}) - \psi\left(\mathbf{x}, g_{\alpha,\beta} \circ f(\mathbf{x})\right)\right)^2$, the squared difference between the human's predictions with and without the machine's advice (as approximated by the surrogate). When this is small, the whole $\mathrm{Var}_{\mathrm{a}_n}\left[\mathrm{m}_n\right]$ term is small and therefore has little effect. Yet when the gap between the predictions is large, then the whole term can only be reduced via the terms $p_{\mathrm{a}}(1 - p_{\mathrm{a}})$. This is the variance of the activation variable, and this is reduced when the probability of activation is strong in either direction. In other words, the second term in the objective is pushing the activation variable to confidently make a choice in the cases in which there is a large discrepancy for with and without AI assistance. Obtaining $\alpha^*, \beta^* = \arg\min_{\alpha,\beta} \ell(\alpha, \beta)$ then gives us a function with which we can re-calibrate future AI assistance: $\texttt{logistic}(\alpha^* \cdot f(\mathbf{x}) + \beta^*)$.

4.3.2 Assisting Humans with Prediction Sets

Given the challenge in assisting humans with confidence scores, Babbar et al. (2022) and Straitouri et al. (2023) propose an alternative approach that uses confidence *sets* instead of scores. Imagine we have a classification problem in which the features are an image of an animal and the label is the species. Instead of showing the human some representation or reproduction of the softmax confidence scores (possibly for every possible species or just the J highest ranked), the human instead sees:

$$C(\mathbf{x}) = \{\texttt{cat}, \texttt{tiger}, \texttt{lion}, \texttt{leopard}\}$$

and is told that the true label is within this set with high-probability. Continuing with design pattern #1 (Subfigure 3a), the human would then choose one label from the set to make the final prediction. Prediction sets are quite natural from a user-interface perspective since, like in search and product recommendations, users are accustomed to having a few candidates from which to select. Furthermore, by not showing the user the confidence scores, we bypass the aforementioned issue of humans not being good at reasoning with explicit probabilities.

I now summarize the approach of Straitouri et al. (2023), which builds the aforementioned prediction sets using *conformal prediction* (CP). Conceptually, given a confidence level $1-\alpha$ (e.g. 90%), CP tries not to calibrate confidence scores per label but finds a threshold τ for which the (presumably) un-calibrated confidence scores *will* cover the true label with probability $1-\alpha$. See Section 1.4 for background information on CP. Angelopoulos and Bates (2021) also provides an accessible introduction. The naive approach to apply CP to HAIC would simply be to choose a reasonable α, say 5%, and show the user the resulting CP sets $C_\alpha(\mathbf{x};\tau)$, where τ would be fit on validation data to ensure the mis-coverage rate is at most α: $\mathbb{P}(y^* \notin C(\mathbf{x};\tau)) \leq \alpha$. However, this approach does not adapt the CP to the user. If the user is forced to choose an element of the set, then we want the sets to be adaptive to the abilities of the human. If the human is a skilled predictor, then we can afford to make the sets larger, since the set can prioritize coverage without sacrificing the human's ability to find the correct label. If the human is unskilled, however, then the set should be small, to steer the human toward a better label than they might pick from the complete label set.

Straitouri et al. (2023) formalize and provide an approach to user-adaptive CP sets as follows. Firstly, they define the goal of a prediction set in the HAIC setting to be:

$$\mathbb{P}\left(\hat{\mathrm{y}} = y^* \mid C_\alpha\right) \;>\; \mathbb{P}\left(\hat{\mathrm{y}} = y^* \mid \mathcal{Y}\right)$$

where $\mathbb{P}\left(\hat{\mathrm{y}} = y^* \mid \cdot\right)$ is the human's marginal probability of success, across all test instances—or in other words, the human's accuracy. So this statement says that the human's accuracy when given the set should be strictly better than their accuracy without the set—an instantiation of our *synergy* principle. Ideally, we'd like to choose an α value that maximizes the probability on the LHS:

$$\alpha^* \;=\; \underset{\alpha\in(0,1)}{\arg\max}\; \mathbb{P}\left(\hat{\mathrm{y}} = y^* \mid C_\alpha\right).$$

Since α directly controls the size of the sets, the above optimization problem captures the interplay between set size and human skill mentioned above.

Unfortunately, we cannot optimize through $\mathbb{P}\left(\hat{\mathrm{y}} = y^* \mid C_\alpha\right)$ directly since this would mean that we are optimizing through the human. Instead, Straitouri et al. (2023) propose to approximate the human's success probability as follows. First, consider the expert's accuracy when the true label is included in the prediction set, $\mathbb{P}\left(\hat{\mathrm{y}} = y^* \mid y^* \in C_\alpha(\mathbf{x})\right)$. They model that quantity using a *discrete choice model*:

$$\mathbb{P}\left(\hat{\mathrm{y}} = y^* \mid y^* \in C_\alpha(\mathbf{x})\right) \;\approx\; p\left(\hat{\mathrm{y}} = y^* \mid y^* \in C_\alpha(\mathbf{x})\right) \;=\; \frac{\exp\left\{\phi(y^*, y^*)\right\}}{\sum_{y'\in C_\alpha(\mathbf{x})} \exp\left\{\phi(y^*, y')\right\}} \tag{76}$$

where $\phi(y^*, y') \in \mathbb{R}$ quantifies the human's preference for selecting label y' when the true label is y^*. Note that the denominator is just summed over the labels in the prediction

set. Given a held-out data set, Straitouri et al. (2023) propose computing $\phi(y^*, y')$ using a confusion matrix: $\phi(y^*, y') = \log \hat{\pi}_{y^*,y'}$ where $\hat{\pi}_{y^*,y'}$ is the confusion matrix entry that represents the fraction of times the human said label y' when the true label was y^*. The model of expert accuracy can then be written as:

$$\hat{p}\left(\hat{\mathrm{y}} = y^* \mid y^* \in C_\alpha(\mathbf{x})\right) \;=\; \frac{\hat{\pi}_{y^*,y^*}}{\sum_{y' \in C_\alpha(\mathbf{x})} \hat{\pi}_{y^*,y'}}, \tag{77}$$

which is the confusion matrix re-normalized over just the column elements included in the prediction set. Given that the true label is in the set (which is guaranteed by CP to happen $(1-\alpha)\%$ of the time), then the above equation makes clear how restricting the decision to the conformal set improves the human's accuracy simply because the denominator usually is smaller than if it were summed over all of $\mathcal{Y}$, making it less than one. Given this model and a second held-out set, we can estimate the humans accuracy, conditioned on seeing the CP set, as:

$$\begin{aligned}
&\mathbb{P}\left(\hat{\mathrm{y}} = y^* \mid C_\alpha\right) \\
&= \mathbb{E}_{\mathbf{x},\mathrm{y}^*}\left[\mathbb{P}\left(\mathrm{y}^* \in C_\alpha(\mathbf{x})\right)\mathbb{P}\left(\hat{\mathrm{y}} = \mathrm{y}^* \mid \mathrm{y}^* \in C_\alpha(\mathbf{x})\right) + \mathbb{P}\left(\mathrm{y}^* \notin C_\alpha(\mathbf{x})\right)\mathbb{P}\left(\hat{\mathrm{y}} = \mathrm{y}^* \mid \mathrm{y}^* \notin C_\alpha(\mathbf{x})\right)\right] \\
&\approx \frac{1}{M}\sum_{\boldsymbol{x}_m} \mathbb{I}[y^*_m \in C_\alpha(\boldsymbol{x}_m)] \cdot \hat{p}\left(\hat{\mathrm{y}}_m = y^*_m \mid y^*_m \in C_\alpha(\boldsymbol{x}_m)\right) \\
&\qquad\qquad + \;\mathbb{I}[y^*_m \notin C_\alpha(\boldsymbol{x}_m)] \cdot \underbrace{\mathbb{P}\left(\hat{\mathrm{y}}_m = y^*_m \mid y^*_m \notin C_\alpha(\boldsymbol{x}_m)\right)}_{=0} \\
&= \frac{1}{M}\sum_{\boldsymbol{x}_m} \mathbb{I}[y^*_m \in C_\alpha(\boldsymbol{x}_m)] \cdot \hat{p}\left(\hat{\mathrm{y}}_m = y^*_m \mid y^*_m \in C_\alpha(\boldsymbol{x}_m)\right) \;\triangleq\; \hat{\mu}_\alpha,
\end{aligned}$$

where M is the number of calibration samples. The term $\mathbb{P}\left(\hat{\mathrm{y}}_m = y^*_m \mid y^*_m \notin C_\alpha(\boldsymbol{x}_m)\right)$ is zero because we assume that the human cannot choose labels that are not in the prediction set. Thus when the true label is outside the prediction set, the human's accuracy is zero. The procedure concludes by choosing the α that maximizes $\hat{\mu}_\alpha$:

$$\alpha^* \;=\; \arg\max_{\alpha \in \mathcal{A}} \; \hat{\mu}_\alpha$$

where $\mathcal{A}$ denotes the space of coverage levels to search across. Note that this optimization does not require re-computing the confusion matrix, only the CP set with a different confidence level, which just requires re-computing $\hat{\tau}$. In practice, Straitouri et al. (2023) incorporate additional tricks, such as adding a finite-sample correction to $\hat{\mu}_\alpha$, and recognizing that the number of α values to test in $\mathcal{A}$ depends on the size of the calibration set used for CP. See the paper for these details.

4.3.3 Indistinguishability: When is Human Effort Helpful?

As mentioned above, design pattern #1 does not benefit from autonomy since the human is always engaged in decision making. Yet, it would be nice if we could use the human selectively, only when their predictions improve upon the model's. Alur et al. (2024) propose a framework for doing just this, calling it *algorithmic indistinguishability*. The idea is that humans outperform predictive models not because they are 'smarter' but because they have

additional information not captured by the feature space that's accessible to the model. The trick, then, is to find the subsets of the feature space where the algorithm can no longer contribute information and check if the human can still contribute an informative prediction.

Again assume the setting of a binary prediction task $\mathrm{y} \in \{0, 1\}$ using a feature space $\mathbf{x} \in \mathcal{X}$. Assume the human generates predictions $\mathrm{m} = h(\mathbf{x})$ and that we also train a binary classifier that outputs confidence scores $f(\mathbf{x}) \in [0, 1]$. Let f come for a set of classifiers $f \in \mathcal{F}$. Alur et al. (2024) define an *α-indistinguishable subset* as a set $S \subseteq \mathcal{X}$ if for $\alpha \geq 0$ and for all $f \in \mathcal{F}$:

$$\left|\mathrm{Cov}\left[\mathrm{y}, f(\mathbf{x}) \mid \mathbf{x} \in S\right]\right| \;\leq\; \alpha$$

where $\mathrm{Cov}[\mathrm{y}, f(\mathbf{x})|\mathbf{x} \in S]$ denotes the covariance between the model output and the label[1], conditioned on the features being in S. That is, once the feature space is restricted to S, any variation if $f(\mathbf{x})$ is not informative about the label. One way to have $\mathrm{Cov}\left[\mathrm{y}, f(\mathbf{x}) \mid \mathbf{x} \in S\right] = 0$ is if the label or classifier's value is constant throughout the subset. If we can partition $\mathcal{X}$ into K α-indistinguishable subsets, then we call the partitioning an *α-multicalibrated partition*. The same intuition extends from the single subset case: the partition has extracted all predictive power from $\mathcal{F}$.

With these partition in hand (assume it is easy to find them), we can then check if the human still has predictive power to contribute: $\left|\mathrm{Cov}\left[\mathrm{y}, \mathrm{m} \mid \mathbf{x} \in S\right]\right| > \alpha$. This implies that the human has additional knowledge that can explain how the label varies within this subset of feature space. Notice the absolute value implies that, even if the human's prediction is negatively correlated with the true label, this is still a useful signal to exploit. Alur et al. (2024) exploit these insights via Algorithm 4 below.

Algorithm 4: Incorporating Human Expertise into Algorithmic Predictions

Input: Training data $\{\boldsymbol{x}_n, y_n\}_{n=1}^N$, an α-multicalibrated partition $\{S_k\}_{k=1}^K$, a binary classifier learning algorithm $\mathcal{R}$, a test observation $\boldsymbol{x}_{N+1}$.

for $k = [1, K]$ **do**

  Collect human predictions to form dataset: $Z_k = \{(y_n, m_n) : \boldsymbol{x}_n \in S_k\}$

  Fit subset specific classifier: $g_k = \mathcal{R}(Z_k)$, such that $\hat{y} = g_k(\mathrm{m})$.

end

Let j be the index of the subset containing $\boldsymbol{x}_{N+1}$.

Query human to obtain prediction $m_{N+1} = h(\boldsymbol{x}_{N+1})$.

Output: Prediction $g_j(m_{N+1})$

Unfortunately, the practical utility of this model relies upon finding the α-multicalibrated partition, which will be very challenging to do for complex model classes $\mathcal{F}$ and in high-dimensional feature spaces.

4.4 Learning to Reject

All methods discussed so far have assumed that both the model and human will make predictions for all test cases. This does not allow for any of the benefits of autonomy. We now turn to an alternative pattern, #3, that allows some decisions to be predicted by the model and others by the classifier. Core to this design paradigm is the concept of a *rejector*—$r : \mathcal{X} \to \{0, 1\}$—some function that acts as a meta-classifier, deciding if the

[1]Recall that the empirical covariance is estimated as: $\widehat{\mathrm{Cov}}\left[\mathrm{y}, f(\mathbf{x})\right] = (1/N)\sum_n (\mathrm{y} - \bar{\mathrm{y}})(f(\mathbf{x}) - \bar{f(\mathbf{x})})$

predictive model should make the prediction or not:

$$f^{\perp}(\mathbf{x}) \;=\; \begin{cases} \arg\max_{y\in\mathcal{Y}} \; f_y(\mathbf{x}) & \text{if } r(\mathbf{x}) = 0 \\ \perp & \text{if } r(\mathbf{x}) = 1 \end{cases}$$

where $f(\mathbf{x})$ is the traditional classifier and $\perp$ denotes the rejection decision. Learning this rejector is known as *learning to reject* (L2R). While presumably a human or backup system would make the prediction if $r(\mathbf{x}) = 1$, the performance of this backup system is not explicitly modeled. This downside will be addressed in the next section on *learning to defer to an expert*.

4.4.1 Chow's Rule: Rejection via Calibrated Confidence Scores

L2R can be derived from first principles using Bayesian decision theory. Consider the following *cost* function for a *decision* $\delta : \mathcal{X} \to \{\mathcal{Y} \cup \perp\}$:

$$c_{\text{L2R}}\left(\delta(\mathbf{x}), \mathrm{y}\right) \;=\; \begin{cases} 0 & \text{if } \delta(\mathbf{x}) = \mathrm{y} \\ 1 & \text{if } \delta(\mathbf{x}) \neq \mathrm{y} \\ \lambda & \text{if } \delta(\mathbf{x}) = \perp \end{cases}$$

where $\lambda \in (0, 1)$ represents the cost incurred for choosing to reject. This cost is equal to the canonical $0-1$ loss when the classifier makes a prediction, only changing for the rejection option. As $\lambda \to 1$, the cost of rejection approaches the cost of making a wrong prediction, and as $\lambda \to 0$, the cost approaches that of making the correct prediction. Thus, in practice, λ is usually set such that $0.01 < \lambda < (K-1)/K$, where values closer to .01 would only be used for extreme safety critical scenarios.

The Bayes risk for the rejection problem is written as:

$$\mathcal{R}(\mathbf{x}, \delta) \;=\; \sum_{y\in\mathcal{Y}} \mathbb{P}(y|\mathbf{x}) \cdot c\left(\delta(\mathbf{x}), y\right).$$

When a prediction is made, the risk is the usual Bayes risk for a classifier. Let $\delta = \hat{y} = \arg\max_y \mathbb{P}(y|\mathbf{x})$):

$$\mathcal{R}(\mathbf{x}, \delta = \hat{y}) \;=\; \sum_{y\neq\delta} \mathbb{P}(y|\mathbf{x}) \;=\; 1 - \mathbb{P}(\mathrm{y} = \delta|\mathbf{x}).$$

When the rejection option is chosen, $\delta = \perp$, the risk is:

$$\mathcal{R}(\mathbf{x}, \delta = \perp) \;=\; \sum_{y\in\mathcal{Y}} \mathbb{P}(y|\mathbf{x}) \cdot \lambda \;=\; \lambda \cdot \sum_{y\in\mathcal{Y}} \mathbb{P}(y|\mathbf{x}) \;=\; \lambda \cdot 1 \;=\; \lambda.$$

Thus, the decision function that minimizes the Bayes risk is:

$$\delta^*\left(\mathbf{x}\right) \;=\; \begin{cases} \arg\max_y \mathbb{P}(\mathrm{y} = y|\mathbf{x}) & \text{if } 1 - \max_y \mathbb{P}(\mathrm{y} = y|\mathbf{x}) \leq \lambda \\ \perp & \text{otherwise.} \end{cases} \tag{78}$$

Of course, in practice, we never have access to the real underlying conditional distribution

$\mathbb{P}(\mathrm{y}|\mathbf{x})$. Instead, we have a model-based approximation:

$$\hat{\delta}(\mathbf{x}) = \begin{cases} \arg\max_y f_y(\mathbf{x}) & \text{if } \max_y f_y(\mathbf{x}) \geq 1 - \lambda \\ \perp & \text{otherwise,} \end{cases} \tag{79}$$

where the model has be substituted for the Bayes classifier and the inequality has been rearranged to make clear that the Bayes optimal rejector is ensuring that the model class has a sufficiently high probability. Equation 79 is known as *Chow's rule*, named after the author who first presented the derivation (Chow, 1957a). Given a perfect / canonically calibrated $f(\mathbf{x})$, then Chow's rule is the Bayes optimal rejection decision. Unfortunately, models are often not calibrated in practice, leading to poor rejection behavior.

4.4.2 Rejection via Optimizing Selective Risk

Given the difficulty of obtaining a calibrated model, Geifman and El-Yaniv (2017) proposed an alternative framework that seeks to control the *selective risk* (SR):

$$\mathcal{SR}(r) = \frac{\mathbb{E}_{\mathbf{x},\mathrm{y}}\left[(1 - r(\mathbf{x})) \cdot \ell_{0,1}(f(\mathbf{x}), \mathrm{y})\right]}{\mathbb{E}_{\mathbf{x}}\left[1 - r(\mathbf{x})\right]} \tag{80}$$

where $\ell_{0,1}$ is the usual $0-1$ loss. The SR trades off the classification loss (numerator) with the coverage (denominator), with both always being bounded between 0 and 1. Samples can be rejected, lowering the classification loss, but the SR will balance this behavior by decreasing the denominator, increasing the overall SR.

Geifman and El-Yaniv (2017) propose parameterizing the rejector via a ranking function $\kappa(\mathbf{x})$, which orders the points by decreasing difficulty, meaning that the higher the score, the more likely the model should make the correct prediction. The rejector, then, chooses all points below some threshold τ to be rejected:

$$r(\mathbf{x}; \kappa, \tau) = \begin{cases} 0 & \text{if } \kappa(\mathbf{x}) \geq \tau \\ 1 & \text{if } \kappa(\mathbf{x}) < \tau. \end{cases}$$

Ideally, $\kappa(\mathbf{x})$ would order the points by their (descending) Bayes risk: $\kappa(\boldsymbol{x}_0) < \kappa(\boldsymbol{x}_1)$ if $\max_y \mathbb{P}(y|\boldsymbol{x}_0) < \max_y \mathbb{P}(y|\boldsymbol{x}_1)$. But, as mentioned above, $\mathbb{P}(y|\boldsymbol{x})$ is unknown. Yet, while Chow's rule depends on the exact value of the confidence score, this ranking only requires matching the ordering—a much easier task. Geifman and El-Yaniv (2017) propose using the softmax score as the ranking function, and finds this works well in practice: $\kappa(\mathbf{x}) = \max_y f_y(\mathbf{x})$. In practice, Geifman and El-Yaniv (2017) find a value τ_0 for which $\mathcal{SR}(r) < \epsilon$ by doing binary search for the threshold over a calibration set (with finite sample corrections). As τ approaches 0, then the model rejects less frequently and more frequently as τ approaches 1.

4.4.3 Consistent Surrogate Loss for Rejection Learning

While optimizing SR, as done above, does not rely upon having a calibrated model, it makes a sacrifice by considering a marginal quantity—that is, the selective risk is an average. The framework does not have the ability to reject particular sample. However, in applications

such as medicine, we might be concerned about a patient's individual probabilities. Charoenphakdee et al. (2021) addressed this problem by having the insight that, if the rejection cost λ is chosen prior to model fitting, we do not need a canonically calibrated classifier. Rather, we just need a classifier that is calibrated at the threshold point $\max_y f_y(\mathbf{x}) \geq 1 - \lambda$. This is much more manageable. Cao et al. (2022) then drove this insight to its full capabilities, deriving a consistent surrogate loss for multi-class classification with arbitrary losses. 'Consistent' here means that the minimizers of the surrogate loss correspond to the Bayes optimal solutions for the rejector and classifier that were identified in Section 4.4.1. For the cross-entropy loss, the consistent surrogate is written as follows. Let $g_1(\mathbf{x}), \ldots, g_K(\mathbf{x})$, such that $g_k : \mathcal{X} \mapsto \mathbb{R}$, be K real valued functions, which are often taken to be the logits of a neural network classifier. Let $g_\perp(\mathbf{x})$ be given the same definition, possibly implemented by having an additional logit output in a neural network. The L2R surrogate loss is then defined point-wise as:

$$\ell_{\text{L2R}}(\mathbf{x}, \mathrm{y}; \lambda, g_1, \ldots, g_\perp) \;=\; -\log \frac{\exp\{g_y(\mathbf{x})\}}{\sum_{y' \in \mathcal{Y}^\perp} \exp\{g_{y'}(\mathbf{x})\}} \;-\; (1-\lambda) \log \frac{\exp\{g_\perp(\mathbf{x})\}}{\sum_{y' \in \mathcal{Y}^\perp} \exp\{g_{y'}(\mathbf{x})\}}$$

where $\mathcal{Y}^\perp = \mathcal{Y} \cup \{\perp\}$. This loss can be interpreted as a traditional softmax-based classification loss but with $K + 1$ outputs instead of the usual K. In turn, the denominators are summed over $K + 1$ terms, which includes the rejection model $g_\perp$. The classifier is obtained by taking the maximum over the g functions corresponding to class dimensions: $\hat{y} = \arg\max_{k \in [1,K]} g_k(\boldsymbol{x})$. The rejection function is given as: $r(\boldsymbol{x}) = \mathbb{I}[g_\perp(\boldsymbol{x}) \geq \max_k g_k(\boldsymbol{x})]$. Thus when the value of $g_\perp(\boldsymbol{x})$ is greater than the other K functions, the model rejects. Otherwise, it makes a prediction corresponding to the maximum over $g_k(\boldsymbol{x})$ for $k \in [1, K]$.

4.5 Learning to Defer to an Expert

L2R does not model what happens to the data point if it is rejected beyond assigning the uniform cost λ. In practice, if the predictive model cannot make an accurate prediction, a human often must step in and make the prediction. *Learning to defer* (L2D) is similar to L2R except that the downstream decision maker's performance is also modeled. This allows the model to make predictions in cases where, even if the model is not strongly confident, the human is even less confident. Fortunately, a lot of the machinery we used to formulate rejection learning, as we will see below.

4.5.1 Bayesian Decision Theory for Deferral

Like L2R, L2D can be derived from first principles using Bayesian decision theory. Consider the following *cost* function for a *decision* $\delta : \mathcal{X} \to \{\mathcal{Y} \cup \{\perp\}\}$, where $\perp$ again represents rejection / deferral:

$$c_{\text{L2D}}\left(\delta(\mathbf{x}), \mathrm{y}, \mathrm{m}\right) \;=\; \begin{cases} 0 & \text{if } \delta(\mathbf{x}) = \mathrm{y} \\ 1 & \text{if } \delta(\mathbf{x}) \neq \mathrm{y} \\ \mathbb{I}[\mathrm{m} \neq \mathrm{y}] & \text{if } \delta(\mathbf{x}) = \perp. \end{cases}$$

There are two crucial differences to the L2R cost. The first is that the cost now takes as input $\mathrm{m} \in \mathcal{Y}$, which is the human's prediction. We assume the human generates predictions

according to the underlying distribution $\mathbb{P}_{\mathrm{m}}(\mathrm{m}|\mathbf{x})$. The second is that, instead of a fixed cost λ, deferral (akin to rejection) now incurs a dynamic cost $\mathbb{I}[\mathrm{m} \neq \mathrm{y}]$, which is one when the human's prediction *does not match* the label, zero when it does match. This is how we will ultimately incentivize the system to understand the human's abilities, since the system should not defer when the human will be wrong as this is no better than the classifier making the wrong prediction.

We can define the risk of the deferral problem similarly to L2R's above, but now we will also incorporate the human's distribution $\mathbb{P}_{\mathrm{m}}(\mathrm{m}|\mathbf{x})$ as well as the classifier's $\mathbb{P}_{\mathrm{y}}(\mathrm{y}|\mathbf{x})$:

$$\mathcal{R}(\mathbf{x}, \delta) \;=\; \sum_{y \in \mathcal{Y}} \sum_{m \in \mathcal{Y}} \mathbb{P}_{\mathrm{y}}(y|\mathbf{x}) \cdot \mathbb{P}_{\mathrm{m}}(m|\mathbf{x}) \cdot c\left(\delta(\mathbf{x}), y, m\right). \tag{81}$$

When the model makes a prediction, i.e. $\delta(\mathbf{x}) = \hat{y}$, the above risk is equivalent to L2R (and that of traditional 0-1 loss classification). In the case of deferral, i.e. $\delta = \perp$, the risk is:

$$\begin{aligned}
\mathcal{R}(\mathbf{x}, \delta = \perp) \;&=\; \sum_{y \in \mathcal{Y}} \sum_{m \in \mathcal{Y}} \mathbb{P}_{\mathrm{y}}(y|\mathbf{x}) \cdot \mathbb{P}_{\mathrm{m}}(m|\mathbf{x}) \cdot c\left(\perp, y, m\right) \\
&=\; \sum_{y \in \mathcal{Y}} \sum_{m \in \mathcal{Y}} \mathbb{P}_{\mathrm{y}}(y|\mathbf{x}) \cdot \mathbb{P}_{\mathrm{m}}(m|\mathbf{x}) \cdot \mathbb{I}[m \neq y] \\
&=\; 1 \;-\; \sum_{y \in \mathcal{Y}} \mathbb{P}_{\mathrm{y}}(y|\mathbf{x}) \cdot \mathbb{P}_{\mathrm{m}}(y|\mathbf{x}) \\
&=\; 1 \;-\; \mathbb{P}_{\mathrm{y,m}}(\mathrm{m} = \mathrm{y}|\mathbf{x}) \\
&=\; \mathbb{P}_{\mathrm{y,m}}(\mathrm{m} \neq \mathrm{y}|\mathbf{x}).
\end{aligned} \tag{82}$$

Thus, the decision function that minimizes the Bayes risk is:

$$\delta^*_{\mathrm{L2D}}\left(\mathbf{x}\right) \;=\; \begin{cases} \arg\max_y \mathbb{P}_{\mathrm{y}}(y|\mathbf{x}) & \text{if } 1 - \max_y \mathbb{P}_{\mathrm{y}}(y|\mathbf{x}) \leq \mathbb{P}_{\mathrm{y,m}}(\mathrm{m} \neq \mathrm{y}|\mathbf{x}) \\ \perp & \text{otherwise.} \end{cases} \tag{83}$$

Like with L2R, in practice we will need to substitute a model for $\mathbb{P}_{\mathrm{y}}(y|\mathbf{x})$. Yet with L2D we encounter an additional challenge in that $\mathbb{P}_{\mathrm{y,m}}(\mathrm{m} \neq \mathrm{y}|\mathbf{x})$ is unknown as well. In turn, directly implementing this decision rule will require two models, $f(\mathbf{x}) \approx \mathbb{P}_{\mathrm{y}}(\mathrm{y}|\mathbf{x})$ *and* $f_{\mathrm{m}\neq\mathrm{y}}(\mathbf{x}) \approx \mathbb{P}_{\mathrm{y,m}}(\mathrm{m} \neq \mathrm{y}|\mathbf{x})$. Plugging these two models into the decision rule, we have:

$$\hat{\delta}_{\mathrm{L2D}}\left(\mathbf{x}\right) \;=\; \begin{cases} \arg\max_y f_y(\mathbf{x}) & \text{if } \max_y f(\mathbf{x}) \geq 1 - f_{\mathrm{m}\neq\mathrm{y}}(\mathbf{x}) \\ \perp & \text{otherwise.} \end{cases} \tag{84}$$

Here there are two sources of calibration error, $f(\mathbf{x})$ and $f_{\mathrm{m}\neq\mathrm{y}}(\mathbf{x})$, making a practical implementation challenging.

4.5.2 Consistent Surrogate Loss for L2D

Fortunately, we can use a similar formulation as the L2R consistent surrogate loss to derive a consistent surrogate for L2D. However, before giving the surrogate, first note that L2D expects we have access to predictions from the human / backup system to which we are deferring; denote this prediction as $\mathrm{m} \in \mathcal{Y}$. An N-sized training dataset is then $\mathcal{D} = \{(\boldsymbol{x}_n, y_n, m_n)\}_{n=1}^N$. As human labels alone are usually expensive to obtain (as we discussed

with crowdsourcing), training an L2D system is even more expensive as it requires the ground-truth label y in addition to the human label m.

Moving on to the cross-entropy surrogate loss for L2D, it is formulated written as follows (Mozannar and Sontag, 2020). Let $g_1(\mathbf{x}), \ldots, g_K(\mathbf{x})$, such that $g_k : \mathcal{X} \mapsto \mathbb{R}$, be K real valued functions, which are often taken to be the logits of a neural network classifier. Let $g_\perp(\mathbf{x})$ be given the same definition, possibly implemented by having an additional logit output in a neural network. The L2D surrogate loss is then defined point-wise as:

$$\ell_{\text{L2D}}(\mathbf{x}, \mathrm{y}, \mathrm{m}; g_1, \ldots, g_\perp) \;=\; -\log \frac{\exp\{g_y(\mathbf{x})\}}{\sum_{y' \in \mathcal{Y}^\perp} \exp\{g_{y'}(\mathbf{x})\}} - \mathbb{I}\left[\mathrm{m} = \mathrm{y}\right] \log \frac{\exp\{g_\perp(\mathbf{x})\}}{\sum_{y' \in \mathcal{Y}^\perp} \exp\{g_{y'}(\mathbf{x})\}}$$

where $\mathcal{Y}^\perp = \mathcal{Y} \cup \{\perp\}$. Like with L2R, this loss can be interpreted as a traditional softmax-based classification loss but with $K+1$ outputs instead of the usual K. In turn, the denominators are summed over $K+1$ terms, which includes the deferral model $g_\perp$. The classifier is obtained by taking the maximum over the g functions corresponding to class dimensions: $\hat{y} = \arg\max_{k \in [1,K]} g_k(\boldsymbol{x})$. The deferral function is given as: $r(\boldsymbol{x}) = \mathbb{I}[g_\perp(\boldsymbol{x}) \geq \max_k g_k(\boldsymbol{x})]$. Thus when the value of $g_\perp(\boldsymbol{x})$ is greater than the other K functions, the model defers to the human. Otherwise, it makes a prediction corresponding to the maximum over $g_k(\boldsymbol{x})$ for $k \in [1, K]$.

The close relationship to the L2R surrogate may not be clear at first glance, but observe that:

$$\begin{aligned}
&\mathbb{E}_{\mathbf{x}} \mathbb{E}_{\mathrm{y},\mathrm{m}|\mathbf{x}} \left[\mathbb{I}\left[\mathrm{m} = \mathrm{y}\right] \log \frac{\exp\{g_\perp(\mathbf{x})\}}{\sum_{y' \in \mathcal{Y}^\perp} \exp\{g_{y'}(\mathbf{x})\}} \right] \\
&\qquad = \; \mathbb{E}_{\mathbf{x}} \left[\mathbb{E}_{\mathrm{y},\mathrm{m}|\mathbf{x}} \left[\mathbb{I}\left[\mathrm{m} = \mathrm{y}\right]\right] \log \frac{\exp\{g_\perp(\mathbf{x})\}}{\sum_{y' \in \mathcal{Y}^\perp} \exp\{g_{y'}(\mathbf{x})\}} \right] \\
&\qquad = \; \mathbb{E}_{\mathbf{x}} \left[\mathbb{P}_{\mathrm{y},\mathrm{m}}\left(\mathrm{m} = \mathrm{y}|\mathbf{x}\right) \log \frac{\exp\{g_\perp(\mathbf{x})\}}{\sum_{y' \in \mathcal{Y}^\perp} \exp\{g_{y'}(\mathbf{x})\}} \right] \\
&\qquad = \; \mathbb{E}_{\mathbf{x}} \left[(1 - \underbrace{\mathbb{P}_{\mathrm{y},\mathrm{m}}\left(\mathrm{m} \neq \mathrm{y}|\mathbf{x}\right)}_{\mathcal{R}(\mathbf{x}, \delta = \perp)}) \log \frac{\exp\{g_\perp(\mathbf{x})\}}{\sum_{y' \in \mathcal{Y}^\perp} \exp\{g_{y'}(\mathbf{x})\}} \right].
\end{aligned} \tag{85}$$

Thus we see that the term in front of the deferral function loss is equal to $(1 - \mathcal{R}_{\text{L2D}}(\mathbf{x}, \delta = \perp))$. L2R had this same form: $(1 - \mathcal{R}_{\text{L2R}}(\mathbf{x}, \delta = \perp)) = (1 - \lambda)$. Here, though, the analogous cost must be estimated from data, as we need to learn about the expert's abilities.

4.5.3 L2D to Multiple Experts

Often complex problems require the expertise of multiple individuals. L2D can be extended to multiple experts as follows (Verma et al., 2023). Assume there are J experts to which we can defer (exclusively), and denote the decision to defer to expert-j as $\perp_j$. The multi-expert decision function is then $\delta : \mathcal{X} \to \{\mathcal{Y} \cup \{\perp_1, \ldots, \perp_J\}\}$. Let $\mathrm{m}_j \in \mathcal{Y}$ denote the prediction from the jth expert, whose predictive distribution is $\mathbb{P}(\mathrm{m}_j|\mathbf{x})$. The cost function then takes

all J expert predictions as input:

$$c_{\text{multi-L2D}}\left(\delta(\mathbf{x}), \mathrm{y}, \mathrm{m}_1, \ldots, \mathrm{m}_J\right) = \begin{cases} 0 & \text{if } \delta(\mathbf{x}) = \mathrm{y} \\ 1 & \text{if } \delta(\mathbf{x}) \neq \mathrm{y} \\ \mathbb{I}[\mathrm{m}_1 \neq \mathrm{y}] & \text{if } \delta(\mathbf{x}) = \perp_1 \\ & \vdots \\ \mathbb{I}[\mathrm{m}_J \neq \mathrm{y}] & \text{if } \delta(\mathbf{x}) = \perp_J. \end{cases}$$

The Bayes optimal decision rule is then:

$$\delta^*_{\text{multi-L2D}}\left(\mathbf{x}\right) = \begin{cases} \arg\max_y \mathbb{P}_{\mathrm{y}}(y|\mathbf{x}) & \text{if } 1 - \max_y \mathbb{P}_{\mathrm{y}}(y|\mathbf{x}) \leq \min_j \mathbb{P}_{\mathrm{y},\mathrm{m}_j}(\mathrm{m}_j \neq \mathrm{y}|\mathbf{x}) \\ \perp_i & \text{otherwise, where } i = \arg\min_j \mathbb{P}_{\mathrm{y},\mathrm{m}_j}(\mathrm{m}_j \neq \mathrm{y}|\mathbf{x}). \end{cases}$$

It is then straightforward to derive the consistent cross-entropy surrogate loss:

$$\ell_{\text{multi-L2D}}(\mathbf{x}, \mathrm{y}, \mathrm{m}_1, \ldots, \mathrm{m}_J; g_1, \ldots, g_K, g_{\perp_1}, \ldots, g_{\perp_J}) = \\ -\log \frac{\exp\left\{g_y(\mathbf{x})\right\}}{\sum_{y' \in \mathcal{Y}^{\perp \times J}} \exp\left\{g_{y'}(\mathbf{x})\right\}} - \sum_{j=1}^{J} \mathbb{I}\left[\mathrm{m}_j = \mathrm{y}\right] \log \frac{\exp\left\{g_{\perp_j}(\mathbf{x})\right\}}{\sum_{y' \in \mathcal{Y}^{\perp \times J}} \exp\left\{g_{y'}(\mathbf{x})\right\}}$$

where $\mathcal{Y}^{\perp \times J} = \{\mathcal{Y} \cup \{\perp_1, \ldots, \perp_J\}\}$. This is very similar to the single-expert L2D surrogate loss, but now there are J rejection functions with corresponding terms in the loss function.

4.6 Sequential Learning with Rejection or Deferral

A natural question is: can we extend rejection and deferral to the sequential setting via an RL formulation? We can do so by formulating a policy such that $\pi : \mathcal{S} \mapsto \mathcal{A}^{\perp}$ where $\mathcal{A}^{\perp} = \{\mathcal{A} \cup \{\perp\}\}$, where $\mathcal{A}$ denotes the traditional space of actions and $\perp$ again notes the option to reject or defer. There are two practical ways to formulate this problem, which are outlined below.

Rejection as Termination We first consider the simplest setting: the policy can choose to reject, and the system will essentially terminate its operation. Let the cost of rejecting again be denoted as λ, and assume the environment's reward function is bounded in $[0, 1]$. We assume that, if the policy chooses to reject, then the system will deterministically stay in a state $s_{\perp}$ for the rest of the infinite horizon, receiving a reward of $(1 - \lambda)$ at each time

step:

$$
\begin{aligned}
V^{\pi}_{\text{L2R}}(s) &= \sum_{a \in \mathcal{A}^{\perp}} \pi(a|s) \cdot Q^{\pi}(s,a) \\
&= \pi(\perp|s) \cdot Q^{\pi}(s,\perp) \; + \; \sum_{a \in \mathcal{A}} \pi(a|s) \cdot Q^{\pi}(s,a) \\
&= \pi(\perp|s) \cdot ((1-\lambda) \; + \; Q^{\pi}(s_{\perp},\perp)) + \sum_{a \in \mathcal{A}} \pi(a|s) \cdot Q^{\pi}(s,a) \\
&= \pi(\perp|s) \cdot \left((1-\lambda) \; + \; \sum_{t=1}^{\infty} \gamma^{t} \cdot (1-\lambda) \right) \; + \; \sum_{a \in \mathcal{A}} \pi(a|s) \cdot Q^{\pi}(s,a) \\
&= \pi(\perp|s) \cdot \frac{1-\lambda}{1-\gamma} \; + \; \sum_{a \in \mathcal{A}} \pi(a|s) \cdot Q^{\pi}(s,a).
\end{aligned}
\tag{86}
$$

This formulation is rather easy to implement with existing RL algorithms simply by augmenting the action space with $\perp$ and adding the terminal state $s_{\perp}$, which receives an expected return of $\frac{1-\lambda}{1-\gamma}$. This provides the agent with an 'escape option' if the expected returns from choosing other actions are sufficiently small. Setting λ in practice will take care in order to balance it with the discount factor (γ) and the return from taking the other actions.

Sequential Learning to Defer We can devise a similar formulation but with deferring to an expert. However, in this case, we need some access to the expert's policy, which we will denote as $H : \mathcal{S} \mapsto \mathcal{A}$. If we make a similar assumption that, once the policy defers to the human it can never regain control, we have that the value function decomposes as:

$$
\begin{aligned}
V^{\pi}_{\text{L2D}}(s) &= \sum_{a \in \mathcal{A}^{\perp}} \pi(a|s) \cdot Q^{\pi}(s,a) \\
&= \pi(\perp|s) \cdot Q^{H}(s,\perp) \; + \; \sum_{a \in \mathcal{A}} \pi(a|s) \cdot Q^{\pi}(s,a)
\end{aligned}
\tag{87}
$$

where $Q^{H}(s,\perp)$ is the crucial quantity that captures the expected return from the human taking over control. This is almost impossible to estimate online, as it would require the human provide demonstrations for every iteration of optimization.

Temporary Deferral and Rejection For both rejection and deferral, one could consider alternative formulations in which rejection and deferral is only temporary and the policy π takes back control eventually. This makes the rejection formulation more difficult to implement since what happens during rejection must be modeled (assuming that rejection isn't simply a pause but rather changes the state to which the agent returns). This requirement makes the rejection and deferral variants more similar in their implementations. For the deferral variant, assuming temporary deferral may be more or less tractable, depending on the estimation of $Q^{H}(s,\perp)$.

4.7 Model Misspecification and Complementing Human Behavior

One downside of learning to defer (L2D) is that is does not necessarily learn a complimentary model of human behavior. That is, it tries to simply fit the training data like a non-L2D

model would. This is the proper thing to do when the model has the potential to faithfully capture the underling data distribution $\mathbb{P}(\mathrm{y}|\mathbf{x})$. However, if the model is misspecifed such that achieving $\mathbb{P}(\mathrm{y}|\mathbf{x})$ is impossible, then the L2D objective results in dramatic underfitting. While underfitting is a common and a natural challenge in predictive modeling, HAIC models have the ability to mitigate underfitting by allocating the unpredictable points to the human (assuming the human at least a powerful predictor).

Example Show example of a true function that is linear in some subspace and highly nonlinear in a complimentary subspace. Here, instead of trying to fit all the data, it would be better for the model to fit only the linear part of the data space. Below I give a brief overview of how prior work has attempted to address the issues of misspecification.

Combinatorial Optimization The most obvious approach to address the misspecification / complementarity issue is to only fit the model to the datapoints for which it can achieve a low loss. Given a training dataset $\mathcal{D} = \{\boldsymbol{x}_n, y_n, m_n\}_{n=1}^N$, with m_n denoting the human's prediction for the nth training example, we can partition $\mathcal{D}$ into disjoint sets $\mathcal{D}_H$ and $\mathcal{D}_M$, where $\mathcal{D}_H$ are the points that the human makes the predictions for $\mathcal{D}_M$ are the points for which the model makes the prediction. We then have that $\mathcal{D} = \mathcal{D}_H \cup \mathcal{D}_M$ and $\mathcal{D}_H \cap \mathcal{D}_M = \emptyset$. Given a model $f_{\boldsymbol{\theta}}(\mathbf{x}) = \hat{y}$, we can decompose the loss over the model and human:

$$\sum_{n=1}^{N} \ell(y_n, f(\boldsymbol{x}_n)) \;\geq\; \sum_{x_i, y_i \in \mathcal{D}_M} \ell(y_i, f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \;+\; \sum_{m_j, y_j \in \mathcal{D}_H} \ell(y_j, m_j). \tag{88}$$

If the human is significantly more powerful than the model, a constraint may need to be added such that the human is not assigned all data points. The optimization problem is then to fit the classifier parameters $\boldsymbol{\theta}$ *and* optimize the points assigned to each set $\mathcal{D}_H$ and $\mathcal{D}_M$. CITE showed that, even for linear regression, such an optimization problem is NP-hard, though there are good greedy approximations. Another difficulty with this formulation is that for each test point, it is not obvious if it should be assigned to the human or model, as there is no explicit rejector function.

Bibliography

Pieter Abbeel and Andrew Y Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the twenty-first international conference on Machine learning*, page 1, 2004.

Rohan Alur, Loren Laine, Darrick K Li, Dennis Shung, Manish Raghavan, and Devavrat Shah. Integrating expert judgment and algorithmic decision making: An indistinguishability framework. *arXiv preprint arXiv:2410.08783*, 2024.

Anastasios Nikolas Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification, 2021. URL `https://arxiv.org/abs/2107.07511`.

Varun Babbar, Umang Bhatt, and Adrian Weller. On the utility of prediction sets in human-ai teams. In *International Joint Conference on Artificial Intelligence*, 2022.

Ondrej Bohdal, Yongxin Yang, and Timothy Hospedales. Meta-calibration: Learning of model calibration using differentiable expected calibration error. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL https://openreview.net/forum?id=R2hUure38l.

Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39:324, 1952. URL https://api.semanticscholar.org/CorpusID:125209808.

Yuzhou Cao, Tianchi Cai, Lei Feng, Lihong Gu, Jinjie GU, Bo An, Gang Niu, and Masashi Sugiyama. Generalizing consistent multi-class classification with rejection to be compatible with arbitrary losses. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=DwHIcEyias.

Nontawat Charoenphakdee, Zhenghang Cui, Yivan Zhang, and Masashi Sugiyama. Classification with rejection based on cost-sensitive classification. In *International Conference on Machine Learning*, 2021.

C. K. Chow. An optimum character recognition system using decision functions. *IRE Trans. Electron. Comput.*, 6:247–254, 1957a.

Chi-Keung Chow. An optimum character recognition system using decision functions. *IRE Transactions on Electronic Computers*, (4):247–254, 1957b.

Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

Sanjoy Dasgupta. Analysis of a greedy active learning strategy. In *Advances in Neural Information Processing Systems*, pages 337–344, 2005.

Alexander Philip Dawid and Allan M Skene. Maximum likelihood estimation of observer error-rates using the em algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 28(1):20–28, 1979.

Peter Englert, Alexandros Paraschos, Jan Peters, and Marc Peter Deisenroth. Model-based imitation learning by probabilistic trajectory matching. In *2013 IEEE international conference on robotics and automation*, pages 1922–1927. IEEE, 2013.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. *Advances in neural information processing systems*, 30, 2017.

Daniel Golovin and Andreas Krause. Adaptive submodularity: Theory and applications in active learning and stochastic optimization. *Journal of Artificial Intelligence Research*, 42:427–486, 2011.

Peter Grünwald. Safe probability, 2016.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.

Chirag Gupta and Aaditya Ramdas. Top-label calibration and multiclass-to-binary reductions. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=WqoBaaPHS-`.

Alan Hájek. The reference class problem is your problem too. *Synthese*, 156:563–585, 2007.

Ursula Hebert-Johnson, Michael Kim, Omer Reingold, and Guy Rothblum. Multicalibration: Calibration for the (Computationally-identifiable) masses. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1939–1948, 2018.

Benedikt Höltgen and Robert C Williamson. On the richness of calibration. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency*, 2023.

Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning. *arXiv Preprint arXiv:1112.5745*, 2011.

Daniel Kahneman and Amos Tversky. Subjective probability: A judgment of representativeness. *Cognitive Psychology*, 3(3):430–454, 1972.

Sham M. Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *International Conference on Machine Learning*, 2002.

Archit Karandikar, Nicholas Cain, Dustin Tran, Balaji Lakshminarayanan, Jonathon Shlens, Michael C Mozer, and Becca Roelofs. Soft calibration objectives for neural networks. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Liyiming Ke, Sanjiban Choudhury, Matt Barnes, Wen Sun, Gilwoo Lee, and Siddhartha Srinivasa. Imitation learning as f-divergence minimization. In *Algorithmic Foundations of Robotics XIV: Proceedings of the Fourteenth Workshop on the Algorithmic Foundations of Robotics 14*, pages 313–329. Springer, 2021.

Gavin Kerrigan, Padhraic Smyth, and Mark Steyvers. Combining human predictions with model probabilities via confusion matrices and calibration. In *Advances in Neural Information Processing Systems*, 2021a.

Gavin Kerrigan, Padhraic Smyth, and Mark Steyvers. Combining human predictions with model probabilities via confusion matrices and calibration. *Advances in Neural Information Processing Systems*, 34:4421–4434, 2021b.

Andreas Kirsch, Joost van Amersfoort, and Yarin Gal. BatchBALD: Efficient and diverse batch acquisition for deep Bayesian active learning. *arXiv preprint arXiv:1906.08158*, 2019.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(Nov):2579–2605, 2008.

David JC MacKay. Information-based objective functions for active data selection. *Neural computation*, 4(4):590–604, 1992.

Hussein Mozannar and David A. Sontag. Consistent estimators for learning to defer to an expert. In *International Conference on Machine Learning*, 2020.

Georgy Noarov and Aaron Roth. Calibration for decision making: A principled approach to trustworthy ml, 2024. Blog post.

Robert Pinsler, Jonathan Gordon, Eric Nalisnick, and José Miguel Hernández-Lobato. Bayesian batch active learning as sparse subset approximation. *Advances in neural information processing systems*, 32, 2019.

John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 10(3):61–74, 1999.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Neural Information Processing Systems*, 2023.

Vikas C. Raykar, Shipeng Yu, Linda H. Zhao, Gerardo Hermosillo Valadez, Charles Florin, Luca Bogoni, and Linda Moy. Learning from crowds. *Journal of Machine Learning Research*, 11(43):1297–1322, 2010. URL `http://jmlr.org/papers/v11/raykar10a.html`.

Stephane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Geoffrey Gordon, David Dunson, and Miroslav Dudík, editors, *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635, Fort Lauderdale, FL, USA, 11–13 Apr 2011. PMLR.

Nicholas Roy and Andrew McCallum. Toward optimal active learning through sampling estimation of error reduction. In *Proceedings of the Eighteenth International Conference on Machine Learning*, pages 441–448, 2001.

Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations*, 2018.

Burr Settles. Active learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 6(1):1–114, 2012.

Glenn Shafer and Vladimir Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9(12):371–421, 2008.

Eleni Straitouri, Lequn Wang, Nastaran Okati, and Manuel Gomez Rodriguez. Provably improving expert predictions with conformal prediction. *International Conference on Machine Learning*, 2023.

Umar Syed and Robert E Schapire. A game-theoretic approach to apprenticeship learning. *Advances in neural information processing systems*, 20, 2007.

Rajeev Verma, Daniel Barrejón, and Eric Nalisnick. Learning to defer to multiple experts: Consistent surrogate losses, confidence calibration, and conformal ensembles. In *International Conference on Artificial Intelligence and Statistics*, pages 11415–11434. PMLR, 2023.

Kailas Vodrahalli, Tobias Gerstenberg, and James Y Zou. Uncalibrated models can improve human-ai collaboration. *Advances in Neural Information Processing Systems*, 35:4004–4016, 2022.

Yan Yan, Romer Rosales, Glenn Fung, Mark Schmidt, Gerardo Hermosillo, Luca Bogoni, Linda Moy, and Jennifer Dy. Modeling annotator expertise: Learning when everybody knows a bit of something. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, 2010.

Yan Yan, Glenn M Fung, Rómer Rosales, and Jennifer G Dy. Active learning from crowds. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pages 1161–1168, 2011.